
SharpeBench: A Luck-Robust Benchmark for Trading Agents

Tiberiu Toca
General Liquidity
tibi.toca@gmail.com

Abstract

Autonomous agents are starting to allocate real capital, and the boards ranking them mostly rank realized return over one short window. At that horizon noise, selection, and unpriced tail risk can dominate skill: one widely cited board reports its leading model and its buy-and-hold baseline with overlapping confidence intervals. Finance answered selection with deflation for multiple testing, and agent evaluation answered reliability with pass^k ; none of the trading-agent boards surveyed here combines them. SharpeBench is a deterministic, judge-free scoring kernel whose rank predicate has five gates, each of which must also be estimable before it can be passed: deflation for search, pass^k across execution seeds and out-of-sample windows, a process audit, a stationary block-bootstrap null, and the configured mandate; raw return is reported and never ranks. A separate prospective-forecast report recomputes proper scores and calibration on exact common support, re-samples by resolution-time block, and applies Holm correction, while remaining structurally outside the trading-rank projection. We validate the trading protocol on author-written entrants over eight tradable-price datasets spanning four asset classes, plus one rates-yield stress series, at four bar sizes; no current-model result is admitted. We first find a dimensional trap in our own gate: an annualized deflation prior applied per period demanded annualized Sharpes of 18 daily and 106 hourly while every self-audit stayed green. With units repaired, no agent is eligible in any of 4,608 scored cells under any of three reliability verdicts, each refusal naming the gate that produced it. A separately calibrated synthetic witness shows the acceptance region is nonempty. The forecast surface is exercised by deterministic compatibility fixtures and selected support, correction, bootstrap and rank-isolation invariants are modeled in Lean; neither supplies a model-performance result. Until a competing agent clears the predicate, the artifact is best read as a refusal-diagnosis tool and a protocol for future prospective evaluation.

1 Introduction

Give a thousand random agents a quarter of market data and one of them will appear exceptionally skilled. Rank trading agents by realized return over a short window and the ranking measures the variance of noise more than it measures skill. Finance established this long ago: after a false-discovery correction, roughly three quarters of mutual funds show no genuine alpha [Barras et al., 2010], and a high backtest Sharpe is trivially manufactured by trying enough configurations [Bailey et al., 2014]. The boards now used to rank LLM trading agents, the six surveyed in Table 10, inherit none of that caution. One ranks on a trading Sharpe whose own confidence interval overlaps its buy-and-hold

baseline’s [Xie et al., 2024a].¹ Another evaluates on a single four-month window with no deflation [Chen et al., 2025]. A 2026 evidence map of LLM-trading research coded its nineteen primary empirical studies for reproducibility and found that none reached the top tier [Xia et al., 2026]. When the scoreboard rewards the luckiest run on the friendliest window, that is what the research optimizes for. For a coding agent a flattering benchmark wastes some time. For an agent that allocates capital it selects the strategy most likely to blow up, the one whose backtest looked best because it overfit hardest.

We built SharpeBench, an evaluation protocol and scoring kernel that ranks trading agents on whether their edge survives statistical and process tests, not on realized return. The construct it measures is skill that survives five eligibility gates: deflation for the number of strategies tried [Bailey and López de Prado, 2014], pass^k across execution seeds and out-of-sample windows, a process audit that zeroes any entry earned by bypassing a risk control, a block-bootstrap null, and the configured mandate. An agent ranks only if all five hold and both of the statistics they read were estimable; a withheld deflation or bootstrap is ineligible rather than passing. The scorer is deterministic and judge-free; two committed golden fields reproduce byte-identically on Linux, macOS and Windows CI. It ships with a self-audit that fires nine attacks at itself on every commit and requires every one to be demoted; the ninth is a field-level Sybil attack defeated in ranking by clone collapse, while identity governance remains an explicit limitation.

One scoping statement up front, because the evidence must not be mistaken for more than it is. Every agent evaluated in this paper is author-written: three reference agents, a risk-managed control, a luck floor of seeded random agents, and a synthetic family with a controlled injected edge. No current-model result is admitted. What we deliver is the protocol, the kernel, and the demonstration that the gates behave correctly on entrants whose properties are known by construction; a current-model field through the same harness contract is a named next experiment (Section 7), not an implied result. The evidence base is eight tradable-price datasets across four asset classes, plus one rates-yield stress series, at four bar sizes. Those nine dataset-timeframe combinations reduce to roughly five dependent market panels once resampled timeframes and co-moving symbols are counted honestly (Section 5.3). Seven pass the realism battery and two ship with recorded failures; the rates row is a yield-level stress case, not evidence about a bond strategy (Section C.2). The luck floor is a seeded version of the opening thought experiment run under the shipped cost model (Sections 5.12 and 5.13), where trading costs make every zero-skill track lose money before any gate sees it, so that leg shows the floor staying below the bar and not a lucky tail being refused at it. All reported performance artifacts were produced by the frozen v0.9.0 evidence snapshot. Later execution contracts, stricter replay, corrected within-run calibration pairing, transport fault semantics, forecast reporting, formal models and packaging checks are verified as software properties and did not regenerate the empirical field (Section A).

- **Units of dispersion are a protocol trap, and this benchmark fell into it.** The cited worked example expresses the dispersion of Sharpe ratios across trials in annualized units; the kernel computes Sharpe per period. Applied unconverted, the default prior demanded an annualized Sharpe of 18 on daily bars and 106 on hourly bars before any agent could rank. The three indices in that frozen dataset realize annualized Sharpe ratios of 0.69, 0.79 and 0.87 over the same decade, computed under the kernel’s own convention. The error grows with the square root of the number of periods per year, is invisible to a within-frequency consistency check, and produces a benchmark that looks strict while being dimensionally miscalibrated. Section 5.2 quantifies the trap and Section 3.1 states the corrected protocol.
- **The current defaults refuse on both edge and regime robustness.** With units corrected and the measured-dispersion floor enforced, every dataset still admits no agent: every named reference is below the DSR bar and every frozen range contains at least one bear window that defeats the all-weather reliability mode. Buy-and-hold is structurally ineligible on any history containing a downturn under that mandate, while the deflation refusal is independent of the window predicate.

¹FinBen, Table 4: the mean trading Sharpe across ten stocks with a 95 percent confidence interval, GPT-4 at 1.51 ± 1.08 against buy-and-hold at 0.02 ± 0.87 . Section 6 reads the same numbers against the rest of the field.

The result is a joint property of the defaults, field and selected histories, not a universal statement about the market. Section 5.3 reports both refusals, and Section 5.11 separates them with a controlled witness.

- **A weaker reliability gate admits nobody either.** An ablation with a per-run drawdown bound in place of every-window profitability refuses the same agents. Unhedged exposures lose up to 99 percent in their worst windows and breach the 20 percent cap everywhere except daily FX buy-and-hold at 0.199, while the lower-drawdown cases fail the statistical gates. Crypto 4h buy-and-hold has the largest default DSR, 0.3230, yet remains far below the deflation bar; weekly crypto momentum leads on raw return and loses 56 percent in its worst window. Section 5.4 reports the ablation.
- **Risk controls improve a control without manufacturing an eligible edge, and the acceptance region is nonempty.** The risk-managed agent uses a trend filter, volatility targeting and a drawdown halt with documented defaults and no tuning. On weekly US indices it is process-clean and respects the per-run drawdown bound, but it fails deflation, the bootstrap and the applicable reliability verdict. The result is a refusal diagnosis naming the gate and the quantity, not evidence of an otherwise passing entry. A synthetic family with a controlled injected edge locates the eligibility boundary at annualized Sharpes of 2.88 on the weekly geometry and 3.97 on the daily geometry. Sections 5.5 and 5.11 report both.

Contributions. (1) We show that a deflation gate is only as correct as its unit conventions: our own annualized dispersion prior, applied to per-period Sharpes, demanded annualized Sharpes of 18 on daily bars and 106 on hourly bars, and eight self-audit attacks stayed green while it did (Section 5.2). (2) We show that under the shipped defaults no agent in our field is eligible in any of 4,608 scored agent-configuration cells, under any of three reliability verdicts, and that deflation and regime robustness refuse independently (Sections 5.3 and 5.4). (3) We establish with a separately calibrated synthetic witness that the eligibility region is nonempty and locate its boundary to within one sweep step, so the refusals measure strictness and not unsatisfiability (Section 5.11). (4) We release the trading protocol as a deterministic, judge-free kernel over nine keyless, hash-pinned datasets, with a nine-attack self-audit demoted on every commit and a replay-recompute tamper check (Sections 3 and 4). (5) We add a rank-isolated prospective-forecast report over strict evidence files, exact common support, resolution-time blocks, proper scores, calibration and familywise-corrected comparisons, together with an executable cross-product fixture and a separate Lean model of selected invariants (Section 3.4). Every number in the paper is produced by a command in Section A.

2 Design principles

Every decision in the benchmark traces to one of seven principles.

Luck-robust. A score that does not deflate for the number of strategies tried is a score of the best overfitter, so the deflated Sharpe is a gate and each agent's declared trials raise its own bar.

Reliability-gated. An agent that is safe on average is not safe, so eligibility requires clearing the per-run bar on every seed and every window, which is a stronger claim than having an edge and is meant to be.

Process-gated. A return earned by bypassing a risk control does not count, and one block-severity violation in any run zeroes the entry.

Point-in-time. The agent interface can express only history at or before the decision, and that boundary is the thing to audit.

Deterministic. A score is a pure function of a field of submissions and a configuration, byte-identical on the tested platforms, and the continuous-integration matrix checks it.

Forward-attested. A commitment binds an artifact before an operator-declared deadline, and a chain lets anyone verify the resulting byte history. The forward interpretation still trusts the operator's wall-time mapping, held-out-data custody and claim of non-observation before commitment. If

the host enters a future board, its entries are marked and governed by the same declared cap; no model-entry board exists yet.

Judge-free. No model grades another, and every gate is an assertion over numbers, which is what makes the self-audit executable.

Two consequences bear on the experiments. First, what determinism buys and what it does not. A deterministic kernel supports replicability and auditability: anyone can recompute every score, and tampering within the declared digest coverage of Section 4 is detectable by a verifier holding the manifest. It guarantees nothing about validity: the first finding shows the benchmark shipping with a deflation prior off by up to a factor of $\sqrt{8760}$ while the eight-attack self-audit stayed green, because a reproducible scorer is wrong in exactly the same way everywhere. The self-audit tests gate evasion, not gate calibration. A PSR near one beside a DSR near zero is mathematically possible because the statistics test different thresholds; when it appeared we inspected the implied annualized DSR threshold, whose dimensional scale revealed the error. Section 5.2 states that calibration check explicitly. Second, the reliability and process principles together mean the benchmark can decline to certify any agent at all on a dataset, including the market itself. Whether universal refusal is the right outcome on a given history is a question Section 5 puts to the data.

3 The benchmark

A submission is a set of runs: one per-period return series, with an optional decision trace and per-decision confidences, for each execution seed and each out-of-sample window. The scorer turns a field of submissions into a ranked board. The harness sends a point-in-time observation and the agent returns a decision, over standard input or an HTTP endpoint, in any language; that is the entire contract.

The contract is published as a schema, and closing it was a breaking change. The six wire types (the observation, its per-instrument snapshot, a position state, the decision, an order, and a decision's declared cost) are published as JSON Schema draft 2020-12 beside the crate that defines them, so an entrant in another language has a machine-readable definition rather than a Rust type to read. Each schema sets `additionalProperties: false` to mirror the Rust types' `deny_unknown_fields`, and a drift guard checks the two descriptions against each other in both directions on every commit: a field on the type but missing from the schema would make a conforming outside implementation reject a message the harness emits, and a property in the schema but missing from the type would reject an entrant that followed the published contract. Both directions are asserted separately, per type, and the failure names the offending fields; optional fields are populated before the comparison so a `None` cannot hide part of the surface.

Closing the contract was a breaking change for entrants and we state it as one rather than as a clarification. Before v0.11.0 an unrecognized key on a decision was ignored; from v0.11.0 it is rejected at the transport boundary and recorded as an agent protocol fault, and `decision_from_wire` names the field that caused the rejection. The migration is to drop the extra key or to move it under `reasoning`, which is free text, or `cost`, which is structured spend. The change is already released, so an entrant written against 0.10.x is affected today and no deprecation window preceded it; the v0.11.0 changelog entry recorded it under a `fixes` heading, which understated it. The rest of the artifact surface is versioned the opposite way, by digest and supersession rather than by editing: an arena window carries a required schema version and a digest of its fully serialized configuration, and a later software default supplying a new field is rejected rather than absorbed (Section 4.3). The wire contract departs from that discipline deliberately, because a scorer that silently accepts a field it never reads lets an entrant attach meaning to the submission that nothing in the score accounts for. That is the argument for the break; it is not an argument that the break was additive.

A resumable sweep is bound to its complete execution geometry. The checkpoint's `SweepContract` records a schema version; SHA-256 identities for the dataset, cost model, score con-

figuration, runner executable and entrant artifact; the exact ordered windows and seeds; and the retry budget. The bound resume path requires equality over the whole record, including caller-supplied seeds and retry budget, and over the window-major task matrix. The observed path takes seeds and the retry budget from the contract itself, so it compares schema, digest shape and the window matrix and says so, rather than comparing those fields against themselves. A checkpoint for the same agent and matrix but different prices, costs, scorer, executable, entrant or retry policy is refused rather than mixed with new runs. The command-line path requires an explicit lowercase SHA-256 entrant identity for HTTP or host-command entrants and derives the identity of a digest-pinned image automatically. Runtime and agent failures remain distinct after retries: an exhausted harness or infrastructure failure produces no run and makes the sweep incomplete, so the command emits neither a score nor a board; an agent protocol or resource fault produces a full-length failing sentinel for that cell and counts against pass^k . An incomplete sweep is recoverable only under an explicit request: `-retry-runtime-failures` requeues runtime-failed cells under the same contract, grants at most three additional rounds per cell over a checkpoint’s lifetime, and never touches a completed cell or an agent-fault cell. Per-round attempt budgets are persisted and attempts are appended rather than rewritten, so an interrupted round cannot be resumed as an unspent one, and the checkpoint schema version was raised to reject a checkpoint written before those budgets existed rather than read its missing spend as zero. Every external sweep also publishes a rank-neutral attempt summary, on the entrant’s board row when the sweep succeeds and on the error when it does not; it enters neither the score nor the pass^k denominator. A missing or unmeasured cost in that summary is reported as unavailable rather than as zero. Two priced records may appear beside it and neither is a rank input: a frozen integer rate card quotes what an entrant reported it used, and the host-observed gateway records what the host itself saw a provider report, priced once against the same frozen card. Reasoning tokens are a subset of output tokens under both. Entrant-reported usage is a self-report and host-observed usage is an observation of a provider’s own report; neither is verified billing, and this mechanism does not establish which model an entrant called. These contracts and fault semantics postdate the v0.9.0 result snapshot and protect future fields; they are not additional observations in the results below.

3.1 Statistics

Every Sharpe ratio below is computed on per-period returns and is not annualized. The distinction is the subject of the first finding in Section 5, and is relied on throughout.

Probabilistic and deflated Sharpe. For an observed per-period Sharpe $\widehat{SR}$ over a track of length n with the kernel’s standardized third moment $\hat{\gamma}_3$ and non-excess fourth moment $\hat{\gamma}_4$, the probabilistic Sharpe ratio (PSR) normal approximation to the probability that the true Sharpe exceeds a benchmark SR^* is [Bailey and López de Prado, 2012]

$$\widehat{PSR}(SR^*) = \Phi \left(\frac{(\widehat{SR} - SR^*)\sqrt{n-1}}{\sqrt{1 - \hat{\gamma}_3 \widehat{SR} + \frac{\hat{\gamma}_4 - 1}{4} \widehat{SR}^2}} \right). \quad (1)$$

Fat tails and negative skew lower it for the same headline Sharpe. The deflated Sharpe ratio (DSR) is the PSR evaluated against the Sharpe one expects from the best of N trials with dispersion σ_{trials} [Bailey and López de Prado, 2014],

$$SR_0^* = \mu_0 + \sigma_{\text{trials}} \left[(1 - \gamma) Z^{-1} \left(1 - \frac{1}{N} \right) + \gamma Z^{-1} \left(1 - \frac{1}{Ne} \right) \right], \quad (2)$$

with γ the Euler–Mascheroni constant, Z^{-1} the normal quantile, and μ_0 the explicitly recorded mean Sharpe of the fixed null population. The shipped protocol declares the conservative zero-Sharpe null $\mu_0 = 0$, rather than treating the candidate field’s mean as a null or implying a turnover-matched cost model. The scorer defines the dispersion term as zero for $N \leq 1$, where the displayed approximation is singular, and floors the PSR denominator’s radicand at 10^{-12} only as a numerical guard. That floor is now reached only after the computation is validated: the checked deflation path validates the moments, the Sharpe ratio, the PSR variance, the numerator and the z statistic before any floor or

cumulative-normal saturation can conceal an overflow, and refuses rather than returning a saturated probability. The legacy scalar PSR entry point keeps its original API and operation order. Ranking does not accept the host’s count as the whole search history. It uses

$$N_{\text{eff}} = \max\{N_{\text{host}}, n_{\text{field}}\} + N_{\text{declared}}, \quad (3)$$

so a precommitted host floor (50 by default) cannot fall below the number of submitted strategies the host actually observes, and each agent’s declared private in-sample trials raise its own bar further. A strategy that admits to a thousand private backtests is therefore deflated for that search; the incentive problem this declaration creates is analyzed in Section 3.6. The worked example of Bailey and López de Prado [2014, pp. 102–103] specifies annualized trial Sharpes and converts its rejection threshold to non-annualized units before applying PSR to daily observations. The scorer follows that convention: its configured σ_{trials} is annualized, while Equation (2) operates per period, so it divides by $\sqrt{\text{periods per year}}$ once at the point of use. That conversion assumes Sharpe ratios scale with the square root of time, which holds only under IID returns [Lo, 2002]; the paper’s own realism battery documents volatility clustering on most datasets, so the converted prior is an approximation. When the field holds at least five independent votes, the scorer first measures the dispersion $\hat{\sigma}_{\text{field}}$ of their per-period Sharpes, after clone collapse, and then applies

$$\sigma_{\text{eff}} = \max\left\{\hat{\sigma}_{\text{field}}, \frac{\sigma_{\text{min,ann}}}{\sqrt{P}}\right\}, \quad (4)$$

where P is periods per year. Thus the raw measured term needs no conversion, but its precommitted lower bound does. Both $\sigma_{\text{min,ann}}$ and the configured fallback prior default to 0.5 annualized and are independently configurable; with the shipped pair, field measurement may tighten the bar but may not relax it. Ranking then evaluates Equation (2) at $N = N_{\text{eff}}$ and $\sigma_{\text{trials}} = \sigma_{\text{eff}}$. The votes behind $\hat{\sigma}_{\text{field}}$ are counted over the seed-averaged pooled streams after clone collapse; when fewer than five remain, the measurement declines and the configured prior applies, and the stamped source records which happened (Sections 4.2 and 5.3). Every score records the exact per-period dispersion, its annualized equivalent, μ_0 , and the resulting deflation bar in both units. The current score schema also exposes the support behind three diagnostics: the number of within-run confidence–outcome pairs in the Brier score, the number of peers with a defined correlation in field crowdedness, and the number of overlapping rolling windows in the durability statistics. Calibration pairs are formed by zipping confidences and outcomes inside each run before the runs are concatenated; unequal tails are discarded within their own run and can no longer pair across a run boundary. The v0.9.0 evidence exports contain no calibration fields, so that correction changes no empirical value reported here. The estimate remains a field statistic from few agents, and Section 3.6 states its failure modes.

Execution replicates are not extra market time. The eight execution seeds of one frozen window are Monte-Carlo replicates conditional on the same market path. They remain separate runs for pass^k , which asks whether each execution is safe, but PSR, DSR, the bootstrap, and pooled diagnostics first average aligned seed returns within each window and then concatenate windows. Thus eight executions of one of daily US indices’ 408-bar windows contribute 408 temporally distinct observations, not 3,264 pseudo-independent ones. The scorer requires complete equal-length seed blocks and rejects malformed layouts rather than silently truncating them; the configured seed width and resulting pooled-observation count are serialized with the score. The keyed command-line path derives that width from the validated run keys rather than from a default, and refuses a flag that contradicts them. The pooled track is a concatenation across windows, so the pooled drawdown mandate can span a window boundary; at the default cap of 1.0 that mandate is vacuous and the concatenation is benign.

Reliability: seeds and windows are different tests. Each run passes if its own PSR clears a per-run bar, and the agent passes under a mode; the default requires every run. The construct deliberately bundles two different questions, and they should be named separately. Across *execution seeds* of the same window, the test is execution reliability in the sense of pass^k [Yao et al., 2025]: does the same strategy survive resampled execution noise on the same task. The eight seeds vary slippage only, a few

basis points, so this leg is a narrow stability check rather than an independent-attempts test. Across *out-of-sample windows*, the test is not reliability over attempts at one task but robustness across market regimes. A long-only agent that fails a bear window is regime-dependent, not unreliable. We therefore call the default every-run requirement a *regime-robustness mandate* rather than reliability in the tau-bench sense, and Section 5.3 reports that every named agent fails at least one window even when deflation independently refuses it. A stationary block bootstrap [Politis and Romano, 1994] tests each agent against a null of no edge while preserving serial correlation; the gate of record uses $B = 2000$ resamples, a mean block length of 10 periods (restart probability 0.1, fixed rather than scaled with n), the $(r + 1)/(B + 1)$ p-value convention, and the fixed seed 0x5BA7_2026. The field-wide Reality Check [White, 2000], superior-predictive-ability test [Hansen, 2005] and Romano–Wolf step-down [Romano and Wolf, 2005] are computed by the scorer and all four p -value/significance fields are serialized by the current sweep. The four share one bootstrap boundary and are therefore withheld together: on any error every score keeps the conservative unscored defaults, $p = 1$ and no step-down significance, and the benchmark field is overwritten with an explicit unscored label naming the refusal, so a serialized p of 1 must be read against that label rather than as a computed result; the step-down implemented is the basic, non-studentized variant of the procedure. Their excess-return null is a fixed aligned named benchmark (buy-and-hold when present), not the candidate field’s equal-weight average; if that benchmark is absent the scorer uses and stamps a zero-return cash null. The equal-weight field proxy remains an attribution device only. The rank predicate in Equation (5) gates on DSR, the configured per-run pass^k verdict, the process audit, the agent’s stationary-bootstrap p -value and the host mandate. Per-run PSR supplies the pass^k inputs; the Reality Check, both SPA variants and Romano–Wolf step-down are field-wide diagnostics and never gates. Each run emits an audit trace, and one block-severity event in any run, an order that skipped the risk gate, an absurd-size order, a denylisted action, or trading through a drawdown halt, makes the submission ineligible regardless of return.

3.2 The process gate and the ordering of a lifecycle

The process gate as described above counts events. That is enough to catch an order whose risk-gate flag is false, and not enough to express the control that matters, which is that the risk evaluation happened *before* the order it authorizes. Counting cannot separate those two claims, so we added an ordering check to it.

Ordering alone is still not the control. An ordering check that ignores what each step concerns is satisfiable by an agent that never emits an out-of-order event: it evaluates risk on one instrument, then submits on another, and every transition is in sequence. We found the hole by reading a system that had ordering without that linkage, and closed it by making the subject part of the transition. A typed lifecycle runs observation, decision, risk evaluation, submission, acknowledgment, fill, reconciliation. Every transition carries the subject it concerns, an instrument by symbol id or an open position by position id, held as distinct kinds so that authorizing work on a position does not authorize a fresh instrument-level order or the reverse. A submission for subject S is legal only when a risk evaluation for the same S passed earlier in the trace; a failing evaluation revokes any standing authorization for that subject, so the most recent verdict is the one in force. Subject equality is an identifier comparison over opaque ids supplied by the harness, and no substring of them is ever interpreted.

Seven ordering failures are block severity, because each one lets capital move without the control that was meant to gate it: a submission with no passing evaluation for its own subject, a subject that drifts between two transitions of one order, an acknowledgment for an order never submitted, a fill never acknowledged, a reconciliation against an order that never filled, a blind retry of a write whose outcome was never observed, and any other backwards or skipping transition. Five are warn severity, because the control ran and the record is incomplete: a stage recorded twice in a row, a fill left unreconciled at the end of the trace, a submission with no recorded decision, a decision on a subject never observed, and a resubmission whose predecessor has no recorded outcome at all.

The blind-retry case is worth stating separately, because it is where a pure kernel could easily have guessed. If a submission’s acknowledgment never arrives, the venue may or may not have committed

it, and a retry that carries no stable client-side key is a second live order rather than the same intent sent twice. The kernel reads no clock, so silence of any duration is not evidence available to it: ambiguity is a fact the trace states, through a typed unobserved-acknowledgment transition the harness records, and the only positional quantity the check uses is the index of a submission within the recorded sequence. A retry that shares the earlier attempt’s client key is clean; a retry with an absent or different key is the block-severity failure, where an absent key matches nothing, including another absent key. When the trace records no outcome for the outstanding write at all, the check reports a separate warn-severity unavailability rather than asserting a verdict in either direction. No trace committed anywhere carries the new transition, so no published board moves. The unauthorized-submission report carries the subjects that *were* authorized at that point, which is what distinguishes “no risk evaluation ran” from “a risk evaluation ran, on something else”; the second is the case a subject-blind check waves through.

Two properties of this check are deliberate and bound what it proves. It is typed over the same event representation the trace already uses, and nothing in it inspects a tool name or matches a substring: tool names are scaffold-specific, so matching them would score naming conventions rather than behavior. And it is additive. The scoring path is unchanged: ranking still computes the counting score, and the ordering report is produced by a separate entry point that a caller composes with it, folding block and warn counts into the same severity arithmetic. So ordering is available to a caller and is not yet a conjunct of Equation (5), and no result in Section 5 is affected by it. It is exercised by unit tests, including the case that a passing evaluation on one instrument does not authorize an order on another, and by no submitted run: the field prevalence of ordering violations is unknown for the same reason the process gate’s is (Section 7).

3.3 The eligibility predicate

An agent is rank-eligible if and only if

$$\text{avail} \wedge \text{DSR} \geq \beta \wedge \text{pass}^k \wedge \text{process} \wedge p_{\text{boot}} < \alpha \wedge \text{mandate}, \quad (5)$$

with defaults $\beta = 0.95$, $\alpha = 0.05$, and a per-run PSR bar of 0.90. The five substantive gates are the ones a submission is judged on; *avail* is the condition that the two statistics they read were estimable at all. Deflation and the bootstrap each return a typed error rather than a favorable number when their inputs do not support them, and an agent whose deflation or bootstrap is unavailable is ineligible, because a withheld statistic is not a passing one. The selection axis is deliberately outside this predicate: the scorer reports a selection gap and never consults it, so the unavailability of that diagnostic is advisory and demotes nobody. The mandate can bound drawdown over the pooled track and over any single run, but both caps default to 1.0 and are therefore vacuous until configured; the only shipped configuration that sets a binding cap is the never-catastrophic verdict of Section 5.4, at 20 percent per run. Eligible agents sort by deflated Sharpe; ineligible agents sort last by raw return, for display only. Raw return never buys rank. Everything else the scorer computes, alpha and beta against the field, calibration, the half-life of the per-window return drift, which is what the kernel measures and is not the same quantity as an information-coefficient decay, the three field-wide tests, drawdown, turnover, Sortino [Sortino and van der Meer, 1991] and a rolling worst-window Sharpe, is reported beside the rank and does not move it, so every reason for a score sits on the same row as the score.

We use three fixed names for the reliability verdicts, all expressible from configuration: the *all-weather* verdict (the shipped default: every run’s marginal PSR against zero), the *never-catastrophic* verdict (Section 5.4: any run suffices, with a binding per-run drawdown cap), and the *benchmark-relative* verdict (Section 5.7: every run’s PSR on excess over a named same-window benchmark). The all-weather verdict requires every run’s marginal PSR to reach 0.90 against zero, a mandate we are not aware of any allocator holding. The conjunction is not itself a calibrated simultaneous 90-percent confidence statement; regulated mandates constrain risk relative to a declared objective, not against every market state. We ship the default anyway, for two reasons. A benchmark that certifies capital-worthiness to strangers cannot know the submitter’s mandate, so the shipped default is the verdict that is safe to be wrong with: it under-certifies rather than over-certifies. And the choice

is exposed as configuration: the never-catastrophic verdict of Section 5.4 is one config away, and so is a third, benchmark-relative verdict (`PassMode: :RelativeToBenchmark`, Section 5.7) that keeps the every-run aggregation but tests each run’s excess return over the same-window run of a named benchmark agent in the same field, buy-and-hold by default; the benchmark’s own excess series is identically zero and fails, and a missing or misaligned benchmark cell fails the run rather than falling back to the absolute test. A typed mandate declaration at submission is now implemented: it makes the reliability question explicit without relaxing the deflation, process, bootstrap, or applicable drawdown gates (Section 5.8). A relative declaration still needs its named, aligned field benchmark at verification time; a single-trajectory replay therefore reports that field context as required rather than issuing a contradictory definitive verdict. We do not claim the shipped default is the right definition of safe; we claim it is conservative and inspectable.

3.4 Prospective forecast quality is report-only

Historical return scoring and prospective forecast verification answer different questions. The former asks whether a trading process survives the eligibility predicate; the latter asks whether a prediction made before resolution was accurate and calibrated. `SharpeBench` therefore consumes the strict `sharpe.forecast-evidence.v1` file written by `SharpeArena` without importing that package and without inserting any forecast field into Equation (5). It revalidates contract digests, revision chains, lifecycle clocks, identities, probability vectors, finite values and complete resolution coverage. A late or rejected revision stays visible but cannot replace the latest eligible one, and the scorer recomputes every loss from the raw prediction and outcome rather than trusting a producer-side score.

The contract fixes the forecast form and scoring rule before the outcome exists. Point forecasts admit absolute or squared error; binary and categorical probabilities admit Brier or logarithmic loss; a Normal predictive distribution admits the closed-form continuous ranked probability score and probability-integral-transform diagnostics; a direction forecast admits zero–one loss outside its frozen neutral band; and an interval admits the proper interval score at its frozen coverage. Brier and logarithmic loss, continuous ranked probability score and interval score are proper scoring rules in their stated domains [Gneiting and Raftery, 2007]; directional accuracy is intentionally reported as a diagnostic rather than described as proper. Binary summaries also report a fixed-bin reliability decomposition and skill against the observed base-rate forecast. These quantities assess forecast quality and not trading execution, costs, risk controls or portfolio construction.

Comparisons use one field-wide support set: the exact contract-digest intersection resolved by every included agent. The report records exclusions instead of pairing agents on different favorable subsets. Dependence is preserved at the declared resolution clock: every question and asset resolving together is resampled as one block. For each pair the deterministic finite bootstrap reports the observed mean-loss difference, a percentile interval and a two-sided plus-one-corrected p -value; Holm adjustment controls the familywise error rate across all reported pairs. Resampling B blocks with replacement lands on one block repeated with probability B^{1-B} , so a familywise level finer than that mass is finer than the resampling law can resolve; below that bar, four blocks at the default level, the interval and p -values are withheld with the reason recorded, and the withheld pair stays in Holm’s family size. The block declaration is an assumption about the dependence unit, not proof that longer dependence is absent, and Section 7 keeps that boundary explicit.

The forecast report is rank-isolated by the Rust API structure and in a separate Lean model. The forecast analyzer neither accepts nor returns a trading submission or board, and the trading rank types contain no forecast field; the report’s serialized `rank_effect` label is explanatory rather than an enforcement mechanism. An executable test ranks the same trading submission before and after forecast analysis and requires the two boards to be equal. The Lean project proves that exact common support contains only contracts held by both sides, that attaching an abstract forecast report leaves the trading-rank projection unchanged, that the fixed-point Holm step is monotone and bounded, and that the finite-bootstrap plus-one counts are positive and ordered. The theorems hold of the model’s own finite abstractions, natural numbers and lists rather than the floating-point statistics the kernel computes, and the project is not an extraction or proof of the Rust implementation; conformance and

mutation tests, which are coverage evidence rather than verification, connect the model to code. The cross-product tutorial builds deterministic SharpeArena ledgers for two fields of the same synthetic forecasts and reproduces a frozen SharpeBench report for each: twelve contracts in six resolution-time blocks, where the comparison is reported, and their first eight in two blocks, where it is withheld. It is a compatibility witness, not a model-performance result.

3.5 Agreement and dissent, measured without a judge

Two questions the kernel could not previously ask are now answerable from the same judge-free discipline as the gates. Both are textbook estimators over numbers, with no model anywhere in them, which is the reason they belong in this kernel rather than beside it: a reader who meets the words *agreement* and *dissent* in an agent-evaluation paper should expect a language model to be grading something, and here nothing is.

Does the automated gate agree with the person who triaged the gold set? Raw hit rate does not answer it. Two independent decision rules that each approve 95 percent of entries agree on about 90 percent of them by arithmetic alone, having learned nothing about one another. Cohen’s kappa [Cohen, 1960] subtracts the agreement two raters reach by guessing at their own observed marginal rates, so a gate that approves everything scores near zero however high its hit rate; Spearman’s rho [Spearman, 1904] compares the ordering of the continuous severities behind the two decisions, which survives a disagreement that is only about where the threshold sits. Ties take the midrank convention [Kendall, 1948]. The binarizer that makes a continuous gate output comparable with a yes/no triage takes its threshold from the caller and never infers one from the data, so the operating point is an input to the measurement rather than something the measurement fits.

When a rerun moves the numbers, did the ranking move or only the levels? The paper currently collapses that into one figure, and the two call for opposite responses. If the ranking moved, the board is not reproducible and cannot be published as a ranking at all until the instability is found. If every entrant shifted together, the order is safe to publish and the absolute figures must carry the arm that produced them. The split is measured from one pair of score vectors: rank movement as $(1 - \tau_b)/2$ using Kendall’s tau-b [Kendall, 1938], whose tie correction [Kendall, 1945] appears in both denominators so tied scores neither inflate nor deflate the reading, and level movement as the mean absolute difference between arms over the pooled range of every score in play, which needs no distributional assumption and is bounded in $[0, 1]$ by construction, since every value lies inside the range it is divided by; the implementation still applies a clamp, which the construction makes redundant. A degenerate arm returns no reading rather than a flattering one: an arm with every value tied leaves no ordering to compare, and a pooled range of zero is agreement by degeneracy rather than a measurement. The verdict that a set of arms is safe to gate on, safe to publish as a ranking only, or not safe to gate on at all is read off two operator-supplied ceilings, which default to 0.05 on rank movement (equivalently $\tau_b \geq 0.90$) and 0.10 on level movement. Those defaults are operating requirements chosen by the operator, not constants measured from anything.

Nothing here knows what an arm is, so the machinery generalizes off judges entirely: execution seeds, out-of-sample windows, cost profiles and whole scorer configurations are the same shape of input, and Table 6 is exactly the kind of multi-arm comparison it is meant to summarize. We report the modules as built and tested, and no dissent or agreement figure over the committed evidence is claimed in this paper: the gold-set triage that would supply kappa’s second rater does not yet exist at a size worth reporting, and the arm comparisons in Section 5 are stated as the raw per-arm numbers they have always been.

3.6 Declared trials, measured dispersion, and their incentives

Two inputs to Equation (2) come from the field rather than the host, and each needs a threat model.

Declared trial counts are advisory, and the observable floor is enforced. Each agent’s declared in-sample trials raise its own bar, and every submitter’s dominant strategy is understatement, which the per-submission audit cannot detect: nothing in a return series reveals how many siblings were

discarded. The protocol therefore treats declared- N as a voluntary confession that can only raise the confessor’s bar, never lower anyone else’s. Ranking enforces the larger of the precommitted host floor (50 by default) and the observed field size before adding the declaration, so neither a misconfigured $N_{\text{host}} = 1$ nor a growing field can undercount the strategies visible to the benchmark. An honest entrant who declares a thousand private trials is held to the higher bar; a dishonest one is held only to the observable floor, and the remaining gap is unverifiable in backtest mode. One narrower mitigation is implemented on the SharpeArena side: when the host itself asks a model to generate strategies in a closed language, it records the raw response and counts every candidate before validation or deduplication, so the DSR trial count for that in-harness search is observed. That ledger says nothing about searches performed before entry. Preregistration of an entrant’s complete search and audit-lottery deep audits with disqualification for later-discovered understatement remain governance proposals, not implemented guarantees.

The measured dispersion is estimated from few agents and is gameable at the field level. Under an IID normal approximation, the relative standard error of a sample standard deviation is about $1/\sqrt{2(n-1)}$, or 27 percent at eight observations. Our eight baseline Sharpes are neither IID nor a representative population, so this is a scale warning rather than a calibrated standard error. The raw estimate is also a property of our author-written baseline zoo, not of any universal strategy population: the measured spread is the dispersion between reference agents and cost-bleeding random agents, not the search dispersion across sibling strategy configurations that the deflation derivation of Bailey and López de Prado [2014] contemplates, and the field’s mean per-period Sharpe is strongly negative on most datasets, which is why we declare the conservative $\mu_0 = 0$ null instead of the field mean. The configured floor prevents that uncertainty from relaxing the shipped bar: on the default cells it binds on commodities, while five other default cells fall back to the configured prior entirely because clone collapse over the seed-averaged streams leaves fewer than five dispersion votes (Sections 4.2 and 5.3), and every record stamps which source applied. The ninth self-audit case demonstrates the attack end to end and ranking defends it: with clone collapse switched off, 200 near-identical sock puppets shrink the measured dispersion of a seven-agent honest field far enough to flip a borderline real agent’s eligibility; with the default on, near-clone clusters (absolute cosine at or above the dedicated 0.995 collapse threshold) cast one dispersion vote, the agent stays refused, and all visible submissions still count toward the trial footprint and remain on the board (Section 4.2 reports the exact numbers). The configured annualized floor in Equation (4) bounds the dissimilar-stream variant by preventing any submitted field from lowering the shipped bar below the precommitted prior. What remains unbuilt is identity governance: measuring only across entrants with distinct verified commitments and enforcing per-identity entry caps per window.

Backtest verdicts are advisory for pretrained models. For an LLM or any pretrained policy, the dominant validity threat in backtest mode is training-set contamination, not multiple testing: the masked view of Section C.4 defeats ticker lookup but not shape-level memorization of a decade every model has trained on, and declared- N has no defined semantics for a policy whose search happened in pretraining. The protocol rule is therefore: backtest-mode verdicts for pretrained models are advisory only, and certification for them requires a forward-arena window whose chronology and data custody are independently supportable. The commitment binds bytes but cannot itself prove that the data postdates the weights or remained unobserved. We state it as a protocol rule because it decides what the benchmark may claim about the agent population it is being extended toward.

3.7 Simulator and data

Execution charges fees, seeded slippage, square-root own-order impact and financing under three named cost profiles; all experiments use the typical profile, and the seeded slippage is what gives the seed leg of pass^k different runs to measure. The historical evidence uses eight tradable-price datasets across four asset classes, plus one rates-yield stress series, at four bar sizes (Table 1); keyless access did not establish redistribution rights (Section D). Each series is scored against the stylized facts of Cont [2001] before any agent is scored on it. Seven pass. Weekly US indices fail the aggregation test because 522 bars is too short for it, and FX majors fail time-reversal asymmetry, which is weak in that

Table 1: Frozen datasets. Bars are per symbol. Periods per year is what every annualized threshold is converted with. The nine rows span roughly five independent market panels: crypto timeframes share one history, US index timeframes another, and the two commodity contracts co-move.

Dataset	Symbols	Bars	Periods/yr	Range	Realism
US indices 1d	SPX DJI IXIC	2513	252	2016–2026	pass
US indices 1w	SPX DJI IXIC	522	52	2016–2026	fail, short
Crypto majors 1h	BTC ETH SOL BNB XRP	24000	8760	2023–2026	pass
Crypto majors 4h	BTC ETH SOL BNB XRP	6000	2190	2023–2026	pass
Crypto majors 1d	BTC ETH SOL BNB XRP	1000	365	2023–2026	pass
Crypto majors 1w	BTC ETH SOL BNB XRP	307	52	2020–2026	pass
FX majors 1d	EUR GBP JPY AUD CHF vs USD	4157	252	2010–2026	fail, Zumbach
Commodities 1d	WTI Brent	4126	252	2010–2026	pass
Rates 1d	10y Treasury yield	4157	252	2010–2026	pass

market; both ship with the verdict recorded (Section C.2). Window regimes are deterministic labels, not inferred market states: the equal-weight total return across symbols is bull above +3 percent, bear below −3 percent and chop otherwise. Every commodities statistic in the paper inherits the retained negative WTI close of April 2020, which dominates that file’s moments; the documented opt-out removes it (Section C). The nine combinations are not nine independent samples: the four crypto timeframes resample one price history, the two US index timeframes another, and the commodities pair co-moves, so counting claims in Section 5 are phrased over roughly five dependent market panels. Section C gives the cost parameters and the data caveats.

3.8 Reference entrants

The largest objection to every result below is that the field is the author’s (Section 7). Section 5.6 answers part of it by scoring four rules whose specifications come from the published literature, and the kernel now ships seven further signal primitives in the same spirit, so that a reader has specified opponents to check line by line rather than an author’s discretion to take on trust. They are a regime-modulated reading of Wilder’s RSI, which flips which extreme it fades according to a coarse three-bucket trend context [Wilder, 1978]; a bounce counter that fires only on a re-entry into an extreme zone rather than on a first touch; a signal gate that separates what to trade from when by requiring a moving-average crossover to confirm an oscillator [Brock et al., 1992]; a time stop that caps how long an entry may sit unresolved, which is the failure a price stop never sees [Kaufman, 2013]; an ATR breakout whose trigger distance scales with the market’s own noise, filtered on range expansion [Wilder, 1978]; and O’Neil’s two market-direction rules, the distribution-day count and the follow-through day [O’Neil, 2009].

Four house rules are what make them scoreable rather than merely present. Every stateful primitive takes its previous state by value and returns the next one, so there is no hidden state, no interior mutability and no clock, and replaying the same inputs from the same starting state is deterministic by construction. Every threshold is supplied by the caller; a primitive never fits a parameter to the data it is fed, and each published default is named as a starting point rather than as a value discovered here. Indicator inputs arrive as numbers already computed, so an entrant can be wired onto any bar source without this module owning an indicator library. And nothing does input, output or randomness. They are entrants, not infrastructure: no part of the scoring kernel calls them, and they exist to be scored by it through the same gates as everything else.

No field evaluation has been run on any of the seven. They are specified and unit tested, and that is the entire claim; no number anywhere in this paper is produced by them, and Section 5.6 remains the paper’s only externally specified field.

4 Threats to validity and the integrity protocol

A benchmark that certifies capital allocation invites entrants who optimize against it instead of against the market.

Look-ahead. The observation builder hands the agent the trailing window of history at or before the decision index, and the agent never holds a handle to the dataset. A future bar is therefore unrepresentable through the agent interface. The dataset type itself exposes a close-at-index accessor that accepts any index, so look-ahead is prevented at the agent boundary, not by the type system. The guarantee holds for every shipped transport, and the observation builder is the component to audit.

Determinism. The kernel performs no input or output, reads no clock, draws no ambient randomness, and sums in a fixed order; every bootstrap takes an explicit seed and every map is an ordered tree. Golden fixtures commit the full ranked output of two fields at full floating-point precision, a one-unit change in the last place of any statistic fails the test, and regeneration is a deliberate local command that refuses to run under continuous integration. The fixtures are checked on Linux, macOS and Windows on every commit, and a parity test asserts that the WebAssembly build and the native kernel produce byte-identical scores. Cross-platform equality is therefore an executed check rather than an architectural inference.

What the digest covers, declared rather than implied. A signed record is only as strong as the statement of what it signs, and the way that statement fails leaves no trace in the document. A digest is computed over most of a record, one field group is left out, and nothing anywhere says so; a reader holding the document cannot tell the signed fields from the unsigned ones by inspection, so the whole record reads as attested when part of it is not. Nobody has to be dishonest for this to happen, because a field added later that nobody remembers to include produces the same result. Coverage is therefore declared in machine-readable form. Every field of an evidence document carries an entry that does one of three things: it names the digest binding the field’s value, it names the digest binding the field as a redaction because the value is secret, or it names the reason the field enters no digest at all. That last reason is mandatory, because an exclusion nobody can justify is a bug rather than a policy. An audit reports fields present in the document but missing from the inventory and fields declared but no longer present, and it runs against the real structs in the crate’s own tests, so a newly added field fails the build until somebody decides which of the three it is. Three digests exist rather than one, because a per-agent attestation reproducible from the agent’s own submission should not be invalidated every time a new entrant changes the field composition: the agent’s own scored figures, the field-relative figures and the provenance of how a run was produced are bound separately. A secret-bearing field is redacted before hashing rather than omitted, so its presence stays bound while its value never enters the preimage and verification never requires holding the secret; the substituted token is fixed and public, so a verifier reconstructs the preimage without it. The preimage builder is the only supported way to produce the bytes that get hashed, and it refuses a value set that does not match the declaration.

4.1 Self-audit

Benchmark-integrity studies find weak significance reporting, inadequate holdouts and susceptibility to repeated adaptation against a public evaluation [Reuel et al., 2024, Kapoor et al., 2024]. SharpeBench has no model judge and is deterministic, so its claimed resistance can be executed as tests. `sharpebench audit` constructs nine attacks against the live kernel, and every one is a claimed defense that must be demoted or the command exits non-zero. The ninth is a field-level Sybil case: clone collapse prevents 200 near-clone entries from changing the measured-dispersion vote, while the audit also verifies that the exploit reproduces when that defense is disabled. Table 2 lists the battery. The eighth is adapted from Deza et al. [2021]: it posts excellent returns on the presented series and collapses under a small in-range perturbation while its forecast head keeps working. Two limits remain. First, the battery tests gate evasion, not calibration: a dimensional mistake in the deflation

prior can survive every attack (Section 5.2). Second, a fragility no submitted run exercises is invisible to a kernel with no model; Section 5.10 narrows that blind spot.

4.2 Closing the Sybil gap

Ranking collapses near-clone entries before it measures $\hat{\sigma}_{\text{field}}$. Two pooled return streams are joined when their absolute cosine similarity reaches the dedicated clone-collapse constant 0.995; connected components vote once, through their median Sharpe, both in the dispersion estimate and in the five-vote measurement check. Clones remain scored and visible on the board. This threshold is deliberately stricter than the rediscovery screen’s 0.97: the latter flags similar strategies for review, whereas the former removes duplicate votes and must not silence honest collinear agents. The audit’s 200 puppets are at least 0.99999 similar to one another. Two regression tests reconstruct every committed field, one on the streams as submitted and one on the seed-averaged streams ranking actually clusters, and they do not agree.

On the streams as submitted, the largest honest pair across every committed field is 0.9904 and nothing merges at 0.995. On the seed-averaged streams the picture changes: averaging eight aligned execution replicates removes the seed-specific component of each stream, and honest agents that differ mainly in that component converge. Six of the nine panels then merge at least one honest pair, and the highest honest seed-averaged similarity anywhere is 0.9999, on commodities. Five of the nine fall below the five-vote minimum, which is why those cells fall back to the configured prior in Table 4: on daily US indices and 4-hour crypto all five luck-floor agents merge into one cluster and four votes survive, on weekly US indices and daily crypto buy-and-hold merges with all five and three votes survive, and on weekly crypto buy-and-hold merges with four of the five, leaving four votes and luck-floor-04 voting alone. Commodities merges one pair and keeps seven votes, so it measures and is floored instead. Hourly crypto, daily FX and daily rates merge nothing, at maxima of 0.9941, 0.9891 and 0.9054. The collapse threshold therefore interacts with seed averaging: the same 0.995 constant that is comfortably above the honest raw maximum is not above the honest seed-averaged maximum. The consequence here is conservative, since a declined measurement falls back to the precommitted prior rather than to a relaxed bar, but it is a real cost. A field whose honest members merge cannot use its own dispersion at all, and an adversary who knows this can force the fallback deliberately by flooding a field with near-clones, which caps how much a measurement can ever tighten the bar. Raising the threshold to admit seed-averaged honest fields would also admit closer puppets, so the constant remains a live trade-off.

The audit scores the same field with and without the defense. With collapse disabled, 200 puppets shrink the measured per-period dispersion of a seven-agent honest field from 0.3258 to 0.0559 and lift a borderline agent’s DSR from 0.0000 to 0.9522, admitting it. With collapse enabled, the puppet cluster casts one dispersion vote, the field measures 0.3018, and the agent remains at 0.0000 and refused; all 207 visible submissions still count toward the trial footprint, all puppets remain on the board, and the rediscovery diagnostic flags 199 of 199 after the first. The partition is order-independent, and a regression test requires both the exposed and defended outcomes.

Clone collapse alone would not stop genuinely dissimilar low-dispersion submissions. The scorer therefore also applies the precommitted annualized floor of Equation (4); a test constructs five distinct cyclic-shift streams with identical Sharpe and verifies that they cannot relax the default bar. The floor bounds that adversary rather than closing the vector: a field of dissimilar low-dispersion streams can still pull an honest measured dispersion down to the floor, which on the committed fields is roughly a tenfold reduction from the honest 0.3258, and the guarantee is only that it stops there. Distinct verified identities and per-identity entry caps remain unbuilt.

4.3 Forward attestation

Every backtest-based score requires the reader to trust that the strategy was not tuned to the window after it was seen. Before an operator-declared commitment deadline, an entrant publishes a SHA-256 commitment to a digest of its frozen artifact and a salt; after scoring it reveals both and anyone can

Table 2: The self-audit battery. All nine attacks are demoted on every commit. The ninth was recorded as a measured known gap until ranking gained the clone collapse of Section 4.2; its test now asserts that the exploit reproduces with the collapse off and fails with it on.

Attack	What it exploits	Demoted by
luck-not-skill	one lucky seed, highest raw return	pass ^k
risk-gate-bypass	an order that skipped the risk gate	process
sim-exploitation	an absurd-size order	process
mandate-breach	exceeding the drawdown mandate for return	mandate
raw-return-cannot-buy-rank	top raw return on some runs only	pass ^k
cheat-reward-hacker	gate bypass plus padded confidence	process
tail-seller	smooth returns from selling tail risk	process
adversarial-input	accurate forecasts, collapse under perturbation	pass ^k
sybil-sock-puppets	near-clones shrink field dispersion	clone collapse

verify the match. The commitment proves byte binding, not chronology: the code does not observe wall time, data availability or what an entrant saw. Published Ed25519 boards chain each entry and anchor each window to the last signature of its predecessor, so alteration, reordering, or wholesale re-signing under a new history is detectable. The crate also retains an HMAC-SHA256 chain, but its guarantee is deliberately narrower: HMAC is shared-secret integrity among key-holders, and anyone given the verification key can also forge a replacement chain. Ed25519 separates the signing key from a publishable verifying key; a third party can verify without gaining signing power. An embedded verifying key proves only that the document is internally consistent under that key, so a verifier seeking host identity must pin the advertised key out of band. The file-backed lifecycle fixes scoring rules before entries exist, refuses late commitments, verifies reveals, and records refusals in the signed header. Each window carries a required schema version, a SHA-256 digest of its fully serialized scoring configuration, and a required digest of the scorer artifact; loading rejects a mismatch, including one caused by a new field being supplied from a later software default. The two initial records under `arena/` were empty and explicitly superseded rather than rewritten: `window-001` lacked the repaired scoring fields, and `window-002` named only a local debug executable rather than a publicly retrievable immutable artifact. Their exact historical bytes, reasons and old/new config linkage are digest-checked at load. The live window, `window-003`, is the first to meet that requirement: it binds the SHA-256 digest `263fadcd...63c2fa1` of the exact publicly retrievable `sharpebench-x86_64-linux-musl` binary attached to the attested v0.9.0 release, with a commit deadline at operator-supplied epoch 20711 and a data reveal at operator-supplied epoch 20720. It holds no commitment and resolves after this paper, so no performance result is claimed here. Intake, epoch advancement, held-out-data custody, sealed scoring, signing and publication remain operator-driven; the operator holds the data and key. The forward interpretation therefore trusts the declared epoch-to-wall-time mapping and the absence of pre-commitment observation. Until neutral hosting, third-party custody and a dispute procedure exist, the chain proves history integrity rather than host disinterest or true chronology.

Replay and contamination. A captured trajectory holds raw decisions and a versioned `TrajectoryContract`. Strict verification binds the dataset, cost model, engine version, ordered windows and seeds, and, for command-line captures, the runner executable; it also requires the complete ordered Cartesian run matrix, every expected decision step and the observation date at each step. A missing or mismatched contract is refused before replay. The explicit `-allow-unbound-trajectory` path exists only for a labeled legacy or cross-version regrade. A separate verifier then recomputes the score, and a test asserts that doubling every order recomputes to different returns, so an artifact cannot substitute self-reported performance. The contract does not bind the entrant commitment, so replay proves what the recorded decisions produce under the bound execution environment, not which strategy produced them. Sealed held-out data under a published hash, a canary token and a masked view that defeats ticker memorization defend against contamination. Both are specified in Section C. An opt-in image preflight adds one further check,

and its scope is narrow enough to be worth stating exactly. Given a policy of whole-file digests and exact UTF-8 sequences, the harness scans a digest-pinned image’s executable configuration and its container export, and refuses before the entrant is launched on a match or on a scan that could not complete; the container is created and exported but never started, and an unverified removal refuses rather than passing. Container export omits volume contents, so an image declaring a volume refuses rather than being reported as scanned over a scope the scan did not cover, and the accepted-output caps bound what the harness reads rather than imposing a disk quota. A completed negative result says only that the named streams did not contain the protected bytes: compressed, encoded, encrypted, chunked and model-internalized copies are outside raw-byte scope, so it is not evidence that an agent has not memorized held-out data and it is not a contamination-free certificate.

4.4 Submission and governance policy

The attestation machinery of Section 4.3 decides what a board can prove. It does not decide who may enter, what happens to a published score, or who adjudicates a dispute, and Singh et al. [2025] document what a leaderboard costs when those questions go unanswered: undisclosed private testing, selective retraction and unstated deprecation let a small number of entrants tune against a public board. The policy below is stated so a reader can hold the artifact to it, and each clause says whether code enforces it or the operator does.

Who may enter. The eligibility policy is open, but no hosted intake exists. An operator can run a digest-pinned container, an explicitly unsandboxed stdin/HTTP endpoint speaking the protocol contract, or a per-run return series through the import adapter of Section 6. The repository defines no fee or approval rule; it does not provide a public submission service. An imported entry carries no audit trace and an unverifiable trial count (Sections 3.6 and 6), so the policy is that it is recorded as imported and is not presented beside a container-run entry without that mark.

Scores are not retractable. If an operator publishes a score to a signed board, it stays on that history. The Ed25519 chain of Section 4.3 anchors each window to the last signature of its predecessor, so removing an entry after publication invalidates every subsequent window, which makes the policy enforceable by any verifier rather than by trust in the operator. No model-entry board has yet been published. An entrant may decline to submit; an entrant may not un-submit after publication.

Variant caps and disclosure. An entrant may commit at most three distinct artifact digests per window, and every commitment is recorded in the window header whether or not it is revealed, so an entrant who tests four variants and reveals the best leaves four rows. The cap is an intake policy the operator applies; the recording is what the lifecycle already does, and a refused late commitment is written into the signed header as a refusal (Section 4.3). Declared trial counts remain an honor system, so the cap bounds the variants an entrant can hide behind one identity and not the search an entrant did before committing.

The host’s own entries. If the host competes on a future board, its entries are marked as host entries. They count toward the observable trial footprint and toward the field-measured dispersion exactly as any other entry does, which is what Section 5.6 demonstrates on an offline author-built field when four added entrants move the dispersion source on five panels. The host has the same three-variant cap and holds the signing key, which is the conflict Section 10 states and Section 7 lists as unclosed.

Scoring defects after publication. Finding 1 makes this a live question. A defect found in the scorer after a board is published does not rewrite that board. The affected window is superseded by an explicit record naming the defect, the old configuration digest and the new one, and the superseded window keeps its original bytes, which is what happened to `window-001` and `window-002`. A re-scored field is published as a new window under the corrected configuration, and both remain retrievable, so the correction is auditable as a correction.

Changing the rules. A window’s scoring configuration is fully serialized and digest-committed before entries exist, and loading refuses a mismatch, including one introduced by a new field arriving from a later software default. A rule change therefore cannot apply to an open window. It takes effect from the next window opened, and the configuration digest is what tells a reader which rules a given board ran under. The agent wire contract is a different surface and carries none of that protection: it is versioned by the release, not by the window, and closing it in v0.11.0 broke entrants written against v0.10.x with no deprecation window (Section 3). The policy that follows is the one we can actually keep, which is disclosure rather than stability: a breaking wire change is named as breaking, carries its migration, and is checked against the published schema in both directions on every commit.

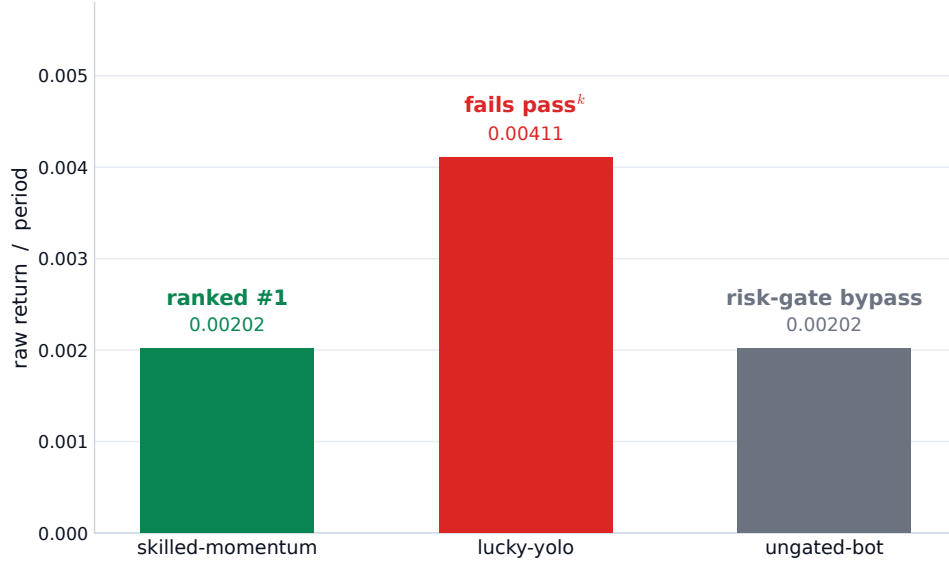
Disputes. A disputed entry is currently adjudicated by the operator, who is also a competitor. That is the weakest clause in this section and we state it as such: there is no third-party arbiter, no neutral host and no key escrow. What a disputant has today is the ability to recompute: the entry’s trajectory replays to its score (Section C.3), the board verifies from the document alone, and the configuration digest pins the rules. What they do not have is a body that can overrule the operator.

5 Experiments

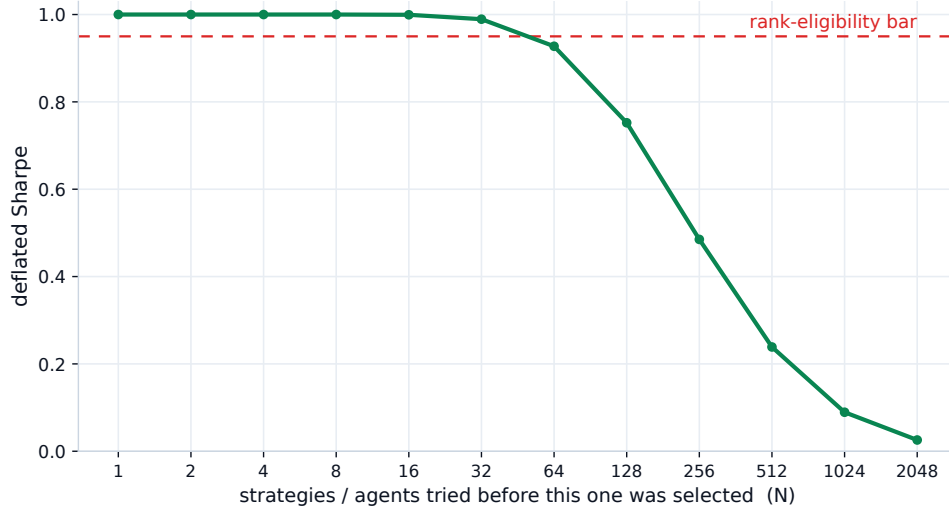
Every number in this section is produced by the commands in Section A from the committed data and the content-hashed source snapshot. All runs use eight execution seeds and six disjoint walk-forward windows sized to the dataset, and every run outside Section 5.6 uses the typical cost profile, with a warmup of one tenth of the series clamped to between 20 and 60 bars. The field on every dataset is three reference agents (buy-and-hold, cross-sectional momentum, and hold, which never trades) plus a luck floor of five seeded random agents: fully invested in random weights each period on multi-symbol universes, and drawing a random gross exposure, flat or a uniform long weight, on one-symbol universes, where full investment would otherwise collapse into buy-and-hold (Section 5.12). The luck floor is the zero-skill distribution an edge must clear. The scoring grid is $\beta \in \{0.80, 0.90, 0.95, 0.99\}$, host $N \in \{1, 10, 50, 200\}$, and σ_{trials} either measured from the field subject to its floor or pinned to one of three annualized priors. Equation (3) makes the effective count at least eight for every ranked cell. There are 64 host configurations per dataset, 512 agent records per dataset and 4,608 records in all.

Compute and cost. Kapoor et al. [2024] argue that an agent benchmark which does not report cost is not comparable, and the numbers below were measured rather than estimated, on one AMD Ryzen Threadripper PRO 5975WX (32 cores, 64 threads, 3.6 GHz base) with 256 GiB of RAM under Windows 11. Every command is a single-threaded process; the kernel spawns no threads and the parallelism in production is one process per shard. Regenerating the entire 4,608-cell grid of Table 4 in one serial process took 2,078 seconds, or 34 minutes 38 seconds, and reproduced all 4,608 committed deflated Sharpes exactly. The externally specified field of Section 5.6, 351 records over nine datasets, thirteen agents and three cost profiles, took 129.3 seconds at a peak resident set of 3,846 MiB, which is the memory envelope of the whole paper and is set by holding the hourly-crypto field of 624 runs over 3,990-bar windows in memory at once. The witness sweep of Section 5.11 took 8.4 seconds at 9.8 MiB, the nine-attack self-audit 0.58 seconds at 6.5 MiB, and the clone-collapse regression of Section 4.2 61.8 seconds. The nine frozen datasets occupy 6,455,868 bytes on disk.

No command in this paper calls a network service, holds an API key or uses a GPU, so the marginal monetary cost of reproducing every number here is the electricity to run one core for about 37 minutes. Scoring one additional submission against a nine-dataset field is a few seconds of that total. This is the axis on which a judge-free *scoring kernel* is not merely cheaper than an LLM-judged benchmark but different in kind: the judging cost does not scale with the number of entrants times the price of a frontier model. Running a hosted-model field is a separate cost that does scale that way, which is why the host-observed accounting of Section 4 exists, and the sweep is embarrassingly parallel across its 36 (dataset, deflation-bar) shards, which is how the committed evidence is actually produced.



(a) Three agents, one board.



(b) One track, ever more trials.

Figure 1: (a) The shipped demonstration. (b) The same track scored against a growing pool of trials, per-period Sharpe 0.85 over 150 periods, cross-trial dispersion 0.30 per period, and normal moments. Both computed with the kernel's own formulas.

5.1 The demonstration that motivated the benchmark

Figure 1a shows the three-agent field the benchmark shipped with: a skilled momentum agent, a lucky agent with twice its raw return that fails pass^k, and a bot that matched the skilled return but placed one order that never passed its risk gate and is zeroed by the process gate. The agent with the highest raw return ranks last. Figure 1b shows why: one fixed track, scored against a growing pool of trials, crosses the eligibility bar between 32 and 64 trials and collapses past a few hundred. Both figures are computed on synthetic returns.

Table 3: The annualized Sharpe an agent had to exceed to reach the deflation benchmark at $N = 50$, for each prior as it was applied (per period) and each bar size. Kernel at v0.2.1, deliberately: this table documents that kernel’s error. The 0.070 row is the dispersion measured from the daily US indices field.

σ_{trials} as applied	SR_0^* per period	1h	4h	1d	1w
0.0700	0.1590	14.9000	7.5000	2.5000	1.1000
0.2000	0.4550	42.6000	21.3000	7.2000	3.3000
0.3500	0.7970	74.6000	37.3000	12.6000	5.7000
0.5000	1.1380	106.5000	53.3000	18.1000	8.2000

5.2 Finding 1: units of dispersion are a protocol trap

The lesson generalizes to any scorer that combines literature constants with per-period statistics: the deflation literature states σ_{trials} annualized, kernels compute Sharpe per period, and the conversion between them is a factor of $\sqrt{\text{periods per year}}$ that no self-consistency check will catch if it is dropped, for the reason Section 2 gives. An earlier kernel omitted it.

The error surfaced when the first run of the full grid on the two original daily datasets produced zero rank-eligible agents in every one of the 128 cells. Buy-and-hold on US indices posted a PSR of 1.0000 against zero and a bootstrap p of 0.0005, and scored a deflated Sharpe of exactly 0.0000. Those values are mathematically compatible because PSR compares with zero while DSR compares with a search-adjusted threshold. Their coexistence nevertheless localized the refusal to that threshold rather than the return estimate and prompted the unit audit; the eight-attack self-audit stayed green throughout (Section 4.1).

The default $\sigma_{\text{trials}} = 0.5$ follows the annualized trial-dispersion example of Bailey and López de Prado [2014, pp. 102–103]. Substituted per period into Equation (2) at $N = 50$ it sets $SR_0^* = 1.14$ per period. Table 3 converts that bar back into the annualized Sharpe an agent would need to reach it. On daily bars the requirement was an annualized Sharpe of 18; on hourly bars, 106. The three indices in `us-indices-1d` realize annualized Sharpe ratios of 0.69 (DJI), 0.79 (SPX) and 0.87 (IXIC) over their 2,512 daily returns, computed under the kernel’s own convention by the script in Section A. The bar was unreachable, and it grew with the square root of the number of periods per year. A separate pre-floor experiment produced DSR 0.984 weekly and 0.000 daily from small field-measured dispersions; that contrast exposed instability in the measured-field estimate, not the configured-prior unit error. The current protocol records the source and applies the precommitted floor, and the current-floor results are reported separately below.

The rival benchmarks of Section 6, which rank LLM trading agents on return-based metrics over short windows, report annualized Sharpe without deflating for selection. This benchmark applied deflation with a prior in the wrong frequency units. These are different failures, an omitted multiple-testing correction and a dimensional miscalibration, rather than opposite unit errors. The latter is harder to notice because it produces a benchmark that appears strict. It also explains, completely, why the demonstration in Section 5.1 had only ever been shown on synthetic inputs: those had per-period returns large enough to clear an annualized bar. The fix, shipped in v0.3.0, leaves the per-period kernel untouched and states the prior annualized with a periods-per-year field that converts it once at the point of use; Section C gives the tests that pin it, which is the pinned unit-convention test the opening lesson calls for.

5.3 Finding 2: the current defaults refuse on both edge and regime robustness

With the units corrected and the field-level defenses enabled, the full grid was rerun on all nine datasets. Table 4 reports the default configuration on each, with the dispersion source and annualized deflation bar printed for every row. Three sources are possible and the live evidence stamps which applied: the configured annualized 0.5 prior, a value measured from the field, or a measured value raised to the annualized 0.5 floor.

The measured path did not apply everywhere. Ranking measures the dispersion over the field’s seed-averaged pooled streams after clone collapse, and it declines to measure when fewer than five votes survive that collapse. On five of the nine panels (both weekly panels, daily crypto, daily US indices and 4-hour crypto) the luck-floor agents merge into one cluster once their eight execution replicates are averaged, joined by buy-and-hold on three of the five, leaving three or four votes; those cells therefore fall back to the configured 0.5 prior, at an annualized bar of 1.1382. Section 4.2 gives the cluster on each panel. Commodities measures a value below the floor and is raised to the same 0.5, also 1.1382. That stamp is frozen evidence: under the repaired simulator the same panel reconstructs too few finite-Sharpe streams to measure at all and would fall back to the configured prior instead (Section 7). Only daily FX, daily rates and hourly crypto measure above the floor, at annualized dispersions of 2.9985, 1.5095 and 12.0067 and annualized bars of 6.8254, 3.4362 and 27.3309. A regression test reproduces the clustering on those exact streams and asserts the vote counts that imply each committed stamp (Section A); the merging itself is analyzed as a finding about the collapse threshold in Section 4.2.

Weekly US buy-and-hold scores 0.1781 and weekly crypto momentum 0.2406, both far below $\beta = 0.95$. The three measured panels show what the annualized bar reveals: the largest bar, 27.33, occurs on the finest bars, and the smallest, 3.44, on daily rates. Frequency alone does not order them, because daily FX sits at 6.83 against daily rates at 3.44 at identical periods per year, so the measured dispersion is panel-specific. The mechanism is cost drag: the luck floor’s per-period cost bleed scales with the number of trades and not with $1/\sqrt{P}$, so the dispersion between a cost-bleeding random agent and a reference agent widens as bars get finer, and Section 5.6 confirms this directly by setting costs to zero, where the measured bar returns to the neighborhood of the precommitted floor on every one of the nine panels. That is the same dimensional mechanism Finding 1 generalizes, now visible on the measured side of Equation (4) instead of the configured side. Measurement may make deflation stricter and can never take it below the precommitted prior.

No agent is eligible in any of the 576 configurations or 4,608 scored agent–configuration cells. At the shipped default, deflation and regime robustness refuse independently: every named reference agent is below the DSR bar, and every one also fails pass^k. The reliability failure comes from windows rather than the narrow seed perturbations (Section 3.1): every frozen range contains at least one labeled bear window. The longer panels include the 2020 and 2022 drawdowns; the crypto 1h, 4h and 1d panels begin in 2023 and contain later drawdowns instead. The default mode requires a marginal per-run PSR of 0.90 on every window. A long-only agent has a negative realized Sharpe in a bear window and cannot clear any positive bar there. Two scopings bound this claim. It is sample-period-contingent: a different range can change the per-window verdict (Section 7 bounds this to the selected histories). And the nine refusals are not nine independent confirmations: the resampled timeframes and co-moving symbols reduce to roughly five market panels with dependent histories. Only the synthetic witness of Section 5.11 separates the gates and shows the conjunction attainable on a sampled grid.

The default mode does not ask only whether an agent has an edge. It requires every run’s marginal PSR against zero to reach 0.90 in every regime and under every execution seed; the conjunction is not itself a simultaneous 90-percent confidence statement. A regime-dependent edge such as equity beta correctly fails that question. On histories containing a downturn, the benchmark is declining to certify that owning the index is safe in a bear market, which it is not.

5.4 Finding 3: a weaker reliability gate admits nobody either

Whether the reliability gate should mean profitable in every regime or never catastrophic in any regime is a design question (Section 3.3 states why the stricter verdict ships as the default), and the kernel was extended so that both are expressible from configuration and the ablation is reproducible (a third, benchmark-relative verdict is added in Section 5.7). The never-catastrophic preset sets the pass mode to any run and adds a per-run drawdown bound of 20 percent, with the edge tested on the pooled track. It is a weaker safety claim, and a test pins its cost: a synthetic agent with one

Table 4: Default configuration ($\beta = 0.95$, host $N = 50$, field size eight) on all nine datasets. One row per dataset: the agent with the highest deflated Sharpe at this configuration. Worst DD is the largest drawdown in any single window. σ source is the dispersion source each record stamps: *configured* is the annualized 0.5 prior, applied because clone collapse over the seed-averaged streams left fewer than five dispersion votes; *measured* is the field-measured value; *floored* is a measured value raised to the annualized 0.5 floor. Bar (ann.) is that cell’s deflation benchmark SR_0^* expressed as an annualized Sharpe. Eligible is the all-weather verdict of Section 5.3. Full per-agent records for all 512 configurations per dataset and the source-snapshot manifest are in [paper/evidence/](#).

Dataset	Agent	DSR	Worst DD	p_{boot}	σ source	Bar (ann.)	Eligible
US indices 1w	buy-and-hold	0.1780	0.3200	0.0015	configured	1.1382	no
Crypto 1w	momentum	0.2410	0.5630	0.0435	configured	1.1382	no
Crypto 1d	buy-and-hold	0.2510	0.4760	0.1219	configured	1.1382	no
Commodities 1d	hold	0.0000	0.0000	1.0000	floored	1.1382	no
US indices 1d	buy-and-hold	0.1560	0.3360	0.0020	configured	1.1382	no
Crypto 4h	buy-and-hold	0.3230	0.4790	0.0735	configured	1.1382	no
Crypto 1h	buy-and-hold	0.0000	0.4900	0.0685	measured	27.3309	no
FX 1d	hold	0.0000	0.0000	1.0000	measured	6.8254	no
Rates 1d	buy-and-hold	0.0000	0.8400	0.1754	measured	3.4362	no

high-return run and four noise runs is refused by the default only through pass^k and is admitted by the preset.

On the real datasets the preset admits nobody. The never-catastrophic verdict returns *no* on all nine rows of Table 4, which is why that table prints one eligibility column and not two, and the worst-drawdown column supplies an additional refusal wherever a trading agent leads the row: seven of the nine rows breach the 20 percent bound outright. The two rows led by hold print a worst drawdown of 0.000 because a cash agent never draws down; the traded exposures on those same panels do breach it, daily-FX buy-and-hold at 0.199 being the single exception in the grid, and both are refused by deflation and the bootstrap instead. The US index drew down 32 percent in its worst weekly window. Crypto buy-and-hold drew down between 48 and 67 percent across timeframes, and hourly momentum lost 99.3 percent of its value in a single window, the largest single-window loss in the grid. Crypto 4h buy-and-hold has the largest default reference-agent DSR at 0.3230, still far below deflation, and that value holds only in the eight-agent field, whose clone collapse leaves too few votes to measure and falls back to the configured prior (Section 5.8). Weekly crypto momentum is also refused on a 56 percent drawdown in one window and is the raw-return leader. On these datasets a 20 percent per-regime drawdown bound is not weaker than every-regime profitability; it is stricter for the unhedged exposure that crosses the cap.

One correction to how the per-run bound is naturally described. Under plain concatenation the usual description holds: drawdown is multiplicative, so a run’s own drawdown never exceeds the pooled track’s, because every within-run peak-to-trough pair is also a pooled pair. That is not the pooling this benchmark performs. Execution seeds are averaged *within* a window before the pooled track is formed, and that averaging breaks the containment: a trough inside one seed can be cancelled by the opposite move in a sibling seed, so it never becomes a pooled pair at all. Two aligned runs $[0.1, -0.2]$ and $[-0.1, 0.2]$, scored as two execution seeds of a single window, average to $[0, 0]$, giving a pooled drawdown of 0.000 against a worst-run drawdown of 0.200. The pooled bound and the per-run bound are therefore *independent* constraints rather than nested ones, neither implying the other, and the kernel retains both and checks each against the quantity it is written for. The per-run bound also bites, as already noted, when set below the pooled cap: a loose whole-track budget with a tight per-regime bound. The preset is configured that way, and a regression pins the anti-correlated-seed case beside the test that constructs the tight-per-regime one.

5.5 Finding 4: the gates discriminate on a disciplined agent without alpha

The findings above show the gates refusing unhedged exposure, which leaves open whether they can distinguish anything finer. To probe that without inventing alpha, a risk-managed reference agent was

added to the field: long the equal-weight basket only when it trades above its trailing mean, gross exposure scaled to an inverse-volatility target, and fully flat after a 10 percent drawdown with re-entry only on a fresh trend signal. It is textbook risk management and nothing more; every parameter is a documented default and none was tuned against the gate.

The result is a negative, and it narrows what the discrimination claim can be. Under corrected execution-seed pooling, weekly-US risk-managed does not clear the per-run PSR/reliability leg or the bootstrap. It remains process-clean and inside its 20-percent per-run drawdown bound, but is refused by deflation, bootstrap, and the applicable reliability verdict. The control therefore does not isolate deflation. What it shows is that documented risk controls improve drawdown on the weekly US panel without manufacturing an eligible statistical edge.

The result remains ineligible over the recorded trial-count sensitivity grid: every effective- N setting is below the 0.95 deflation bar, and the other refusing legs remain visible in the same records. On seven of the nine datasets the agent's worst-window drawdown is the smallest of any trading agent (Figure 2a), so the discipline registers where it was designed to. What discipline does not manufacture is a deflation-surviving edge.

The two exceptions cut the other way. On hourly crypto the trend filter is whipsawed into an 87 percent drawdown in one window, far worse than buy-and-hold's 49, and on daily FX its 33 percent trails buy-and-hold's 20: the same rules that protect capital on daily and weekly equity bars amplify losses where the trend signal churns. Risk management is frequency-dependent, and a benchmark that scores per window is what makes that visible instead of averaged away.

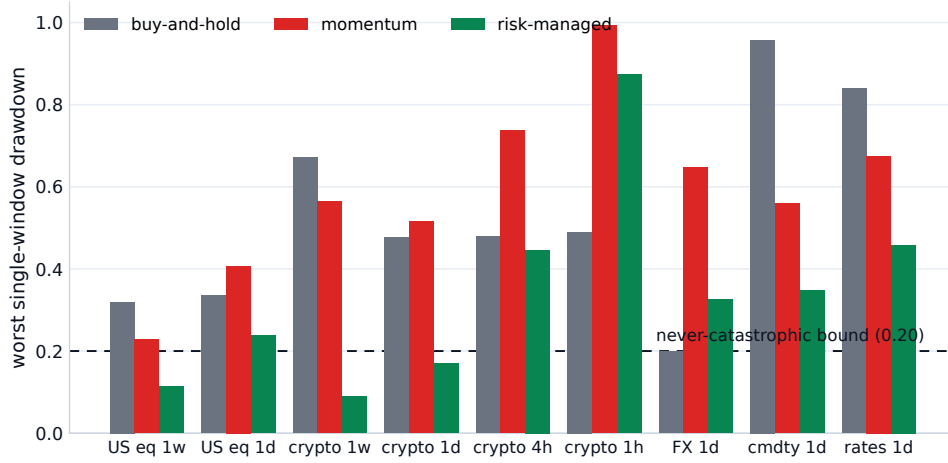
Together with Section 5.3 this shows the gates discriminate rather than blanket-refuse: unhedged beta fails on regime robustness, and discipline without edge, under the verdict that waives regime robustness, fails on deflation, each with the quantity that justifies it on the same row.

5.6 Finding 5: externally specified rules, and what the cost profile moves

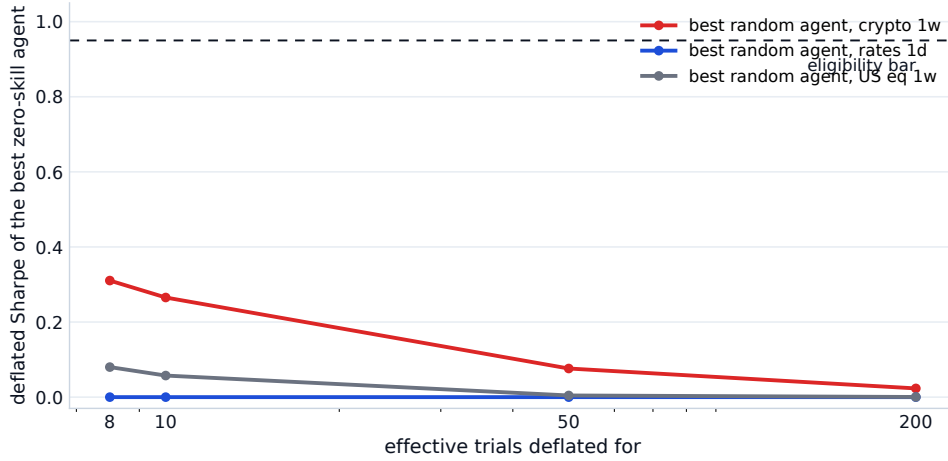
Every entrant above was written for this repository, so a reader cannot yet separate a strict gate from a weak field. This subsection scores four trading rules whose specifications come from the published literature, and scores them under each of the three named cost profiles, which answers the cost-calibration limitation at the same time.

The rules. `donchian-20-10` is the Donchian channel breakout at the Turtle System 1 parameters [Faith, 2007]: enter long when the latest close exceeds the maximum of the prior 20 closes, exit to cash when it falls below the minimum of the prior 10. `b11-vma-1-50` is the variable-moving-average rule of Brock et al. [1992], long when the close is above its trailing 50-bar mean and flat otherwise, with no band. `faber-10m` is the ten-month simple-moving-average timing rule of Faber [2007], converted to each dataset's own bar count. `rsi-14-wilder` is Wilder [1978]'s RSI(14) with the 30/70 thresholds: enter long below 30, exit above 70, hold between. Each allocates $1/n_{\text{symbols}}$ to every symbol it is currently long, so gross exposure never exceeds one and the rules are comparable with the long-only references. We implemented each rule from its published statement and tuned no parameter here; the specification is external and the implementation is ours. Sullivan et al. [1999] applied White's Reality Check to exactly this rule universe and found that its apparent profitability does not survive a data-snooping adjustment, which is the same correction Equation (2) applies at the level of a single entrant.

Evidence. The field is the eight named agents (buy-and-hold, momentum, hold, the risk-managed agent of Section 5.5, and the four rules above) plus five luck-floor agents, thirteen in all, on the same nine datasets, the same window rule, the same eight execution seeds and the same default configuration, scored once per cost profile: 351 records. Table 5 lists the four rules under the typical profile and Table 6 gives the cost sweep.



(a) Worst single-window drawdowns.



(b) Deflation against zero-skill agents.

Figure 2: Computed from the committed evidence records by a committed script. (a) The risk-managed agent posts the smallest worst-window drawdown of any trading agent on seven of nine datasets, while both long-only references breach the 20 percent per-regime bound almost everywhere and hourly momentum loses 99.3 percent in one window; the two exceptions, hourly crypto and daily FX, are where the trend filter itself whipsaws. (b) The best zero-skill agent's operational DSR as the effective trial count grows. A host setting of one becomes eight because the submitted field has eight streams; the measured dispersion is also bounded below by the annualized prior. The curve therefore describes the shipped ranking path, not an unfloored diagnostic.

Result. No record is rank-eligible and no record passes pass^k , in any of the 351 cells, under any of the three cost profiles. The highest deflated Sharpe any entrant has reached anywhere in this paper is now an external one, *donchian-20-10* on daily crypto at 0.6534 frictionless, 0.4155 typical and 0.1185 stressed, against the 0.95 bar. That is above the 0.3230 of Section 5.4 and still short of eligibility by a wide margin, and its block-bootstrap p moves from 0.0425 to 0.0655 to 0.1499 across the same three profiles. Removing every trading friction does not admit anybody, so the universal refusal of Section 5.3 is not an artifact of the chosen cost calibration.

The cost profile moves the bar, not only the returns. Table 6 prints the annualized deflation benchmark on every cell, and it is the more interesting half of the sweep. Under the frictionless profile the measured field dispersion sits at or near the precommitted annualized floor on every one of the nine panels, so the bar is 1.1382 on six of them and 1.2024, 1.2576 and 1.2825 on daily, hourly and 4-hour crypto. Under the typical profile five panels rise well above that: hourly crypto to 27.5397, daily FX to 6.7466, 4-hour crypto to 5.2022, daily rates to 3.2994 and daily US indices to 1.9664. Under the stressed profile those same five read 31.9269, 10.1954, 7.9760, 2.8223 and 1.8665. This locates the mechanism Section 5.3 attributes to sampling frequency: at zero cost the bar is near the floor at every bar size, so what widens the field’s dispersion is the per-trade cost drag on the zero-skill agents, and frequency matters only because a finer bar buys more trades. Daily rates falls from 3.2994 to 2.8223 as costs rise, so the ordering is panel-specific even in cost.

The window geometry bounds the lookback a published rule may use. The protocol’s observation carries twenty trailing closes and each run starts fresh at its window boundary, so a rule needing L closes emits no signal for the first $L - 20$ bars of every window. *rsi-14-wilder* needs 15 and is never blind; *donchian-20-10* needs 21 and is blind for one bar; *b11-vma-1-50* needs 50 and is blind for 30. *faber-10m* needs 210 daily bars, 43 weekly, 1,825 at 4-hour and 7,300 at hourly, against windows of 408, 682, 677, 156, 78, 46, 990 and 3,990 bars, so it is blind for 190 of every 408-bar US daily window, for 23 of every 46-bar weekly crypto window, and for every bar of all six windows on daily crypto, 4-hour crypto and hourly crypto. The canonical twelve-month trend rules of the published literature are therefore not expressible on six of the nine panels at the shipped window rule, which is a property of the benchmark to fix, and Section 7 records it.

Field composition and comparability. Adding four agents changes the field, so these deflated Sharpes are not the same statistic as Table 4’s. Where the effective bar is unchanged the numbers reproduce exactly: weekly US buy-and-hold is 0.1781 and weekly crypto momentum is 0.2406 in both tables, because the eight-agent field falls back to the configured 0.5 prior there while the thirteen-agent field measures a value below the floor and is raised to the same 0.5, giving an identical bar of 1.1382 by two different routes. Where the wider field supplies enough surviving votes to measure above the floor, the bar moves: daily US indices, 4-hour crypto and daily crypto all stamp *configured* in Table 4 and *measured* here, at typical-profile bars of 1.9664, 5.2022 and 1.4079 against 1.1382. Every record here serializes `trials_sr_std_source` and the annualized bar beside the score, so the two tables reconcile cell by cell instead of merely differing.

Command. The producing command is listed in Section A. It writes one record per (dataset, cost profile, agent), 351 in all, each carrying the command, the rule’s published source, the closes the rule requires, the blind-bar count, the dispersion source, the annualized bar and the full gate vector.

The other two reliability verdicts. Findings 2 and 3 fix the verdict and vary the agent. The next two subsections do the reverse: they hold the reference field fixed and vary the reliability question the host asks, first as a host configuration and then as a claim the submitter declares.

5.7 A mandate-relative reliability verdict

The default reliability gate tests each run’s Probabilistic Sharpe against zero: $\text{PSR}(r, 0) \geq 0.90$ in every window and under every execution seed. That is an all-weather absolute-return mandate, and

Table 5: The four externally specified rules on all nine frozen datasets under the typical cost profile, in a thirteen-agent field, at the default configuration ($\beta = 0.95$, host $N = 50$). Worst DD is the largest drawdown in any single window. Blind is the bars of every window in which the rule cannot form its signal, over the window length. No cell passes pass^k and no cell is eligible.

Dataset	Rule	DSR	Worst DD	p_{boot}	Blind	pass^k	Eligible
US indices 1d	donchian-20-10	0.0000	0.1190	0.0635	1/408	no	no
US indices 1d	bll-vma-1-50	0.0000	0.2640	0.2189	30/408	no	no
US indices 1d	faber-10m	0.0002	0.1010	0.0030	190/408	no	no
US indices 1d	rsi-14-wilder	0.0000	0.2800	0.0240	0/408	no	no
US indices 1w	donchian-20-10	0.0247	0.1530	0.0565	1/78	no	no
US indices 1w	bll-vma-1-50	0.0259	0.1560	0.0695	30/78	no	no
US indices 1w	faber-10m	0.1254	0.1560	0.0105	23/78	no	no
US indices 1w	rsi-14-wilder	0.1010	0.1670	0.0185	0/78	no	no
Crypto 1h	donchian-20-10	0.0000	0.5370	1.0000	1/3990	no	no
Crypto 1h	bll-vma-1-50	0.0000	0.6230	1.0000	30/3990	no	no
Crypto 1h	faber-10m	0.0000	0.0000	1.0000	3990/3990	no	no
Crypto 1h	rsi-14-wilder	0.0000	0.4070	1.0000	0/3990	no	no
Crypto 4h	donchian-20-10	0.0000	0.2920	0.2444	1/990	no	no
Crypto 4h	bll-vma-1-50	0.0000	0.3670	0.4648	30/990	no	no
Crypto 4h	faber-10m	0.0000	0.0000	1.0000	990/990	no	no
Crypto 4h	rsi-14-wilder	0.0000	0.3740	0.2134	0/990	no	no
Crypto 1d	donchian-20-10	0.4155	0.1650	0.0655	1/156	no	no
Crypto 1d	bll-vma-1-50	0.1807	0.2640	0.1384	30/156	no	no
Crypto 1d	faber-10m	0.0121	0.0000	1.0000	156/156	no	no
Crypto 1d	rsi-14-wilder	0.0213	0.2460	0.3913	0/156	no	no
Crypto 1w	donchian-20-10	0.1221	0.4320	0.0785	1/46	no	no
Crypto 1w	bll-vma-1-50	0.0011	0.4270	1.0000	30/46	no	no
Crypto 1w	faber-10m	0.0021	0.4340	1.0000	23/46	no	no
Crypto 1w	rsi-14-wilder	0.0333	0.1460	0.1569	0/46	no	no
FX 1d	donchian-20-10	0.0000	0.1700	1.0000	1/682	no	no
FX 1d	bll-vma-1-50	0.0000	0.2260	1.0000	30/682	no	no
FX 1d	faber-10m	0.0000	0.1510	1.0000	190/682	no	no
FX 1d	rsi-14-wilder	0.0000	0.1610	1.0000	0/682	no	no
Commodities 1d	donchian-20-10	0.0007	0.3420	0.0955	1/677	no	no
Commodities 1d	bll-vma-1-50	0.0000	0.5860	1.0000	30/677	no	no
Commodities 1d	faber-10m	0.0000	0.4660	1.0000	190/677	no	no
Commodities 1d	rsi-14-wilder	0.0000	0.9400	0.1049	0/677	no	no
Rates 1d	donchian-20-10	0.0000	0.5970	1.0000	1/682	no	no
Rates 1d	bll-vma-1-50	0.0000	0.6110	1.0000	30/682	no	no
Rates 1d	faber-10m	0.0000	0.4170	0.3478	190/682	no	no
Rates 1d	rsi-14-wilder	0.0000	0.8220	0.3513	0/682	no	no

Section 5.3 showed its consequence on these histories: the index cannot pass on any range containing 2020 or 2022, and neither can anything that holds the index. The never-catastrophic verdict of Section 5.4 relaxes the *aggregation* (one run is enough) and adds a drawdown bound. This section adds a third verdict that keeps the aggregation strict and changes the *series under test*: each run is judged relative to owning the universe in that same window, which is the question most real mandates ask.

Definition. Let $r_t(a, w, s)$ be agent a 's per-bar return in window w under execution seed s , and let b be the benchmark agent, buy-and-hold of the same universe. Under `PassMode::RelativeToBenchmark`, run (w, s) of agent a passes if and only if, for the excess series $e_t = r_t(a, w, s) - r_t(b, w, s)$,

$$\text{std}(e) > 0 \quad \text{and} \quad \text{PSR}(e, \text{SR}_{\text{run}}^*) \geq 0.90, \quad (6)$$

where SR_{run}^* is the per-period benchmark the `annualized_per_run_min_annual_sharpe` converts to, zero by default. The agent passes pass^k if and only if every run passes, exactly as under the default. Every other gate (pooled deflated Sharpe, block bootstrap, process audit, drawdown mandate) is computed on the agent's own raw returns and is byte-identical under both verdicts; the relative verdict is a different reliability question, not a weaker set of gates.

Table 6: Cost sensitivity across the three shipped profiles, thirteen-agent field, default configuration. Ext. is the highest deflated Sharpe among the four externally specified rules; Field is the highest in the whole thirteen-agent field; Bar is that cell’s deflation benchmark expressed as an annualized Sharpe. Frictionless charges no fee, slippage, impact or financing; typical charges 2, 3, 50 and 5 basis points; stressed charges 10, 15, 150 and 20 with a 10 percent participation cap. The stressed profile’s two-bar decision delay is not applied, because the backtest driver consumes the cost model only. Zero cells are eligible and zero pass pass^k under every profile.

Dataset	Frictionless			Typical			Stressed		
	Ext.	Field	Bar	Ext.	Field	Bar	Ext.	Field	Bar
US indices 1d	0.2910	0.2910	1.1382	0.0002	0.0002	1.9664	0.0004	0.0004	1.8665
US indices 1w	0.1797	0.1918	1.1382	0.1254	0.1781	1.1382	0.0932	0.1550	1.1382
Crypto 1h	0.4315	0.6518	1.2576	0.0000	0.0000	27.5397	0.0000	0.0000	31.9269
Crypto 4h	0.5974	0.5974	1.2825	0.0000	0.0000	5.2022	0.0000	0.0000	7.9760
Crypto 1d	0.6534	0.6534	1.2024	0.4155	0.4155	1.4079	0.1185	0.1185	1.5695
Crypto 1w	0.1327	0.2935	1.1382	0.1221	0.2406	1.1382	0.0747	0.1451	1.1382
FX 1d	0.0000	0.0000	1.1382	0.0000	0.0000	6.7466	0.0000	0.0000	10.1954
Commodities 1d	0.0114	0.0114	1.1382	0.0007	0.0007	1.1382	0.0000	0.0000	1.1382
Rates 1d	0.0004	0.0007	1.1382	0.0000	0.0000	3.2994	0.0000	0.0000	2.8223

Three properties of Equation (6) matter. First, the benchmark series is not an input: it is the run of the agent named `benchmark_agent_id` (default buy-and-hold) in the *same field being ranked*, at the same position in the window-major layout, so it was produced from the same frozen bars, the same window, the same execution seed and the same cost model as the run it is subtracted from. Nothing is fetched from outside the field; a field without the named agent, or a misaligned cell, fails the run rather than falling back to the absolute test. Second, the benchmark cannot certify itself. Its own excess series is identically zero, and a zero-dispersion series is no evidence of outperformance: the kernel defines its Sharpe as 0, so PSR returns $\Phi(0) = 0.5$ and the run fails any bar above one half; the explicit `std(e) > 0` clause makes the refusal unconditional, so an operator who lowered the bar to 0.5 would still not admit buy-and-hold, and a clone of the benchmark under another name fails for the same reason. A run indistinguishable from the benchmark has no excess edge, and the verdict is a claim about excess edge in every window. Third, the reliability verdict is opt-in. `pass_mode` deserializes to `all` from every existing config, so the benchmark-relative *pass* predicate is not silently selected. The named benchmark also anchors the fixed aligned series for the reported RC/SPA/Romano–Wolf diagnostics; changing it can therefore change those diagnostics, while the default eligibility predicate remains unchanged. It is selected with `sharpebench run -pass-mode relative-to-benchmark [-benchmark-agent <id>]`, the same flags on `sharpebench score`, `ScoreConfig::relative_to_benchmark(id)` in Rust, or `relative_to_benchmark_config(id)` in Python. `sharpebench_core::per_run_passes` exposes the kernel’s per-run vector so a report can say which windows passed, which is what the table below uses.

Evidence. The same field as Section 5.5 (buy-and-hold, momentum, hold, the risk-managed agent, and five luck-floor agents), the same window rule, the same eight execution seeds and the same cost model, on all nine frozen datasets, scored under the default verdict and under Equation (6) with buy-and-hold as the benchmark. Six windows times eight seeds is 48 cells per agent per dataset. Table 7 lists every named agent on every dataset; the 45 luck-floor records are summarized in the text because none of them changes.

Result. No agent becomes eligible under the relative verdict on any dataset, and no agent passes pass^k under it: the best any agent manages is two of six windows (momentum on weekly crypto, 16 of 48 cells), and every other named agent passes at most one window. The 45 luck-floor records are unchanged in kind: at most 12 of 48 cells pass under the default and at most 3 under the relative verdict, no window passes on all eight seeds under either, and none is eligible. The “nothing passes”

Table 7: Per-window reliability under the default verdict (marginal PSR against zero at least 0.90) and the relative verdict of Equation (6) (marginal PSR on excess over buy-and-hold at least 0.90), all nine datasets. These are per-run thresholds, not simultaneous 90-percent confidence statements. Regimes are the six out-of-sample windows in order: U bull, D bear, C chop. A window is marked P when all eight seeds pass and a period otherwise. Cells is the number of the 48 (window, seed) cells that pass. Gained lists windows that pass relative and failed default; Lost the reverse. Every other gate statistic is identical under both verdicts. No agent is rank-eligible under either verdict on any dataset.

Dataset	Agent	Regimes	Default / Relative (windows 0 to 5)	Cells def. rel.	Gained	Lost	Eligible def. rel.
US indices 1d	buy-and-hold	UUUDUU	P...P. /	16 0	none	0, 4	no no
US indices 1d	momentum	UUUDUU	 /	0 0	none	none	no no
US indices 1d	hold	UUUDUU	 /	0 0	none	none	no no
US indices 1d	risk-managed	UUUDUU	 /	0 0	none	none	no no
US indices 1w	buy-and-hold	UUUDUU	..P.P. /	16 0	none	2, 4	no no
US indices 1w	momentum	UUUDUU	..P.P. /	16 0	none	2, 4	no no
US indices 1w	hold	UUUDUU	 /	0 0	none	none	no no
US indices 1w	risk-managed	UUUDUU	 /	0 0	none	none	no no
Crypto 1h	buy-and-hold	UDUDD	P.P... /	16 0	none	0, 2	no no
Crypto 1h	momentum	UDUDD	 /	0 0	none	none	no no
Crypto 1h	hold	UDUDD	 /P	0 8 5	none	none	no no
Crypto 1h	risk-managed	UDUDD	 /	0 0	none	none	no no
Crypto 4h	buy-and-hold	UDUDD	P.P... /	16 0	none	0, 2	no no
Crypto 4h	momentum	UDUDD	 /	0 0	none	none	no no
Crypto 4h	hold	UDUDD	 /P	0 8 5	none	none	no no
Crypto 4h	risk-managed	UDUDD	 /	0 0	none	none	no no
Crypto 1d	buy-and-hold	UCUDD	P..P.. /	16 0	none	0, 3	no no
Crypto 1d	momentum	UCUDD	 /	0 0	none	none	no no
Crypto 1d	hold	UCUDD	 /P	0 8 5	none	none	no no
Crypto 1d	risk-managed	UCUDD	P..P.. /P	16 8 5	0, 3	no no	no no
Crypto 1w	buy-and-hold	UDUUUD	..PPP. /	24 0	none	2, 3, 4	no no
Crypto 1w	momentum	UDUUUD	P..P.. / P....P	16 16 5	3	no no	no no
Crypto 1w	hold	UDUUUD	 /P	0 8 5	none	none	no no
Crypto 1w	risk-managed	UDUUUD	 /P	0 8 5	none	none	no no
FX 1d	buy-and-hold	UDUCUD	 /	0 0	none	none	no no
FX 1d	momentum	UDUCUD	 /	0 0	none	none	no no
FX 1d	hold	UDUCUD	 / .P....	0 8 1	none	none	no no
FX 1d	risk-managed	UDUCUD	 /	0 0	none	none	no no
Commodities 1d	buy-and-hold	UDUDUC	..PP.. /	16 0	none	2, 3	no no
Commodities 1d	momentum	UDUDUC	 /	0 0	none	none	no no
Commodities 1d	hold	UDUDUC	 / .P....	0 8 1	none	none	no no
Commodities 1d	risk-managed	UDUDUC	 / .P....	0 8 1	none	none	no no
Rates 1d	buy-and-hold	DUUDUU	...P. /	8 0	none	4	no no
Rates 1d	momentum	DUUDUU	 /	0 0	none	none	no no
Rates 1d	hold	DUUDUU	 /	0 0	none	none	no no
Rates 1d	risk-managed	DUUDUU	 /	0 0	none	none	no no

result is therefore not an artifact of the all-weather mandate: the mandate-relative verdict, which asks only for outperformance of the universe, refuses every entrant too.

What the relative verdict changes is *which* windows pass, in the direction implied by the benchmark definition. Buy-and-hold loses every window it passed under the default (16 windows across the nine datasets) because it is now measured against itself, and gains none, by the zero-excess rule. Every window gained by any agent is a bear window: window 5 on the four crypto datasets (a bear window on each), window 1 on FX and commodities (bear on both). They are gained by the agents that were out of the market when the universe fell: hold, which holds cash and therefore has excess return equal to minus the index in every bar; the risk-managed agent, whose drawdown halt and trend filter took it flat; and momentum on weekly crypto, which closes to cash on a negative lookback and was flat for the closing bear. The default verdict cannot see this, because an agent that is flat in a bear window has zero raw return there and $PSR(0, 0) = 0.5$; the relative verdict sees it and credits it. Symmetrically, windows lost are bull windows in which the agent was profitable but did not beat the

index: momentum’s window 3 on weekly crypto, both of the risk-managed agent’s windows on daily crypto, and all of momentum’s windows on weekly US indices, where it tracks the index closely enough that its excess does not reach the marginal 0.90 PSR threshold in either direction.

The two verdicts fail the same agents for complementary reasons, and neither reason is a benchmark defect. Under the default a long-only agent fails on the bear windows; under the relative verdict it fails on the bull windows, because it does not beat the market beta it is carrying. An agent that passes both would have to be both profitable and index-beating in every one of six regimes on eight seeds, which on these histories no reference agent is. One structural caveat: a cash agent has a positive excess series on every bear window, so on a range consisting only of bear windows the relative verdict would pass `hold` on `passk`. It would still not be eligible, because the pooled deflated Sharpe and the bootstrap are computed on its raw returns, which are zero. A flat position in a falling market satisfies the relative reliability test without supplying any edge, which is why the relative mode changes the per-run series only and leaves the edge gates absolute.

Command. The producing command is listed once in Section A. It writes one record per (dataset, agent), 81 in total, each carrying the command, the benchmark id, the per-window pass vectors under both verdicts, the windows gained and lost, the cell counts, and the gate statistics. The per-window vectors are produced by the kernel’s own `per_run_passes`, not by a reimplementaion.

5.8 A mandate declared at submission

The design direction that Section 3.3 and the limitations state, a mandate declared at submission and certified against instead of selected by the host, is built, and this section reports what it does to the reference field. A submission may carry a typed declaration, `DeclaredMandate`, with four variants: `AbsoluteReturn` (the shipped default verdict, restated); `RelativeTo{benchmark_id}` (the benchmark-relative verdict of Section 5.7 against a named agent); `DrawdownCapped{max_per_run_drawdown}` (the never-catastrophic verdict of Section 5.4, with the per-run bound declared by the submitter); and `OutperformBuyAndHold`, an explicitly excess-return mandate resolving to `RelativeTo` buy-and-hold. The former wire spelling `long_only_beta` remains readable for old artifacts but is not used for new declarations: outperformance is not a claim that an agent is beta-tracking or long-only. This release therefore implements declared *verdict selection*, not certification that a portfolio fulfils a beta-tracking or other portfolio-purpose mandate; a defensible tracking predicate needs an independently specified benchmark, horizon and tolerance. The declaration is optional and defaults to absent.

Mechanism. A declaration selects which reliability question `passk` asks; it never relaxes a gate. Let m_a be agent a ’s declaration and $v(m_a)$ the verdict it resolves to. The agent’s declared-mandate eligibility is the predicate of Equation (5) with only the reliability conjunct exchanged:

$$\text{eligible}_v(a) = \text{DSR} \geq \beta \wedge \text{pass}_v^k \wedge \text{process} \wedge p_{\text{boot}} < \alpha \wedge \text{mandate} \wedge \text{dd}_v, \quad (7)$$

where pass_v^k uses the verdict’s per-run series and aggregation (raw returns under every-run aggregation for `AbsoluteReturn`; excess over the named agent’s same-cell run, every run, for the relative verdicts, with the zero-excess and missing-benchmark refusals of Equation (6) unchanged; raw returns under at-least-one aggregation for `DrawdownCapped`) and dd_v is the declared per-run drawdown bound, vacuously true except under `DrawdownCapped`, where a bound outside $(0, 1]$ is a misdeclaration and fails. The deflated Sharpe, the bootstrap, the process audit and the host’s drawdown mandate in Equation (7) are the same tests on the same raw returns as in Equation (5); a declaration cannot flip any of them, and a unit test asserts that raising the deflation bar refuses every declaration. Both verdicts are recorded on the score (`declared_mandate`, `verdict_applied`, `declared_passed_k`, `declared_mandate_eligible`), and the board row prints both clauses, so buy-and-hold’s row reads “ineligible under declared verdict (relative to buy-and-hold); host-board ineligible” rather than a bare refusal.

Ranking. Eligibility is reported per verdict and the verdicts never mix. The board sorts on Equation (5) under the host’s configuration, byte-identically with or without declarations, and the ordinal

rank counts host-eligible agents only. Declared eligibility is an additional labeled column; among agents that are declared-eligible under the *same* resolved verdict (the agent’s mandate class: same variant, same benchmark id, same bound), a within-class ordinal orders them by deflated Sharpe. Agents under different mandates have answered different questions and are never ordered against each other, or against the all-weather board.

Evidence. The field of Section 5.5, with buy-and-hold carrying the explicit `OutperformBuyAndHold` comparison, hold and momentum declaring `AbsoluteReturn`, and the risk-managed agent declaring `DrawdownCapped` at 0.20. The five luck-floor agents are undeclared, and their 45 records are identical to the undeclared board. Table 8 lists every declared agent on every dataset.

That field has nine agents, one more than the eight-agent field of Table 4, and two rows differ between the two tables for that reason alone. Seven of the nine leading cells agree to four decimals. Daily US indices and 4-hour crypto do not: Table 4 prints buy-and-hold at 0.156 and 0.323 and Table 8 prints 0.0000 on both. The added risk-managed agent supplies a fifth surviving dispersion vote after clone collapse on exactly those two panels, so they measure the field’s own dispersion instead of falling back to the configured 0.5 prior, which raises their bar and drives the deflated Sharpe to zero. This is the mechanism Section 5.3 states and Section 5.6 measures directly across a thirteen-agent field. The consequence is that the largest reference-agent deflated Sharpe in this paper, 0.3230, is conditional on the eight-agent field’s configured fallback and is not a property of the agent alone. The mandate records do not serialize `trials_sr_std_source`, so a reader who wants to check this cell by cell should use the external-rule records of Section 5.6, which do.

Result. No declared agent meets its declared eligibility predicate in any of the 36 declared agent-dataset cases. There is exactly one declared reliability pass: daily-crypto risk-managed. Its score is nevertheless refused by deflation ($DSR = 0.2118$ against $\beta = 0.95$) and the block bootstrap ($p_{boot} = 0.1949$); its declared drawdown is within the 0.20 bound (worst 0.170). This is a more informative refusal than the host verdict because Equation (7) records which reliability question was asked without pretending that a declaration can waive an edge gate.

Buy-and-hold’s `OutperformBuyAndHold` declaration is deliberately self-defeating: it resolves to “beats buy-and-hold in every window”, while buy-and-hold *is* that reference. Its excess series is identically zero, the zero-excess rule of Equation (6) refuses it in all 48 cells on every dataset, and the same holds for any clone. The name now says exactly what the test asks; it does not misdescribe an excess-return test as a beta mandate. The mechanism still delivers legibility, “ineligible under declared verdict (relative to buy-and-hold); host-board ineligible” in place of a blanket refusal, but the reading is that declaration changes what the refusal *says*, not whether it happens: on these histories and gates, stating the mandate admits nobody. Hold and momentum, whose declaration restates the default verdict, fail exactly as the default fails them, including hold: a cash agent breaches nothing, and a zero series is evidence of no edge, not of safety worth certifying.

Command.

```
cargo run --release -p sharpebench-harness \
  --example mandate_eval -- \
  paper/evidence/final/mandate-declaration.jsonl
```

One record per (dataset, agent), 81 in total: 36 declared cases and 45 undeclared luck-floor records. Each carries the command, declaration, applied verdict and both eligibility columns. The declared columns are produced by the kernel’s own `rank_declared`, not by a reimplementation, and the host-verdict columns are byte-identical to the undeclared board.

Table 8: Declared-mandate verdicts for every declared reference agent on all nine frozen datasets. OBH is OutperformBuyAndHold (relative to buy-and-hold), ABS is AbsoluteReturn, and DD .20 is DrawdownCapped at a per-run drawdown of 0.20. DSR and p_{boot} are pooled gates; pass_v^k is the declared reliability leg; Meets is Equation (7). None of the 36 declared agent–dataset cases meets its declared eligibility predicate.

Dataset	Agent	Declared	DSR	p_{boot}	pass_v^k	Meets
US indices 1d	buy-and-hold	OBH	0.0000	0.0020	no	no
US indices 1d	momentum	ABS	0.0000	1.0000	no	no
US indices 1d	hold	ABS	0.0000	1.0000	no	no
US indices 1d	risk-managed	DD .20	0.0000	1.0000	no	no
US indices 1w	buy-and-hold	OBH	0.1781	0.0015	no	no
US indices 1w	momentum	ABS	0.0264	0.0775	no	no
US indices 1w	hold	ABS	0.0003	1.0000	no	no
US indices 1w	risk-managed	DD .20	0.0262	0.0715	no	no
Crypto 1h	buy-and-hold	OBH	0.0000	0.0685	no	no
Crypto 1h	momentum	ABS	0.0000	1.0000	no	no
Crypto 1h	hold	ABS	0.0000	1.0000	no	no
Crypto 1h	risk-managed	DD .20	0.0000	1.0000	no	no
Crypto 4h	buy-and-hold	OBH	0.0000	0.0735	no	no
Crypto 4h	momentum	ABS	0.0000	1.0000	no	no
Crypto 4h	hold	ABS	0.0000	1.0000	no	no
Crypto 4h	risk-managed	DD .20	0.0000	1.0000	no	no
Crypto 1d	buy-and-hold	OBH	0.2505	0.1219	no	no
Crypto 1d	momentum	ABS	0.0051	1.0000	no	no
Crypto 1d	hold	ABS	0.0343	1.0000	no	no
Crypto 1d	risk-managed	DD .20	0.2118	0.1949	yes	no
Crypto 1w	buy-and-hold	OBH	0.1244	0.0795	no	no
Crypto 1w	momentum	ABS	0.2406	0.0435	no	no
Crypto 1w	hold	ABS	0.0044	1.0000	no	no
Crypto 1w	risk-managed	DD .20	0.0057	0.4608	no	no
FX 1d	buy-and-hold	OBH	0.0000	1.0000	no	no
FX 1d	momentum	ABS	0.0000	1.0000	no	no
FX 1d	hold	ABS	0.0000	1.0000	no	no
FX 1d	risk-managed	DD .20	0.0000	1.0000	no	no
Commodities 1d	buy-and-hold	OBH	0.0000	0.1244	no	no
Commodities 1d	momentum	ABS	0.0000	1.0000	no	no
Commodities 1d	hold	ABS	0.0000	1.0000	no	no
Commodities 1d	risk-managed	DD .20	0.0000	1.0000	no	no
Rates 1d	buy-and-hold	OBH	0.0000	0.1754	no	no
Rates 1d	momentum	ABS	0.0000	1.0000	no	no
Rates 1d	hold	ABS	0.0000	1.0000	no	no
Rates 1d	risk-managed	DD .20	0.0000	1.0000	no	no

5.9 The seed leg under a richer execution-noise model

The Reliability paragraph of Section 3 concedes that the seed leg of pass_v^k is a narrow stability check: under the shipped cost model the eight execution seeds vary only the base slippage draw, a few basis points, so every refusal on the paper’s grid is attributable to the window leg. This section adds an opt-in execution-noise model under which the seeds resample fills rather than basis points, re-runs the reference field under both profiles on three daily datasets, and answers two questions the concession leaves open: whether the seed leg ever becomes the refusing leg once execution is genuinely noisy, and how far per-seed Sharpes then disperse.

Definition. Let an order at bar t on symbol i under execution seed s request a change Δ in position value toward its target weight, after the liquidity cap. The model draws three uniforms $u_d, u_f, u_q \sim \mathcal{U}[0, 1)$ from a SplitMix64 stream keyed on the triple, $R(s, t, i)$, so each quantity below is a pure function of (s, t, i) and the base slippage draw of the default model is untouched. Three

effects apply in order.

$$\text{fill bar } \tau = t + \mathbb{I}[u_d < p_d], \quad (8)$$

$$\text{filled value } \Delta_{\text{fill}} = \begin{cases} \varphi \Delta & \text{if } (1 - \varphi) |\Delta| > \kappa \text{NAV}_t, \\ \Delta & \text{otherwise,} \end{cases} \quad \varphi = \varphi_{\min} + (1 - \varphi_{\min}) u_f, \quad (9)$$

$$\text{queue slippage } s_q = u_q \rho_t \min\left(1, \frac{\pi}{\pi_{\text{ref}}}\right), \quad \rho_t = \left| \frac{c_t}{c_{t-1}} - 1 \right|, \quad \pi = \frac{|\Delta_{\text{fill}}|}{\text{NAV}_t}. \quad (10)$$

A fresh order uses Equation (8); a carried order fills at its scheduled bar without a second delay. A delayed order and the unfilled remainder $\Delta - \Delta_{\text{fill}}$ of a partial fill (Equation (9)) are carried to bar $t + 1$ as an order for the same target weight, where they fill at bar $t + 1$'s price before that bar's fresh orders; a fresh order on the same symbol supersedes the carried one, which is cancel-and-replace. The carry floor κ makes a remainder worth less than κ of NAV fill in full, so a carry drains in finitely many bars instead of shrinking geometrically forever. The queue term (Equation (10)) is an adverse move of up to one bar's range, scaled linearly by participation up to π_{ref} , and is added to the base seeded slippage and the square-root impact term before the fill price is formed, with the sign against the trade as before. The frozen datasets carry closes only, so ρ_t is the close-to-close absolute move rather than a high-low range; it is a proxy, and Equation (10) should be read as such. The shipped parameters are $p_d = 0.25$, $\varphi_{\min} = 0.5$, $\kappa = 10^{-3}$ and $\pi_{\text{ref}} = 0.10$; none is calibrated to a venue, and the model is stylized by design.

The model is opt-in. `CostModel` gains a noise: `Option<ExecutionNoise>` field that deserializes to `None` from every existing configuration, and with `None` the fill arithmetic is the historical path in the same order, so the golden fixtures, the native/wasm parity tests and the evidence sweeps are byte-identical. `CostProfile::Realistic` resolves to the typical cost model with `ExecutionNoise::default()`. The carried orders live in the book state and are absent from a serialized snapshot when empty. Given (s, t, i) every quantity is deterministic, and two runs under the same seed replay bit for bit.

Evidence. The field of Section 5.5 (buy-and-hold, momentum, hold, the risk-managed agent, and five luck-floor agents), the same window rule and the same eight execution seeds, on three daily datasets from three market panels: US indices (2513 bars, 3 symbols, six windows of 408 bars, regimes UUUDUU), crypto majors (1000 bars, 5 symbols, six windows of 156, regimes UCUUDD) and FX majors (4157 bars, 5 symbols, six windows of 682, regimes UDCUDU), scored under the default `ScoreConfig` for each dataset's periods per year. Every (window, seed) cell is tested against the per-run bar $\text{PSR}(r, 0) \geq 0.90$ through `sharpebench_core::per_run_passes`, and each agent's verdict is decomposed by leg: a window whose eight sampled execution draws all fail is a *window-leg* refusal, while a window with some sampled draws passing and some failing is a *seed-leg* refusal. The decomposition is exhaustive for this eight-draw experiment: pass^k holds if and only if neither leg refuses. For each (agent, window) the across-seed sample standard deviation of per-run annualized Sharpe is recorded, and for scale the across-window standard deviation of the per-window mean Sharpe. Table 9 lists the three trading reference agents on every dataset under both profiles; hold never trades, so both profiles give it identically zero returns, and the five luck-floor agents are summarized in the text because their seed dispersion is not an execution effect.

Result. The realistic profile makes the seed leg measure something, and it still never refuses. Pooled over the three trading agents and six windows, the across-seed standard deviation of per-run annualized Sharpe rises from 0.0015, 0.0010 and 0.0024 under the default profile on US indices, crypto and FX to 0.1451, 0.2422 and 0.1026 under the realistic profile, roughly two orders of magnitude; the largest single-window value is 0.5969 (momentum on crypto). For the two agents that trade every bar the ratio of per-window seed dispersion to across-window dispersion is now between 0.13 and 0.26, where under the default it was below one percent; for buy-and-hold, which trades once and then drifts, the seed dispersion stays below 0.06 because the model charges the fills, and buy-and-hold has almost none. The refusing leg is nevertheless *windows* in all eighteen rows of Table 9 under both profiles: for the reference agents not one window is mixed. The one change in a

Table 9: The seed leg of pass^k under the default cost model and under `CostProfile::Realistic` (Equations (8) to (10)), three daily datasets, eight seeds, six windows. Windows: P when all eight seeds pass the per-run bar, M when the seeds disagree, a period when all eight fail. Leg is the refusing leg. Seed std is the across-seed sample standard deviation of per-run annualized Sharpe, mean and maximum over the six windows; window std is the across-window standard deviation of the per-window mean Sharpe; mean SR is the mean per-run annualized Sharpe over the 48 cells. Closest fail is the highest PSR of any cell in an all-fail window, the nearest the window leg came to admitting a window. No agent passes pass^k or is rank-eligible under either profile.

Dataset	Agent	Profile	Windows (0 to 5)	Leg	Seed std		Window std	Mean SR	Closest fail
					mean	max			
US indices 1d	buy-and-hold	default	P . . . P .	windows	0.0001	0.0002	0.8153	+0.984	0.879
US indices 1d	buy-and-hold	realistic	P . . . P .	windows	0.0063	0.0092	0.8073	+0.966	0.878
US indices 1d	momentum	default		windows	0.0025	0.0034	0.8849	−0.535	0.723
US indices 1d	momentum	realistic		windows	0.2210	0.3142	0.9366	−0.897	0.674
US indices 1d	risk-managed	default		windows	0.0019	0.0033	0.8637	−0.163	0.861
US indices 1d	risk-managed	realistic		windows	0.2079	0.3446	0.8081	−0.801	0.688
Crypto 1d	buy-and-hold	default	P . . P . .	windows	0.0001	0.0001	1.7941	+0.716	0.890
Crypto 1d	buy-and-hold	realistic	P . . P . .	windows	0.0332	0.0536	1.8182	+0.679	0.882
Crypto 1d	momentum	default		windows	0.0019	0.0024	1.7856	−0.631	0.898
Crypto 1d	momentum	realistic		windows	0.4354	0.5969	2.0989	−1.982	0.687
Crypto 1d	risk-managed	default	P . . P . .	windows	0.0009	0.0013	1.6159	+0.574	0.854
Crypto 1d	risk-managed	realistic	P	windows	0.2579	0.4267	1.9319	−0.311	0.785
FX majors 1d	buy-and-hold	default		windows	0.0001	0.0001	0.6502	−0.070	0.774
FX majors 1d	buy-and-hold	realistic		windows	0.0074	0.0114	0.6442	−0.084	0.775
FX majors 1d	momentum	default		windows	0.0034	0.0044	0.8563	−4.142	0.000
FX majors 1d	momentum	realistic		windows	0.1742	0.2265	0.7068	−4.031	0.000
FX majors 1d	risk-managed	default		windows	0.0036	0.0109	0.5786	−1.917	0.035
FX majors 1d	risk-managed	realistic		windows	0.1263	0.1745	0.5882	−2.345	0.033

window verdict, risk-managed on crypto losing its second passing window (P . . P . . to P), is a whole-window shift, all eight seeds failing where all eight had passed, driven by the level effect of the noise on a rebalancing agent (mean Sharpe from +0.574 to −0.311), not by a seed-dependent verdict. The level effect is itself the largest visible consequence: momentum loses 0.36, 1.35 and gains 0.11 of annualized Sharpe on the three datasets, risk-managed loses 0.64, 0.88 and 0.43, and buy-and-hold loses 0.02, 0.04 and 0.01. Among the luck-floor agents the two mixed windows that exist on the grid (luck-floor-00 and luck-floor-01 on crypto under the default profile, two and one of eight seeds passing window 0) disappear under the realistic profile, where every luck-floor window fails on all eight seeds; but the luck floor’s seed dispersion of 0.14 to 0.31 under the default profile is a policy effect, each `RandomAgent` being re-seeded per run, and says nothing about execution noise.

Interpretation. The seed leg does not refuse because the per-window PSRs of these agents sit far from the bar on both sides. The Closest fail column shows the nearest any all-fail window came to 0.90: 0.898 for momentum on crypto under the default, where the across-seed PSR range on that window was 0.0014, and 0.687 under the realistic profile, where the range had grown to 0.464 but the level had moved away from the bar faster than the dispersion grew. On the passing side the tightest margin is buy-and-hold’s second passing window on crypto, whose lowest seed PSR is 0.9012 under the default and 0.9086 under the realistic profile, with the across-seed PSR range on that window at 0.0001 and 0.0402; the realistic profile brings the seed range to the same order as that margin, and the window stays unanimous by roughly one range width. That is the honest reading: under the shipped model the seed leg cannot flip a window because the seeds barely move the PSR, and under the realistic model it could flip a window that sits within a few hundredths of the bar, but on this grid no window does. The paper’s statement that every refusal is a window-leg refusal therefore survives the richer noise model on three panels, with the qualification that the seed leg is now a live test rather than a formality.

Three caveats bound the claim. The noise model is stylized: its four parameters are set by hand, the range proxy is close-to-close because the datasets carry no high or low, and carried orders fill at the next close, so the model has the structure of delay, partial fill and queue loss but not the magnitudes of any particular venue. The across-seed standard deviations are estimated from eight seeds, for which the relative standard error of a sample standard deviation is about 27 percent under an IID normal approximation (Section 3.6), so the two-orders-of-magnitude contrast is robust and the third digit is not. And three daily datasets from three panels are enough to show that the seed leg can be made to measure execution and still not be the refusing leg for these agents, not enough to say it never is; an agent whose per-window Sharpe sits near the bar, which none of the author-written references does, is exactly the case in which the realistic seed leg would decide, and that is the case the arena is meant to see.

5.10 Robustness under perturbation

The self-audit’s adversarial-input attack records its own limit: fragility that no submitted run exercises is invisible to a deterministic kernel. The harness now generates perturbed datasets whose bar-to-bar moves are nudged within the empirical range of the original series, a constraint asserted per bar, so every perturbed path is one the market could have printed. Scoring a field across perturbations reports each agent’s spread between best and worst per-run PSR. On weekly US indices the committed report gives a spread of 0.0928 for the risk-managed agent and 0.0534 for buy-and-hold. Both are far larger than the pre-repair pooled figures, because averaging aligned execution seeds within a window removes the replicate noise a concatenated track averaged over, leaving a per-run PSR that responds to the perturbation. They are also of the same order as each other, so the committed report on this panel separates nothing: a factor of 1.7 between two agents, neither of which is designed to be fragile, is not a diagnosis. The separation the diagnostic exists to produce is pinned by a unit test on synthetic data rather than by the committed report: a deliberately fragile agent that latches onto an exact price value shows more than twice buy-and-hold’s spread, and the test (`cargo test -p sharpebench-harness perturb`, Section A) asserts that separation on every commit. On the real panel the diagnostic reports spreads; it is the synthetic test, not the committed report, that establishes the separation is detectable at all.

5.11 A synthetic pass witness: the acceptance region is nonempty

Universal refusal is compatible with two hypotheses: the gates discriminate, or the eligibility conjunction is jointly unsatisfiable at the shipped defaults on these histories. The reference field cannot separate them, so a witness was constructed. A family of synthetic agents with a controlled injected edge, per-period returns drawn with true Sharpe s by construction, was scored through the shipped default configuration on two window geometries matching the real datasets’ window rule: six 77-bar windows at 52 periods per year, and six 409-bar windows at 252 periods per year. Before the edge sweep, a separate five-agent zero-edge calibration field fixes the measured dispersion (including its precommitted floor); that value is then serialized as an exogenous fixed bar for every witness point. The witness and its display controls therefore cannot set their own DSR threshold.

The acceptance region is nonempty and its boundary is located to within one grid step (Figure 3). On the weekly geometry the witness becomes rank-eligible at an injected per-period Sharpe of 0.40 (annualized 2.88); on the daily geometry at 0.25 per period (annualized 3.97). Under the exogenously calibrated bar, pass^k is the binding gate on both geometries. DSR clears from the first nonzero sampled edge, $s = 0.05$, on the daily geometry (1.0000) and from $s = 0.10$ on the weekly geometry (0.9984, after 0.5097 at $s = 0.05$), and reaches 1.0000 from $s = 0.15$ upward on both; what moves the onset is the per-run leg, which requires every one of 48 window-times-seed runs to clear a marginal per-run PSR of 0.90 and does not do so until 0.25 daily and 0.40 weekly. The weekly onset is the later of the two because its windows are 77 bars against 409. Both onsets are grid points on a sweep in steps of 0.05 under one common-random-number draw. The experiment therefore locates an interval, not a point: eligibility first appears in $(0.35, 0.40]$ weekly and $(0.20, 0.25]$ daily. The sweep is not replicated over independent noise seeds, so those intervals carry unquantified sampling

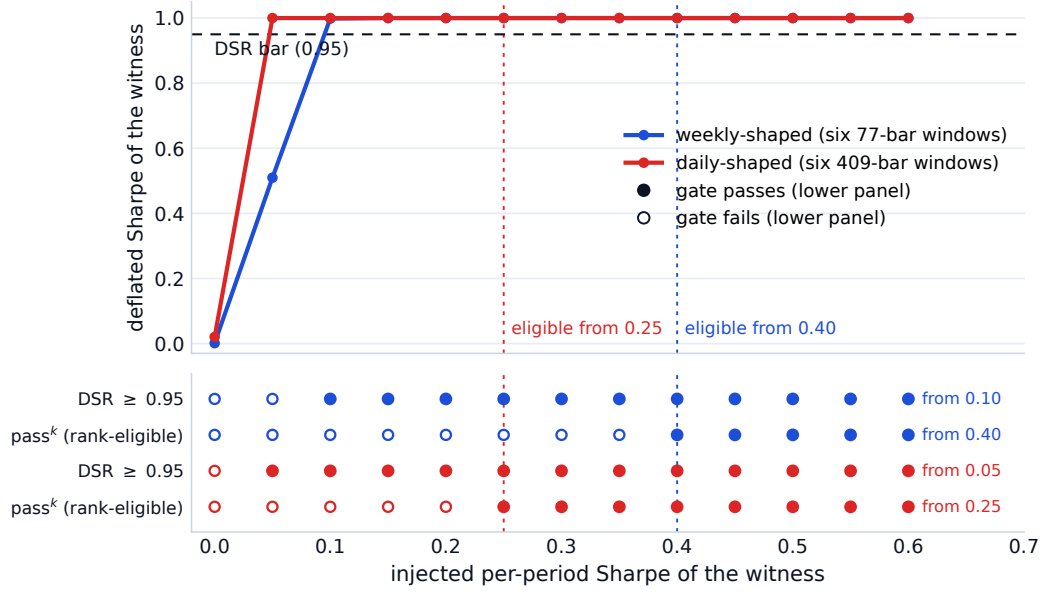


Figure 3: The pass-witness boundary, computed from the committed witness records by the evidence-figure script. Top: the witness’s deflated Sharpe against its injected per-period Sharpe on the two window geometries, with the 0.95 bar; the dotted verticals mark the onset of rank eligibility, 0.40 on the weekly geometry and 0.25 on the daily. Bottom: the two gate outcomes at every injected edge, filled where the gate passes. DSR clears from the first nonzero sampled edge, $s = 0.05$, on the daily geometry and from $s = 0.10$ on the weekly, so pass^k is the gate that binds at both onsets: it turns on at 0.25 daily and 0.40 weekly, which are the onsets marked above.

uncertainty. One earlier draw is on record: the frozen predecessor of this field, generated under the collided seeds described in Section 7, placed both onsets one grid step lower, at 0.35 weekly and 0.20 daily, under the same floored bar. These boundaries reflect the $\sqrt{n-1}$ behavior of Equation (1). The witness turns the paper’s central negative into a calibration: on window lengths of one to two years, the shipped defaults admit the sampled witness only between annualized Sharpe 2.9 and 4. An operator who wants a lower bar must lengthen the windows or lower the per-run bar explicitly in configuration. Because the producer aborts if the deflated Sharpe decreases or eligibility closes after opening, “boundary” here means the first point of a monotone sampled acceptance set and not a local crossing between unrelated draws.

5.12 The falsification leg

A benchmark whose gates are strict can still be wrong in the other direction, by rewarding noise, so the luck floor is scored everywhere. The original floor was fully invested in random weights each period, and on a one-symbol universe the normalized weight is deterministically 1.0: on the rates dataset all five “random” agents were bit-identical clones of buy-and-hold, sharing its raw return to the last digit, so that construction was not a control there. The floor now draws a random gross exposure per period on one-symbol universes (flat half the time, otherwise a uniform long weight), a regression test pins the non-degeneracy, and the rates evidence is generated under that construction; all multi-symbol datasets reproduce byte-identically, since their path is untouched.

Under the corrected floor, on every dataset the luck floor’s best agent has a lower raw return than the best reference agent, with zero exceptions in the nine dataset-timeframe combinations. The operational path also applies the precommitted annualized dispersion floor and the observable field-size floor before deflation.

The grid’s nominal $N = 1$ corner no longer disables deflation. Ranking observes eight submitted streams, so Equation (3) raises the effective count to eight before adding any declared search history. On weekly crypto the best random agent then scores 0.3105 at host $N = 1$, where Equation (3) has already raised the effective count to eight, and 0.0762 at host $N = 50$; on weekly US indices the corresponding values are 0.0800 and 0.0044. Neither is eligible, and pass^k refuses them independently. On the corrected rates floor no random agent scores above zero: genuinely random exposure on the yield series loses money after costs. The degenerate score the earlier one-symbol construction produced there is recorded with the other protocol corrections in Section 7. Figure 2b plots that corner as a curve: the best zero-skill agent’s operational DSR against the effective trial count, entering at the eight-stream floor and decaying monotonically, so no host setting on this grid recovers an eligible zero-skill score. The counterfactual $N = 1$ score remains useful only through the direct single-agent scoring API, where it is explicitly not a ranked field; a leaderboard cannot make its submitted alternatives disappear by declaring one trial.

5.13 The thousand-agent luck floor

The opening image is a thousand random agents; the original evidence floor used five seeded agents per dataset, which located the floor but not its extreme tail. The experiment runs 1,000 distinct seeds on `us-indices-1d` and `crypto-majors-1d`, under the same windows, eight execution seeds and costs as the evidence sweep. The first five return streams are byte-identical to the shipped floor, but all scores are recomputed with the observable 1,000-entry trial footprint. Three scores distinguish an operational result from a diagnostic: the configured annualized prior, the raw field-measured dispersion with its safety floor deliberately disabled, and that measured dispersion after applying the shipped floor. After execution-seed de-pseudoreplication, the raw diagnostic on daily crypto reaches 0.2500, versus 0.1913 among the first five streams in the same 1,000-entry context. The measured annualized field dispersion is 0.0462, so applying the precommitted 0.5 floor lowers the operational maximum to 0.0012; the configured-prior path is identical. The diagnostic maximum therefore rises to 0.2500 while the operational maximum stays at 0.0012. None of the 2,000 agent-dataset cells is eligible, and no random agent beats the best reference agent on raw return on either dataset. Figure 4 shows both distributions.

5.14 What the evidence supports

Taken together, the findings support one claim and decline another. The claim: applied to reference agents on eight tradable-price datasets across four asset classes, plus one rates-yield stress series, at four bar sizes, the benchmark does not certify any unhedged long-only strategy as safe to hand capital under any of the three reliability verdicts (all-weather, never-catastrophic, or relative to same-window buy-and-hold), and it reports the drawdown that justifies the refusal; the corrected luck floor never beats a reference agent even when expanded to 1,000 agents on two daily datasets; four rules taken from the published literature are refused in all 351 cells of Section 5.6, including under a frictionless cost profile, so the refusal is neither an artifact of the author’s agent designs nor of the cost calibration; all nine self-audit attacks are demoted; the two committed golden fields reproduce byte-identically on Linux, macOS and Windows CI; and a controlled injected edge shows the acceptance region is nonempty between annualized Sharpe 2.9 and 4. The eligibility conclusions are unchanged if the two realism-failing datasets are excluded: the verdict is zero eligible on all seven passing datasets as well, so nothing rests on the failed two. The claim it declines: that any external entity can attain eligibility, and that eligibility, where granted, means deployability. Eligibility certifies a statistically survivable edge under stylized execution on narrow universes, not capacity or breadth (Section 7), and every completed entrant here is author-written. The forward window `window-003` is open but unresolved, so a competing agent with genuine edge remains an outstanding experiment.

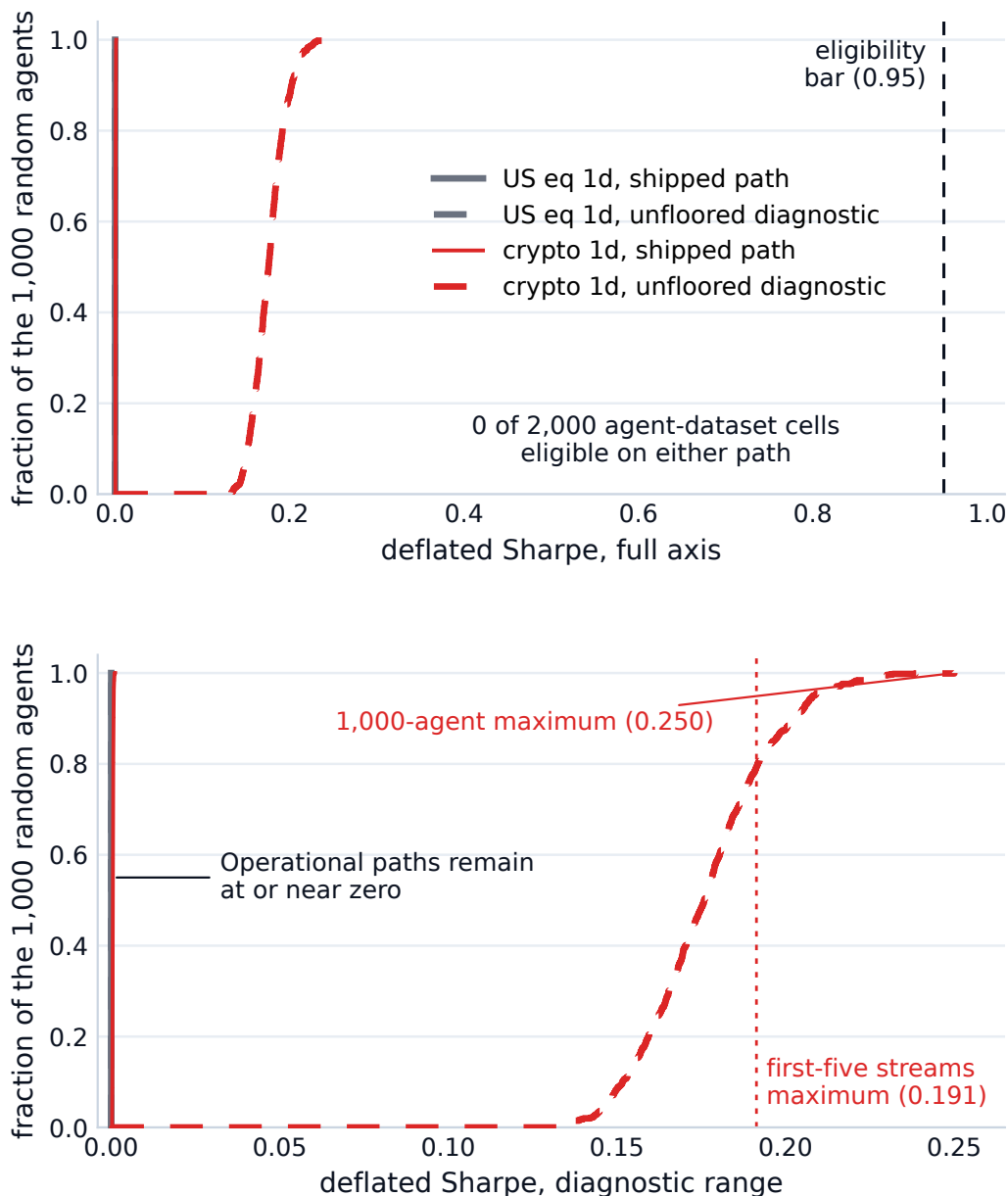


Figure 4: Empirical distributions over the 1,000 random agents on each daily dataset, all scored at the observable 1,000-trial footprint. The solid operational curves apply the shipped annualized dispersion floor; the dashed curves are deliberately unfloored diagnostics. The daily-crypto diagnostic reaches 0.2500, but that value is not used by ranking; its operational maximum is 0.0012. Top: the full axis with the $\beta = 0.95$ bar. Bottom: the diagnostic range. No agent is eligible.

6 Related work

Skill and luck in finance. The distinction the benchmark operationalizes is old. Applying a false-discovery correction to two decades of mutual funds, Barras et al. [2010] find that roughly three quarters had no genuine alpha and that most apparent winners were false positives; Kosowski et al. [2006] reach a more nuanced verdict by bootstrap, and Fama and French [2010] the same conclusion by a different route. Lo [2002] works out the sampling distribution of the Sharpe ratio and shows its square-root-of-time scaling fails off IID returns, which is the statistical ancestor of this paper’s first

finding. Bailey et al. [2014] show that a high backtest Sharpe is trivially achievable by trying enough configurations, Harvey and Liu [2015] turn the observation into a haircut that scales with the number of strategies tried, and Harvey et al. [2016] apply the same discipline to the factor literature. The probabilistic Sharpe ratio of Bailey and López de Prado [2012] and the deflated Sharpe ratio of Bailey and López de Prado [2014] are the instruments this benchmark uses; the Reality Check of White [2000], the superior-predictive-ability test of Hansen [2005] and the step-down procedure of Romano and Wolf [2005] are the field-wide tests it reports. Arnott et al. [2019] set out the reporting protocol a backtest should follow in the era of machine learning, and Section 5 follows it: every Sharpe is reported beside its number of trials, its track length, and the cost profile it was earned under.

Forward and adaptive evaluation. Committing to an evaluation before the data exists is not new, and the benchmark’s intended forward protocol is positioned against that lineage. The M competitions run genuine forward windows on data that does not exist at submission time; M6 in particular scored forecasting and investment performance jointly over a year of live monthly submissions [Makridakis et al., 2025]. On the adaptive-overfitting side, Dwork et al. [2015] give a mechanism that keeps a holdout valid under repeated adaptive queries, and Blum and Hardt [2015] bound leaderboard overfitting from adaptive resubmission; both attack the same disease as deflation by controlling the query channel instead of correcting the statistic. At least one live trading arena for LLM agents exists [Li et al., 2025]. What the chained board adds is independently checkable byte history: scoring rules and entries are bound before operator-declared deadlines, and a re-signed forgery is exposed by the next window’s anchor (Section 4.3). The hash chain does not prove the deadline’s wall time, data custody or prior non-observation. SharpeBench has no completed forward result and no multi-window operating history: its two empty pre-entry records were superseded when the scoring-provenance requirements changed, and its first live window is open but unresolved.

Benchmarks for financial agents. Table 10 positions the benchmark against the boards it is most often compared with, on the axes the principles in Section 2 make load-bearing. Table 11 then draws the same comparison on the axes the rivals hold: every rival that fields agents scores real LLM or learned agents, most present a far richer information environment, StockBench runs single-name equities on a genuinely post-cutoff window with a live board, and the Open FinLLM Leaderboard is hosted under neutral governance. SharpeBench’s row in that table is empty, its empty cells are the limitations of Section 7, and the two tables together state the actual trade: statistical and adversarial properties, each an executed check rather than a guarantee, were bought at the price of field breadth, and the program of Section 7 is to pay that price back without giving up the guarantees. Each mark in both tables records what the cited paper or its public board states, not an inference; the per-cell sources, including the deliberate blanks, are committed beside the evidence (paper/evidence/table-provenance.md).

Each rival’s ranking metric explains one of the axes. FinBen [Xie et al., 2024a] ranks GPT-4 first on its trading task, on the overlapping 95 percent confidence intervals across ten stocks quoted in Section 1, so the board cannot separate its leader from the baseline it is meant to beat. StockBench [Chen et al., 2025] evaluates on a single four-month post-cutoff window and ranks on cumulative return, drawdown and Sortino, with no deflation. QuantBench [Wang et al., 2025] names overfitting as an open problem and ranks pipeline performance with no luck-corrected metric, though its simulation does charge commissions and transaction fees. InvestorBench [Li et al., 2024] evaluates thirteen LLM backbones as decision-making agents across stocks, crypto and ETFs, and ranks on return-based metrics with no deflation or reliability gate. FinRL-Meta [Liu et al., 2022] is scoped differently: it supplies hundreds of gym-style market environments and data pipelines for training RL agents, a benchmark of environments rather than a gated board of submitted agents, so most gate axes do not apply to it; it is in the tables because any serious agent field would plausibly be trained in it. The FINOS Open Financial LLM Leaderboard [FINOS Foundation, 2025] measures financial knowledge, scored on accuracy and F1, and has no trading-performance axis. The reproducibility gap across this field is quantified by the evidence map cited in Section 1 [Xia et al., 2026].

Table 10: Positioning along the axes this benchmark adds. A mark means the cited source states the property; an empty cell means the provenance sheet records no positive evidence, and cells labeled unchecked there are not verified absences (per-cell sources in [paper/evidence/table-provenance.md](#)). τ -bench, a customer-service agent benchmark, appears because it is the construct source for the seed leg of the reliability gate. Read together with Table 11: a comparison drawn only on one’s own axes is advertising.

	Deflates	pass ^k	Process gate	Costs	Deterministic	Forward commit
FinBen [Xie et al., 2024a]						
StockBench [Chen et al., 2025]						
QuantBench [Wang et al., 2025]				✓		
InvestorBench [Li et al., 2024]						
FinRL-Meta [Liu et al., 2022]						
Open FinLLM [FINOS Foundation, 2025]						
τ -bench [Yao et al., 2025]		✓				
SharpeBench (this paper)	✓	✓	✓	✓	✓	✓

Table 11: The same comparison on the axes the rival benchmarks hold. A mark means the cited source states the property; an empty cell means the provenance sheet records no positive evidence, and unchecked cells are not verified absences (per-cell sources in [paper/evidence/table-provenance.md](#)). FinRL-Meta’s near-empty row reflects its different scope, an environment library rather than an agent board. SharpeBench’s empty cells are its limitations (Section 7), and three of them, the missing LLM field, the thin information environment, and the absent live board, are the price paid for the guarantees in Table 10.

	LLM agents	Rich info	Single names	Live board	Post-cutoff run
FinBen [Xie et al., 2024a]	✓	✓			
StockBench [Chen et al., 2025]	✓	✓	✓	✓	✓
QuantBench [Wang et al., 2025]	✓				
InvestorBench [Li et al., 2024]	✓		✓		
FinRL-Meta [Liu et al., 2022]					
Open FinLLM [FINOS Foundation, 2025]	✓	✓		✓	
τ -bench [Yao et al., 2025]	✓				
SharpeBench (this paper)					

Re-scoring a rival field is infrastructure here, not a result. An import adapter is contributed that converts a rival board’s published per-period return series into scoreable submissions, embedding the caveat that imported fields carry no audit trace and unknown trial counts understate deflation, and retaining each column’s own observed date axis so that two series missing different dates are not paired by position. No rival re-scoring has been executed and no demotion of any rival’s leader is claimed: StockBench publishes environment inputs and summary statistics but no per-agent return series, so the experiment awaits either the authors’ raw artifacts or a reproduction with their harness, and until then the adapter is infrastructure, not evidence.

Evaluation as a scientific object. The reliability-versus-capability split that motivates the seed leg of pass^k originates in Yao et al. [2025], which asks whether an agent succeeds on all k attempts rather than on at least one. It is now a theme across agent evaluation: Miller [2024] argues for error bars on every evaluation score, Kapoor et al. [2024] for cost-controlled evaluation and adequate holdout sets, and Rabanser et al. [2026] treats reliability as an axis separate from capability and finds that gains in the latter have not bought the former. The surveys that shape this paper’s integrity section, Reuel et al. [2024] and Bean et al. [2025], find that most benchmarks neither report the statistical significance of their rankings nor allow their results to be replicated, and recommend an operational construct-validity checklist, which Section B answers. A 2026 survey of financial multi-agent evaluation [Nguyen and Pham, 2026] catalogues five recurring failure modes: look-ahead, survivorship, backtest overfitting, transaction-cost neglect and regime-shift blindness. SharpeBench addresses look-ahead through point-in-time observations, selection through deflation, transaction

costs through the simulator and regime dependence through pass^k. It does not close survivorship in the fixed universes, which Section 7 states.

7 Limitations

There is no external ceiling, and that is the paper’s largest gap. Benchmark papers that report a low score bracket it with an entity above the bar: OSWorld reports 12.24 percent for its best model against a 72.36 percent human ceiling [Xie et al., 2024b], GAIA 15 percent for GPT-4 with plugins against 92 percent for human respondents [Mialon et al., 2024], and SWE-bench 1.96 percent for its best model against a human-written patch that resolves every instance [Jimenez et al., 2024]. SharpeBench has none. The synthetic witness of Section 5.11 shows the acceptance region is nonempty on a sampled grid, but we constructed the witness, so it bounds the predicate and does not measure any population. Section 5.6 is the closest available substitute and it did not become a ceiling: four rules specified in the published literature, scored under three cost profiles, produce zero eligible cells and zero pass^k passes in 351 records, with the best of them at a deflated Sharpe of 0.6534 frictionless against the 0.95 bar. A reader therefore still cannot distinguish a well-calibrated gate from a miscalibrated one on external evidence alone.

Two things would supply a ceiling and neither is in reach here. The first is a real track record scored through the same gates: an audited fund or managed account’s per-bar returns over six disjoint out-of-sample windows, which requires proprietary or permission-compatible data. The current frozen bundle does not meet that distribution condition (Section D). The second is a competing agent that clears the bar on the forward arena, which is what window-003 is for and what no window has yet resolved. Until one of the two exists, every eligibility statement in this paper is a statement about the predicate and not about attainability by anyone.

No current-model result is admitted. Every entrant in Section 5 was either written for this repository or implemented here from a published specification: reference agents, a risk-managed control, seeded random agents, a synthetic injected-edge family, and the four external rules of Section 5.6. In the last case the specification is external and the implementation is ours, which removes the objection that the field encodes only its author’s ideas and does not remove the objection that it encodes only its author’s code. A model-agent path is implemented: SharpeArena runs the point-in-time environment and canonical fail-closed parser, writes a complete append-only field journal, and compiles it across an artifact boundary into the submissions SharpeBench evaluates. Its scheduler, sharding, resume, provenance and fault semantics are tested with deterministic doubles. None of that is an agent result. Every empirical claim in this paper therefore remains about the protocol’s behavior on entrants whose properties are known by construction, not about the population the benchmark is built to evaluate.

The window rule bounds the lookback an entrant may use. The observation carries twenty trailing closes and each run starts fresh at its window boundary, so a rule needing L closes is blind for the first $L - 20$ bars of every window (Section 5.6). The canonical trend rules of the published literature are stated over ten to twelve months, and at the shipped window rule faber-10m cannot form its signal on any bar of the six windows on daily crypto, 4-hour crypto or hourly crypto, and is blind for 190 of every 408-bar daily US window. Any entrant whose edge lives at that horizon is therefore refused by the experiment design and not by the gates, which makes the six-window rule a load-bearing choice this paper has not ablated. Lengthening the windows trades that away against the number of out-of-sample regimes the reliability leg can test, and the trade is not evaluated here.

Named experiments still open. No current-model field has completed or produced an admitted performance artifact. The implemented path records model, runtime, scaffold, decoding, cadence and fault metadata, but those are protocol capabilities rather than results. A future field must predeclare its target population and treat materially different inference configurations as separate arms. The forward arena has one open window, window-003, bound to the attested v0.9.0 release binary; it

carries no commitment yet and, at the manuscript cut, has not reached its declared resolution epoch, so no forward result is reported here (Section 4.3). Until neutral hosting and key custody exist the board remains a single-author artifact. The StockBench reproduction through the import adapter still awaits per-agent return series (Section 6).

The forecast report has no empirical field here. The deterministic cross-product tutorial in Section 3.4 proves file compatibility and exercises the scorer, not forecast skill. A superseded convenience-sample lifecycle pilot remains archived so that its protocol history is not rewritten, but its obsolete checkpoint scores are excluded from the paper, model comparison and trading rank. Even a future field inherits three limits. Exact common support prevents pair-specific cherry-picking but may leave a small or unrepresentative intersection; resolution-time blocks encode a declared dependence unit and do not prove longer dependence absent; and proper forecast scores measure predictive distributions, not execution quality, transaction costs, risk controls or portfolio value. Those are the reasons the report cannot enter Equation (5).

Coverage and observation. Single-name equities are absent for want of a keyless feed, gold because the public daily series were withdrawn, and two of nine datasets fail the realism battery and ship with the verdict recorded. The crypto and index symbol sets were selected with hindsight from assets that survived into the collection period; the fixed universes therefore do not close survivorship bias. The nine dataset-timeframe combinations reduce to roughly five independent market panels, all containing at least one labeled bear window; only the longer panels contain both the 2020 and 2022 drawdowns. The refusals of Section 5.3 are properties of the gates on these selected histories rather than of the gates alone. The rates file contains the 10-year constant-maturity yield itself, not a tradable Treasury price or total-return index; treating its percentage level as a price makes that row a simulator diagnostic rather than evidence about a bond strategy, and none of the eligibility conclusions depends on retaining it. The agent sees twenty trailing closes; the protocol carries fields for fundamentals and news and the simulator leaves them empty, so this is a thinner information environment than FinBen or StockBench present.

The reliability gate is strict by construction. The default certifies profitability in every regime, and Sections 5.3 and 5.4 show that both it and a per-run drawdown bound refuse every long-only agent on every dataset containing a downturn; Section 5.7 reaches the same verdict under the benchmark-relative question. Which of the three is the right definition of safe for a given mandate is the operator’s choice, now exposed as configuration, and Section 3.3 states why the conservative default ships anyway; the paper does not claim to have found the right default. A typed mandate declared at submission is built (Section 5.8), but a relative declaration cannot be definitively verified from a lone trajectory: the verifier fails closed with an explicit aligned-field-benchmark requirement. On short windows the per-run PSR bar is weak because the statistic scales with $\sqrt{n-1}$, which is a property of the PSR and not a choice. Eligibility, where it is ever granted, certifies a statistically survivable edge under stylized execution on narrow universes; it is not a deployability, capacity, or breadth certificate.

Honor-system inputs and field-level gaming. Declared trial counts remain unverifiable for arbitrary submissions. The generated-strategy path closes only the part it observes: every raw candidate produced inside that harness is counted before validation or deduplication and the selected candidate alone reaches a disjoint test interval. It cannot count private or pretraining search, so Section 3.6 keeps declared- N advisory outside that path. The measured dispersion is set by whoever populates the field, and its near-clone Sybil vector is now closed: ranking collapses clone clusters to one vote before measuring, and the ninth audit case asserts on every commit that 200 sock puppets no longer move the bar (Section 4.2). The configured dispersion floor bounds, rather than closes, the adversary who submits genuinely dissimilar low-dispersion streams: such a field can still relax an honest measured dispersion down to the precommitted prior, roughly a tenfold reduction on the committed fields, and the guarantee is only that it stops there. Clone collapse also interacts with seed averaging, and on five of the nine panels it merges honest agents and forces the configured fallback (Section 4.2). Distinct verified identities and per-identity entry caps remain unbuilt. Backtest verdicts for pretrained models

are advisory by protocol rule because contamination, not multiple testing, is the binding threat there (Section 3.6); the shape-level contamination probe that would test this, scoring the same agent on real and moment-matched surrogate series, is specified but not run.

Built and not yet public. The arena lifecycle is built and one window, window-003, is open, but it holds no commitment and no completed board exists. A daily workflow advances deadline states, and there is no hosted intake and no live board UI; scoring, signing and publication remain operator-driven, so the league is not yet a running service and the governance policy of Section 4.4 is a policy the operator applies rather than one a third party can enforce. External agents are sandboxed by container isolation only, which is the subject of the paragraph below; multi-tenant hosting of untrusted submissions remains unbuilt. No completed real entrant triggers the process gate in the evidence field: that gate is exercised by synthetic demonstrations and all nine adversarial audit cases, so its enforcement is tested but its field prevalence is unknown. The perturbation diagnostic of Section 5.10 narrows the self-audit’s blind spot but samples perturbations rather than searching adversarially, so a fragility that survives sampling can still hide. The calibration module’s epistemic leg reads agreement among an agent’s confidence streams as low uncertainty, so unanimous-but-wrong signals read as agreement and the leg is informative only when it reads high. Every experiment other than Section 5.6 uses the typical cost profile. Table 6 reports all three, and it bounds the cost objection without removing it: no cell is eligible under any profile, so the universal refusal is not produced by the cost calibration, but the profile does move the deflation bar itself, from 1.2576 frictionless to 27.5397 typical on hourly crypto, because the measured field dispersion widens as the cost drag on zero-skill agents grows. The frictionless column is also not a clean counterfactual: it removes fees, slippage, impact and financing at once, and the stressed column does not apply `CostProfile::WorstCase`’s two-bar decision delay, because the backtest driver consumes the cost model only. The falsification leg still inherits the narrower version of this limitation: every zero-skill track in the 1,000-agent floor has a negative raw mean return under the typical profile, so the floor demonstrates that costs plus deflation keep noise below the bar, and the cost-free lucky tail was not rerun at 1,000 agents.

The live sandbox boundary is exercised in one Docker configuration. Everything the container sandbox decides before launch is a pure function and is tested without Docker. The boundary itself is not: the dedicated sandbox workflow pulls `alpine:3.22`, resolves it to an immutable digest, refuses a mutable fixture and runs exactly the ten ignored live tests on a Docker-enabled Linux runner. The in-container readiness probe checks a non-root user, zero Linux capabilities, no-new-privileges, loopback-only network visibility, a read-only root, a bounded writable temporary filesystem and a `noexec` temporary mount. Separate probes attempt egress to cloud metadata, the wider link-local range, all three RFC1918 blocks, the public internet and a live listener on the host loopback; each must be classified as a policy denial within the configured time bound, rather than merely failing after a timeout. A production-spawn smoke test verifies that the fixture actually starts, and a 32 MiB live cgroup test must end with Docker reporting `OOMKilled=true`, which the harness maps to a non-retryable agent resource fault before removing the named container.

These observations establish less than a general containment proof. They cover one hosted-runner and daemon configuration, one benign POSIX fixture and the named probes; no hostile trading entrant or multi-tenant service has been evaluated, and they do not rule out a Docker or kernel escape. Ordinary suites report the ten live-boundary tests as ignored unless their Docker prerequisites are explicitly enabled; only the dedicated workflow counts them as evidence. That distinction is deliberate: an earlier runtime skip and a weak “no orders” assertion could both pass when the container never started. The current tests require a live prerequisite, a specific denial class or an observable Docker state, so missing infrastructure cannot masquerade as evidence. A harness terminated by uncatchable `SIGKILL` between spawn and cleanup can leave a stopped, inspectable named container; normal completion and object destruction both remove it, but the implementation does not claim leak-freedom against termination that prevents either path.

Protocol corrections and their scope. The paper records four limitations of earlier protocol configurations: a deflation prior in the wrong units (Section 5.2); an incorrect documentation claim

that field-wide significance tests gated rank (Section 3.1); a private HMAC chain described as publicly verifiable (Section 4.3); and a one-symbol luck floor that degenerated into a buy-and-hold clone, whose degeneracy produced a spurious rates-panel DSR of 0.993 that the corrected construction removes (Section 5.12). The current protocol and evidence address these specific defects. They do not turn a reference-agent study into evidence of deployability, field breadth or a live league. A fifth change belongs beside them without belonging to the list, because it is not a correction: closing the agent wire contract in v0.11.0 broke entrants written against v0.10.x, shipped with no deprecation window, and was recorded in the v0.11.0 changelog entry under a fixes heading that understated it (Sections 3 and 4.4). It is released and cannot be unshipped, and the migration is to drop the offending key or move it into one of the two fields that accept free content.

Repairs landed after this paper’s evidence was frozen. An independent audit of the 0.18.4 tree in September 2026 found defects in code that produced numbers reported here, and the repairs merged after the evidence was frozen. Three change what the current engine would compute. The standardized-moment estimators mixed finite-sample normalizations, so every probabilistic and deflated Sharpe value in this paper, together with its standard error and interval, was computed under the superseded convention; refreshing the two deterministic score fixtures under the corrected estimator moves those statistics and leaves their eligibility and ordering unchanged. The simulator let a zero target cross through flat into a short position, which changes simulated trajectories and removes residual trading that the momentum entrant’s no-op events had counted. That entrant also ignored its declared lookback, and rescored the synthetic fixture under the repair reorders two ineligible display rows and changes field-relative diagnostics while leaving all four eligibility and ordinal values as they were. A fourth divergence is recorded rather than resolved: on the commodities panel the current simulator reconstructs two finite-Sharpe streams where the frozen default carries eight measured-dispersion rows, so that panel now has insufficient measured support and falls back to the configured prior. A fifth affected the synthetic witness of Section 5.11, and it is the one case where the committed evidence was regenerated rather than left frozen. The producer derived every return stream from an expression that overlapped the calibration-member index with the execution-seed index, so the five zero-edge calibrators that fix the dispersion bar shared draws: the frozen predecessor of the committed field was drawn under those collided seeds and carried thirteen distinct streams per window where the design calls for forty. The corrected producer derives each stream from an unambiguous domain, member, window and execution-seed tuple, and the field reported in Section 5.11 is a declared rerun under it, performed on the same machine and recorded as a new experiment rather than as a correction of what the predecessor measured; every one of its forty streams per window is distinct in each of the three synthetic populations. The crossing point moved by exactly one grid step on both geometries: from a per-period Sharpe of 0.35 to 0.40 on the weekly geometry (annualized 2.52 to 2.88) and from 0.20 to 0.25 on the daily geometry (annualized 3.17 to 3.97). The calibrated dispersion bar did not move, because in both draws the measured zero-edge dispersion fell below the precommitted annualized floor of 0.5 and the floor set the bar; what moved was the per-run pass^k leg, which needs one more step of injected edge before all 48 runs clear the marginal PSR. The rerun executed the current kernel, which also carries the corrected standardized-moment estimators, so the one-step shift is the joint effect of the independent draw and the corrected estimators and is not attributed to the seeds alone. The interpretation is unchanged: the acceptance region is nonempty, and its boundary lies one grid step higher than the predecessor placed it. No other table in this paper has been regenerated. Rerunning any other field would be a new empirical run rather than a correction to what the frozen evidence measured, so those numbers are reported here with their provenance stated instead of being silently refreshed. The repository’s audit checklist tracks the disposition of every finding, including those still open.

A second repair round, and what it does and does not move. The paragraph above is the ledger of one audit and not of every repair since. A further independent review in September 2026 produced a second round, merged after the evidence above was frozen, and none of it regenerates a number reported here. Most of that round hardened refusal paths that the evidence never exercised: withholding the whole field-wide snooping family when its shared bootstrap boundary fails, deriving the

effective seed width from validated run keys and refusing a contradictory flag, refusing unsupported seed-bootstrap intervals at both language boundaries, preserving a baseline score and pass rate as unavailable rather than as a numeric zero, refusing unobserved or overflowing candidate utilities, retaining each imported column's own observed date axis, naming statistical disqualification reasons instead of returning an empty list, and validating computed quantities in the checked deflation path before a floor or a saturated cumulative normal can conceal an overflow. The evidence above was produced by the example sweep rather than by these boundaries, so none of the tables moves; what changes is what the current engine would do with an input the frozen evidence never contained. Five later findings round out the same review, and two of those are worth naming because they change what the engine reports rather than what it computes. The disqualification taxonomy published `SelectionUnavailable` as a hard reason while the scorer never gated on the selection axis at all, so a submission whose candidate set refused could be printed as disqualified beside a rank-eligible verdict; the reason is now advisory, which is the label consistent with the scorer, and no eligibility, ordering or score in this paper depends on it because that taxonomy is a legibility layer that computes nothing new. The percentile-selection refusal path reported an alpha warning unconditionally, so a refusal caused by too few observations blamed an alpha that was never the problem; the flag now reports where alpha sits and nothing else. The remaining three are internal: a checkpoint schema version raised so that a checkpoint predating the persisted per-round budget is refused rather than granted a fresh round, a corrected label on four finite-computation checks that validate an agent mean rather than a field maximum, and the deletion of two contract legs in one resume path that compared a contract's fields against themselves. None of them touches the scoring kernel, the goldens, the examples or the frozen evidence.

Capabilities added after this revision, and their bounds. The special functions behind PSR and the deflation bar were measured against a 60-digit reference and the substitution was then rejected, which is a disposition rather than a deferral. A `stats` implementation is closer to the reference on almost every argument the kernel evaluates, improving the worst absolute error by three to seven orders of magnitude, and neither implementation is correctly rounded. It is rejected on a property the accuracy work did not measure: the two code goldens regenerated under it reproduce on the platform that produced them and fail on the other two supported targets, while the shipped bodies pass on all three. A disclosed and bounded error identical everywhere is compatible with a benchmark that promises byte-identical rescoring; a smaller error that varies by target is not, because it makes two honest operators disagree about the same submission. The shipped error sits orders of magnitude below every bar the kernel tests against, so nothing reported here turns on it, and the exact returned bits are pinned in a test so a silent change fails before it reaches a golden fixture. The image preflight of Section 4 is opt in and raw-byte scoped, and a negative result is not an absence-of-memorization result. The host-observed model gateway routes an entrant's model calls over a host-owned pipe with host-owned credentials, bounded envelopes, a reservation taken and persisted before each dispatch and an append-only money journal; no networked transport ships in the repository, the operator supplies one, and what the host observes is a provider's own usage report rather than a verified invoice. The Lean model of Section 9 proves only its declared scope, and the seeded property and mutation tests that connect it to the implementation are evidence of coverage rather than formal verification of the Rust code. None of these mechanisms is claimed to be first of its kind or universally secure, and none of them was exercised by an empirical field here.

8 Conclusion

SharpeBench is an evaluation protocol that treats a trading agent's score the way a statistician would: deflated for search, required to survive every seed and window, zeroed on process violations, and tested against a serial-correlation-preserving null, with every number recomputable from committed data by a listed command. Applied to reference agents across nine frozen dataset-timeframe combinations, the gates refused everything and gave a reason naming the gate and the quantity each time; a controlled injected edge shows the bar sits near an annualized Sharpe of 3 on these window geometries, so the bar is strict rather than unsatisfiable. A benchmark-relative verdict, which asks only for outperformance

of same-window buy-and-hold, refuses every entrant as well, so the refusal is not an artifact of the all-weather mandate. Four rules taken from the published literature, scored under each of three cost profiles, refuse in all 351 cells, and removing every trading friction admits nobody either, so the refusal is not an artifact of the cost calibration; what the cost profile does move is the deflation bar itself. Risk management proved frequency-dependent in the same field, improving worst-window drawdown on seven of nine panels and amplifying it on the two where the trend signal churns, which a per-window score keeps visible. The thousand-agent floor leaves every zero-skill entrant ineligible, and the ninth audit case, which showed that field-measured deflation was not Sybil-resistant, is now demoted by the clone collapse that closed that gap. One of the paper's four numbered findings is a correction to the benchmark itself, and three further defects found while writing are recorded above; the general lesson of that first finding, that a deflation gate is only as correct as its unit conventions, applies to any scorer that mixes annualized literature constants with per-period statistics. The report-only prospective surface extends the same refusal discipline to forecast contracts, proper scores, common support and dependence-aware comparison without allowing forecast quality to buy trading rank. The protocol and kernel are the contribution. What the paper still lacks is a ceiling it did not build itself: no external entrant and no published rule has cleared the bar, so attainability rests on a synthetic witness. A current-model field and the resolution of window-003, the open forward window bound to the attested v0.9.0 scorer artifact, are the two tests that would change that.

9 Reproducibility statement

The Rust workspace lists fourteen members: twelve packages under `crates/`, `xtask` and the reference-agent example. The twelve packages under `crates/` are eleven library packages plus the `sharpebench` CLI binary; a thirteenth crate directory, the published PyO3 binding, is excluded from the workspace because `maturin` builds it separately. The project code is licensed MIT and Apache-2.0 and published to `crates.io`, `npm` and `PyPI`, with a pinned toolchain and lockfile; the code license does not license the frozen data (Section D). Each of the twelve workspace package roots under `crates/` declares `forbid(unsafe_code)`. The excluded PyO3 package is the disclosed exception: generated FFI glue expands to unsafe operations and its root does not make that declaration. Every historical dataset in Table 1 is committed with a SHA-256 sidecar; its fetch or derivation script identifies how the exact evidence bytes were obtained, but the online FRED fetches are not a permission to repeat the service use under current terms. Every figure in this paper is produced by a committed script from the kernel's own formulas, and every table is produced by the commands in Section A from the committed data. Golden fixtures pin the full ranked output of two fields at full floating-point precision and those two fixtures are checked on Linux, macOS and Windows. A WebAssembly parity test checks the corresponding native and WebAssembly results on its configured CI host; it is not evidence for every input on every platform. The self-audit fails if any of its nine claimed defenses regresses, and the Sybil case additionally fails if the attack stops reproducing with clone collapse switched off.

The empirical and engineering versions are different strata. The performance records, tables and figures are the v0.9.0 evidence snapshot described in Section A; they were not regenerated after the code hardening documented in this revision. The exact current source and artifact scope is delegated to `paper/evidence/provenance.json` rather than copied into prose as a version or test count that will drift. Continuous integration covers release rehearsal, formatting and lints, workspace and Python tests, cross-platform determinism, the live container boundary, self-audit, realism, supply-chain policy, documentation and paper provenance; separate workflows cover `npm` and the forward-arena state. `paper-provenance` runs `check-provenance.py`, so a manifest that no longer matches the tree fails rather than sitting stale. The live Docker tests run only in the explicit sandbox job and have the narrower interpretation stated in Section 7.

The model-agent build preserves the evaluator boundary. `SharpeArena` owns point-in-time interaction, strict decision validation, execution and append-only cell journals; validated file artifacts then cross into `SharpeBench` for field-level scoring. `SharpeBench` does not depend on the full `SharpeArena` package. The compatibility examples and bridge are tested without admitting a model result, and

any future field must bind its model artifact, inference runtime, prompt and scaffold, decoding configuration, raw responses and fault classes. A model’s output is not covered by the environment’s byte-determinism claim.

The prospective path uses the same package boundary. SharpeArena freezes contracts and writes append-only `sharpe.forecast-evidence.v1` ledgers; SharpeBench revalidates and scores the raw predictions independently. The deterministic tutorial fixes two synthetic producer files and the complete expected report. The Lean project under `formal/` models common-support intersection, forecast-report rank isolation, the ordered fixed-point Holm step and the finite-bootstrap plus-one correction. These theorems are about a deliberately small mathematical model, not an extraction of Rust semantics; executable conformance and mutation tests are the bridge to the implementation. No current-model result is admitted through either product surface.

No large language model is a component of the benchmark or of any result in this paper; the kernel has no judge. Language models were used to draft and edit prose; no citation in the bibliography was generated by one, and every arXiv entry was checked against its abstract page.

10 Ethics and broader impact

A benchmark that certifies trading agents has a shorter path to harm than a benchmark that certifies code edits. An eligibility verdict is a fitness claim about software that moves money, and a badge is the artifact a retail marketer would reach for first. This section states what the verdict does and does not assert, the conflicts the artifact carries, and why it is released anyway.

What eligibility is not. Eligibility certifies that an entrant’s edge survived five statistical and process tests on frozen historical bars under a stylized execution model. It is not investment advice, not a prediction of live performance, not a capacity, breadth or liquidity certificate, and not a statement about any instrument. It is conditional on the configuration digest, the field the entrant was scored against and the histories in the window: Sections 5.3 and 5.6 show the same agent’s deflated Sharpe moving when the field composition changes and when the cost profile changes, so a verdict quoted without its configuration is not a verdict. The board prints the refusing gate and the quantity that produced it, which is the only form in which we intend a score to be quoted.

Dual use. The same properties that make the protocol hard to game make it attractive to cite selectively. An entrant that clears the gate on one dataset can advertise the pass and omit the eight refusals; an operator can lower `dsr_bar` or the per-run bar and publish the resulting admissions under the SharpeBench name. Two mechanisms bound this and neither closes it. The scoring configuration is serialized and digest-committed on every board, so a relaxed bar is visible to anyone who verifies the board, and the signed chain of Section 4.3 makes a board that omits its own configuration unverifiable. What we cannot prevent is a screenshot. The countervailing consideration is that the failure mode this work exists to correct, a leaderboard that ranks the best overfitter and calls it the best agent, is already in circulation, and an artifact that refuses is a weaker marketing instrument than one that ranks.

Asymmetry of errors. The gate is calibrated to under-certify. A false positive hands capital to a strategy whose edge was noise; a false negative costs an entrant a badge. We chose the direction deliberately, and the cost of that choice falls on entrants rather than on the public. It is still a cost: Section 5.6 shows four published rules and a risk-managed control refused on every panel, and a strict gate that refuses everything is not automatically a correct gate. The synthetic witness of Section 5.11 is what stops that argument from being unfalsifiable, and it is the only demonstration in this paper that anything can pass.

Conflict of interest. The policy permits the host to compete on a future board, but no model-entry board has been published. Every offline entrant reported here is either written for this repository or implemented here from a published specification, the operator holds the Ed25519 signing key, and

intake, sealed scoring, signing and publication are all operator-driven (Sections 4.3 and 4.4). The chain proves that a published history was not altered; it does not prove that the host was disinterested when it produced that history. Neutral hosting, third-party key custody and an external dispute procedure are named in Section 4.4 as unbuilt.

Data and licensing. All nine datasets were collected without API keys, but keyless access did not establish redistribution permission. The three US index series are copyrighted and marked pre-approval required; no approval is recorded. FRED’s federal-source series carry its “Public Domain: Citation Requested” category but remain subject to the service-wide prohibited uses, and the reviewed Binance pages grant access without granting this repository redistribution rights. The code is MIT or Apache-2.0; that license does not extend to the observations. This is an unresolved data-distribution blocker, not a claim that the historical evidence used different bytes. Section D gives the series-level record and remediation choices.

Market effects. The benchmark does not trade. It replays frozen historical bars through a simulator, so scoring an agent has no market impact and consumes no venue capacity. The forward arena of Section 4.3 scores on data that postdates the commitment, and it likewise scores rather than executes: no window has ever placed an order.

Release calculus. The tension is that publishing a certification protocol for capital-allocating agents makes certification easier to claim as well as easier to check. We release because the checking half is verifiable by anyone and the claiming half is not: every number here recomputes from committed data by a listed command, the nine self-audit attacks run on every commit, and a reader who doubts a verdict can regenerate it. A protocol that only its author can execute would carry the same dual-use exposure and none of the accountability.

References

- Robert D. Arnott, Campbell R. Harvey, and Harry Markowitz. A backtesting protocol in the era of Machine Learning. *The Journal of Financial Data Science*, 1(1):64–74, 2019. doi: 10.3905/jfds.2019.1.064. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3275654.
- David H. Bailey and Marcos López de Prado. The Sharpe ratio efficient frontier. *Journal of Risk*, 15(2):3–44, 2012. doi: 10.21314/JOR.2012.255.
- David H. Bailey and Marcos López de Prado. The Deflated Sharpe ratio: Correcting for selection bias, Backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107, 2014. doi: 10.3905/jpm.2014.40.5.094.
- David H. Bailey, Jonathan M. Borwein, Marcos López de Prado, and Qiji Jim Zhu. Pseudo-mathematics and Financial charlatanism: The effects of Backtest overfitting on out-of-sample performance. *Notices of the American Mathematical Society*, 61(5):458–471, 2014. doi: 10.1090/noti1105. URL <https://www.ams.org/notices/201405/rnoti-p458.pdf>.
- Laurent Barras, Olivier Scaillet, and Russ Wermers. False discoveries in Mutual Fund performance: Measuring luck in estimated alphas. *The Journal of Finance*, 65(1):179–216, 2010. doi: 10.1111/j.1540-6261.2009.01527.x.
- Andrew M. Bean, Ryan Othniel Kearns, Angelika Romanou, Franziska Sofia Hafner, Harry Mayne, Jan Batzner, Negar Foroutan, Chris Schmitz, Karolina Korgul, Hunar Batra, Oishi Deb, Emma Beharry, Cornelius Emde, Thomas Foster, Anna Gausen, María Grandury, Simeng Han, Valentin Hofmann, Lujain Ibrahim, Hazel Kim, Hannah Rose Kirk, Fangru Lin, Gabrielle Kaili-May Liu, Lennart Luettgau, Jabez Magomere, Jonathan Rystrom, Anna Sotnikova, Yushi Yang, Yilun Zhao, Adel Bibi, Antoine Bosselut, Ronald Clark, Arman Cohan, Jakob Foerster, Yarin Gal, Scott A. Hale, Inioluwa Deborah Raji, Christopher Summerfield, Philip H. S. Torr, Cozmin Ududec, Luc Rocher, and Adam Mahdi. Measuring what matters: Construct validity in large language model

- benchmarks. In *Advances in Neural Information Processing Systems 38, Datasets and Benchmarks Track*, 2025. URL <https://arxiv.org/abs/2511.04703>.
- Avrim Blum and Moritz Hardt. The Ladder: A reliable leaderboard for machine learning competitions. In *Proceedings of the 32nd International Conference on Machine Learning*, 2015. URL <https://arxiv.org/abs/1502.04585>.
- William Brock, Josef Lakonishok, and Blake LeBaron. Simple technical trading rules and the stochastic properties of stock returns. *The Journal of Finance*, 47(5):1731–1764, 1992. doi: 10.1111/j.1540-6261.1992.tb04681.x.
- Yanxu Chen, Zijun Yao, Yantao Liu, Amy Xin, Jin Ye, Jianing Yu, Lei Hou, and Juanzi Li. StockBench: Can LLM agents trade stocks profitably in real-world markets?, 2025. URL <https://arxiv.org/abs/2510.02209>.
- Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960. doi: 10.1177/001316446002000104.
- Rama Cont. Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001. doi: 10.1080/713665670.
- Gabriel Deza, Adelin Travers, Colin Rowat, and Nicolas Papernot. Interpretability in safety-critical Financial trading systems, 2021. URL <https://arxiv.org/abs/2109.15112>.
- Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015. doi: 10.1126/science.aaa9375.
- Mebane T. Faber. A quantitative approach to tactical asset allocation. *The Journal of Wealth Management*, 9(4):69–79, 2007. doi: 10.3905/jwm.2007.674809.
- Curtis M. Faith. *Way of the Turtle: The Secret Methods that Turned Ordinary People into Legendary Traders*. McGraw-Hill, New York, 2007.
- Eugene F. Fama and Kenneth R. French. Luck versus skill in the Cross-Section of Mutual Fund returns. *The Journal of Finance*, 65(5):1915–1947, 2010. doi: 10.1111/j.1540-6261.2010.01598.x.
- FINOS Foundation. Open Financial LLM leaderboard. <https://huggingface.co/spaces/finosfoundation/Open-Financial-LLM-Leaderboard>, 2025. Financial-knowledge axis only; no trading-performance axis.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12): 86–92, 2021. doi: 10.1145/3458723.
- Tilman Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437.
- Peter Reinhard Hansen. A test for Superior Predictive Ability. *Journal of Business & Economic Statistics*, 23(4):365–380, 2005. doi: 10.1198/073500105000000063.
- Campbell R. Harvey and Yan Liu. Backtesting. *The Journal of Portfolio Management*, 42(1): 13–28, 2015. doi: 10.3905/jpm.2015.42.1.013. URL https://people.duke.edu/~charvey/Research/Published_Papers/P120_Backtesting.PDF.
- Campbell R. Harvey, Yan Liu, and Heqing Zhu. ... and the cross-section of expected returns. *The Review of Financial Studies*, 29(1):5–68, 2016. doi: 10.1093/rfs/hhv059.

- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter, 2024. URL <https://arxiv.org/abs/2407.01502>.
- Perry J. Kaufman. *Trading Systems and Methods*. Wiley, Hoboken, New Jersey, fifth edition, 2013.
- M. G. Kendall. A new measure of rank correlation. *Biometrika*, 30(1/2):81–93, 1938. doi: 10.1093/biomet/30.1-2.81.
- M. G. Kendall. The treatment of ties in ranking problems. *Biometrika*, 33(3):239–251, 1945. doi: 10.1093/biomet/33.3.239.
- M. G. Kendall. *Rank Correlation Methods*. Charles Griffin, London, 1948.
- Robert Kosowski, Allan Timmermann, Russ Wermers, and Hal White. Can mutual fund “stars” really pick stocks? New evidence from a bootstrap analysis. *The Journal of Finance*, 61(6):2551–2595, 2006. doi: 10.1111/j.1540-6261.2006.01015.x.
- Changlun Li, Yao Shi, Yuyu Luo, and Nan Tang. Will LLMs be professional at fund investment? DeepFund: A live arena perspective, 2025. URL <https://arxiv.org/abs/2503.18313>.
- Haohang Li, Yupeng Cao, Yangyang Yu, Shashidhar Reddy Javaji, Zhiyang Deng, Yueru He, Yuechen Jiang, Zining Zhu, Koduvayur Subbalakshmi, Guojun Xiong, Jimin Huang, Lingfei Qian, Xueqing Peng, Qianqian Xie, and Jordan W. Suchow. InvestorBench: A benchmark for financial decision-making tasks with LLM-based agent, 2024. URL <https://arxiv.org/abs/2412.18174>.
- Xiao-Yang Liu, Ziyi Xia, Jingyang Rui, Jiechao Gao, Hongyang Yang, Ming Zhu, Christina Dan Wang, Zhaoran Wang, and Jian Guo. FinRL-Meta: Market environments and benchmarks for data-driven financial reinforcement learning. In *Advances in Neural Information Processing Systems 35, Datasets and Benchmarks Track*, 2022. URL <https://arxiv.org/abs/2211.03107>.
- Andrew W. Lo. The statistics of Sharpe ratios. *Financial Analysts Journal*, 58(4):36–52, 2002. doi: 10.2469/faj.v58.n4.2453.
- Spyros Makridakis, Evangelos Spiliotis, Ross Hollyman, Fotios Petropoulos, Norman Swanson, and Anil Gaba. The M6 forecasting competition: Bridging the gap between forecasting and investment decisions. *International Journal of Forecasting*, 41(4):1315–1354, 2025. doi: 10.1016/j.ijforecast.2024.11.002.
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general AI assistants. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2311.12983>.
- Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations, 2024. URL <https://arxiv.org/abs/2411.00640>.
- Phat Nguyen and Thang Pham. Toward reliable evaluation of LLM-based Financial multi-agent systems: Taxonomy, coordination primacy, and cost awareness. In *DMO-FinTech Workshop, PAKDD 2026*, 2026. URL <https://arxiv.org/abs/2603.27539>.
- William J. O’Neil. *How to Make Money in Stocks: A Winning System in Good Times and Bad*. McGraw-Hill, New York, fourth edition, 2009.
- Dimitris N. Politis and Joseph P. Romano. The Stationary Bootstrap. *Journal of the American Statistical Association*, 89(428):1303–1313, 1994. doi: 10.1080/01621459.1994.10476870.

- Stephan Rabanser, Sayash Kapoor, Peter Kirgis, Kangheng Liu, Saiteja Utpala, and Arvind Narayanan. Towards a science of AI agent reliability, 2026. URL <https://arxiv.org/abs/2602.16666>. Accepted at ICML 2026.
- Anka Reuel, Amelia Hardy, Chandler Smith, Max Lamparth, Malcolm Hardy, and Mykel J. Kochenderfer. BetterBench: Assessing AI benchmarks, uncovering issues, and establishing best practices. In *Advances in Neural Information Processing Systems 37*, 2024. URL <https://arxiv.org/abs/2411.12990>. Spotlight.
- Joseph P. Romano and Michael Wolf. Stepwise multiple testing as formalized Data Snooping. *Econometrica*, 73(4):1237–1282, 2005. doi: 10.1111/j.1468-0262.2005.00615.x.
- Shivalika Singh, Yiyang Nan, Alex Wang, Daniel D’Souza, Sayash Kapoor, Ahmet Üstün, Sanmi Koyejo, Yuntian Deng, Shayne Longpre, Noah A. Smith, Beyza Ermis, Marzieh Fadaee, and Sara Hooker. The Leaderboard illusion, 2025. URL <https://arxiv.org/abs/2504.20879>.
- Frank A. Sortino and Robert van der Meer. Downside risk. *The Journal of Portfolio Management*, 17(4):27–31, 1991. doi: 10.3905/jpm.1991.409343.
- C. Spearman. The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101, 1904. doi: 10.2307/1412159.
- Ryan Sullivan, Allan Timmermann, and Halbert White. Data-snooping, technical trading rule performance, and the bootstrap. *The Journal of Finance*, 54(5):1647–1691, 1999. doi: 10.1111/0022-1082.00163.
- Saizhuo Wang, Hao Kong, Jiadong Guo, Fengrui Hua, Yiyang Qi, Wanyun Zhou, Jiahao Zheng, Xinyu Wang, Lionel M. Ni, and Jian Guo. QuantBench: Benchmarking AI methods for quantitative investment, 2025. URL <https://arxiv.org/abs/2504.18600>.
- Halbert White. A Reality Check for Data Snooping. *Econometrica*, 68(5):1097–1126, 2000. doi: 10.1111/1468-0262.00152. URL <https://users.ssc.wisc.edu/~bhansen/718/White2000.pdf>.
- J. Welles Wilder. *New Concepts in Technical Trading Systems*. Trend Research, Greensboro, North Carolina, 1978.
- Yihan Xia, Panpan You, Taotao Wang, Fang Liu, Han Qi, Xiaoxiao Wu, and Shengli Zhang. Agentic trading: When LLM agents meet Financial markets, 2026. URL <https://arxiv.org/abs/2605.19337>.
- Qianqian Xie, Weiguang Han, Zhengyu Chen, Ruoyu Xiang, Xiao Zhang, Yueru He, Mengxi Xiao, Dong Li, Yongfu Dai, Duanyu Feng, Yijing Xu, Haoqiang Kang, Ziyang Kuang, Chenhan Yuan, Kailai Yang, Zheheng Luo, Tianlin Zhang, Zhiwei Liu, Guojun Xiong, Zhiyang Deng, Yuechen Jiang, Zhiyuan Yao, Haohang Li, Yangyang Yu, Gang Hu, Jiajia Huang, Xiao-Yang Liu, Alejandro Lopez-Lira, Benyou Wang, Yanzhao Lai, Hao Wang, Min Peng, Sophia Ananiadou, and Jimin Huang. FinBen: A holistic Financial benchmark for large language models. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024a. URL <https://arxiv.org/abs/2402.12659>.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024b. URL <https://arxiv.org/abs/2404.07972>.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2406.12045>.

A Commands that produce every number

All commands run from the repository root. Commands that reproduce empirical numbers use the v0.9.0 evidence snapshot; commands that test the newer contracts use the current engineering tree. The kernel is deterministic, so its golden outputs reproduce on the three CI targets (Linux, macOS, Windows).

Version. Findings 1 and 2 were first observed at v0.2.1, and Table 3 was computed against v0.2.1 deliberately because it documents that kernel’s unit error. The units fix shipped in v0.3.0. The admitted result artifacts are from v0.9.0, a cut and published tag, and were regenerated from the content-hashed source snapshot containing corrected execution-seed pooling, the observed-field trial floor, the measured-dispersion floor, clone collapse and the strict forward-window schema. Historical package labels are not substituted for that exact content snapshot. Later checkpoint, trajectory, transport, forecast-quality, formal and diagnostic-schema changes are software claims exercised below, not a new empirical validation. The manifest identifies their exact engineering tree; the result JSONL and figures were not regenerated, and no current-model result is admitted.

Content provenance.

```
python paper/src/make-provenance.py
-> paper/evidence/provenance.json
python paper/src/check-provenance.py
```

The manifest records the commit at which it was generated, whether that tree was dirty, a content hash over the Rust, Python, published-schema, producer and frozen-data source snapshot, and the SHA-256 digest of every admitted result JSONL and figure. Source text is canonicalized from CRLF to LF before hashing so the same Git content has one identity on Windows and Unix; result artifacts remain byte-exact. Build outputs, virtual environments, bytecode caches and dependency trees are excluded by path component, so generated Rust files under a `target/` directory cannot enter the source identity. Incomplete model fields remain excluded because they contain no admitted model result. The externally specified rule field of Section 5.6 is an ordinary recorded artifact, not a separately quoted exception. The validator recomputes every digest and the aggregate source hash, re-expands both recorded scopes to catch files added or removed since generation, and exits nonzero on any discrepancy. It also checks the two recorded generation fields, and what it enforces is narrower than what they record. The dirty flag is enforced outright: a manifest generated from a worktree with uncommitted changes describes source bytes that are in no commit, and the validator refuses it. The recorded head cannot be checked for equality with the current commit, because a manifest cannot contain the hash of the commit that will contain it; the enforced condition is ancestry, that the recorded commit is reachable from the checked-out one. That requires the recorded object, so the `paper-provenance` job checks out full history; in a shallow clone the validator fails and names the remedy rather than skipping the check. Outside a Git worktree, as in an unpacked source archive, the ancestry leg is reported as not enforced and the dirty flag still is. No digest is copied into this appendix: the manifest is the machine-readable authority, which removes the circularity that arose when this source file quoted a hash over a snapshot containing itself.

The shipped demonstration (Figure 1a).

```
sharpebench score suites/example_submissions.json
```

The deflation curve (Figure 1b) and the demotion chart.

```
python paper/src/make-figures.py
```

The script reimplements the kernel’s PSR, expected-maximum-Sharpe and normal-quantile functions in Python and states that it does; the figures are the kernel’s formulas, not an independent computation.

The self-audit (Table 2).

```
sharpebench audit
```

The command reports nine defended cases and no known gaps. A claimed defense that regresses fails its commit-time tests; the Sybil case also fails if the attack stops reproducing with `dedup_clones_for_measured_sr_std` switched off.

The realized index Sharpe ratios quoted in Section 5.2.

```
python paper/evidence/index_sharpe.py
```

Prints the symbol, the number of per-period returns and the annualized Sharpe ratio of each of the three series in `data/us-indices-1d.csv`, using the kernel's convention: mean over sample standard deviation with one degree of freedom on simple per-period returns, a zero risk-free rate, scaled by $\sqrt{252}$.

Data realism (Table 1).

```
sharpebench realism --data data/<dataset>.csv
```

Regenerating any dataset and checking its hash.

```
python scripts/data/fetch_crypto.py --interval 1h --check
python scripts/data/fetch_fred.py fx --check
python scripts/data/derive_weekly.py --check
```

The evidence sweep (Table 4, Section 5.2, Section 5.3, Section 5.4, Section 5.12).

```
cargo run --release -p sharpebench-harness \
  --example evidence_sweep -- out.jsonl [dataset] [dsr_bar]
```

One JSON record per (dataset, configuration, agent), 512 per dataset. The grid, seeds, window rule, cost profile, effective trial count, dispersion-floor settings, both SPA values and both reliability configurations are emitted on every record. The optional arguments restrict a run to one dataset or one deflation bar so the grid can be split across processes; because every seed is pinned, the four bar shards together carry the same set of records with the same contents as a serial run. `paper/evidence/assemble_sweep.py` validates that assembly and writes the original JSON lines back out in a fixed grid-key order, which is not the serial producer's score-rank order, so the assembled file is identical to a serial run record for record and not byte for byte. `analyze.py` reduces the records into the tables.

The risk-managed agent (Section 5.5), its deflation N-sensitivity, and the perturbation report (Section 5.10).

```
cargo run --release -p sharpebench-harness \
  --example risk_managed_eval -- out.jsonl
```

Runs the risk-managed agent beside buy-and-hold and the luck floor on all nine datasets with the documented defaults, both reliability verdicts per row; emits the `n_sensitivity` records for weekly US indices at $N \in \{7, 10, 25, 50, 100\}$; and appends the perturbation spread report for weekly US indices. The fragile-agent perturbation separation quoted in Section 5.10 is produced by a unit test on synthetic data, not by this command:

```
cargo test -p sharpebench-harness perturb
```

The externally specified rules and the cost sweep (Table 5, Table 6).

```
cargo run --release -p sharpebench-harness \
  --example external_rules_eval -- \
  paper/evidence/final/external-rules.jsonl [dataset]
```

Scores a thirteen-agent field (buy-and-hold, momentum, hold, the risk-managed agent, the four published rules, and five luck-floor agents) on all nine datasets under the frictionless, typical and stressed profiles (`CostProfile::None`, `CostProfile::Typical` and `CostProfile::WorstCase`; the records carry the reported names), at the default `ScoreConfig` for each dataset's periods per year: 351 records, one per (dataset, cost profile, agent). Each record carries the command, the rule's published source, the closes the rule requires, the blind-bar count implied by the window rule, the dispersion source, the annualized deflation bar and the full gate vector. The four rules are implemented inside the example from their published statements; no parameter is fitted to any dataset here. The optional argument restricts the run to one dataset.

The synthetic pass witness (Section 5.11).

```
cargo run --release -p sharpebench-harness \
  --example pass_witness -- pass-witness.jsonl
```

Sweeps a family of injected-edge agents (known per-period Sharpe by construction) on two window geometries, and writes one record per (shape, edge, agent). A separate five-agent zero-edge calibration field is run first; its floored per-period dispersion is frozen as the exogenous DSR bar for the complete witness sweep, so the candidate and display-control records cannot move their own threshold. The witness uses common random numbers across the edge grid and the producer aborts if DSR decreases or eligibility closes after opening, so "boundary" means the first point of a monotone sampled acceptance set rather than a local crossing between unrelated draws.

The evidence figures (Figure 2a, Figure 2b, Figure 3, Figure 4).

```
python paper/src/make-evidence-figures.py
python paper/src/make-evidence-figures.py pass-witness
python paper/src/make-evidence-figures.py luck-floor-1000
```

The first command writes all four figures; the two selectors regenerate the pass-witness boundary and the thousand-agent floor figures alone. Every plotted result and data-dependent crossing is a reduction over the committed records. The horizontal 0.95 line is the protocol's declared eligibility bar rather than an estimated result.

The thousand-agent luck floor (Section 5.13).

```
cargo run --release -p sharpebench-harness \
  --example luck_floor_1000 -- \
  paper/evidence/final/luck-floor-1000.jsonl
```

The file contains one row per agent and one explicit summary row for each of the two daily datasets. The first five return streams reproduce the shipped floor inside the same run; all 1,000 streams are scored with the observable trial count fixed at 1,000. Every row distinguishes the configured-prior score, the raw measured-dispersion diagnostic with the floor intentionally disabled, and the operational measured path after the annualized floor is applied.

The relative-mandate verdict (Table 7).

```
cargo run --release -p sharpebench-harness \
  --example relative_mandate_eval -- \
  paper/evidence/final/relative-mandate.jsonl
```

Scores the risk-managed field on all nine datasets under the default verdict and under `PassMode::RelativeToBenchmark` with buy-and-hold as the benchmark: 81 records, one per (dataset, agent), each carrying the per-window pass vectors under both verdicts, the windows gained and lost, and the cell counts. The same verdict is selected on any field with `sharpebench run -pass-mode relative-to-benchmark [-benchmark-agent <id>]` or the same flags on `sharpebench score`.

The forward arena (Section 4.3).

```
sharpebench arena init <dir>
sharpebench arena open <dir> <window> <commit_deadline> <reveal_epoch> \
  --scorer-artifact-sha256 <hex> \
  [--config <score_config.json>] [--sealed-eval-salt-sha256 <hex>]
sharpebench commit <agent_id> <target_window> <artifact_digest> <salt> \
  > commitment.json
sharpebench arena commit <dir> <window> <commitment.json>
sharpebench arena advance <dir> <epoch>
sharpebench arena score <dir> <window> <dataset> <entries.json>
sharpebench arena publish <dir> <window> <key>
sharpebench arena verify <dir> [--pubkey <hex>]
```

The opening command refuses to create a window without the exact scorer artifact digest. The standalone `commit` command produces the commitment document that `arena commit` registers; the latter does not accept the agent, artifact and salt as positional fields.

The prospective forecast report and formal model (Section 3.4).

```
cargo run -q -p sharpebench -- forecast-quality \
  examples/forecast-quality/fixtures/agent-alpha.json \
  examples/forecast-quality/fixtures/agent-beta.json \
  --bootstrap-samples 400 --seed 23 --confidence 0.9 \
  --alpha 0.05 --bins 5 --json
cd formal && lake build
```

The first command reproduces the complete committed report from two deterministic SharpeArena ledgers, including exact common support over twelve contracts in six resolution-time blocks, late-revision exclusion, calibration, block-bootstrap comparison and Holm adjustment. The same command over `fixtures/withheld/`, the first eight of those contracts in two blocks, reproduces the companion report in which inference is withheld with its reason recorded (Section 3.4). The second checks the separate abstract proofs of common-support membership, trading-rank isolation, Holm monotonicity and boundedness, and finite-bootstrap count bounds. Neither command produces an empirical model result, and the Lean project is not an extraction of the Rust implementation.

Capturing and strictly replaying a trajectory.

```
sharpebench capture <agent> <trajectory.json> [--data <csv>]
sharpebench verify-trajectory <trajectory.json> [--data <csv>]
```

The second command is strict by default: it verifies the data, cost, engine, optional runner, ordered-window and ordered-seed contract and requires the complete run matrix before replay. `-allow-unbound-trajectory` is the explicitly labeled legacy or cross-version regrade, not a fall-back used after strict verification fails.

Importing a rival field.

```
sharpebench import csv <dir-or-file> --out subs.json [--trials N]
sharpebench score subs.json
```

The pre-floor experiment quoted in Section 5.2. The DSR 0.984 weekly and 0.000 daily pair predates the dispersion floor and is read from the retained artifacts in `paper/evidence/after-v0.3.0/`, which are kept as the record of that intermediate kernel and are not used for any current result.

The clone-collapse regression tests (Section 4.2).

```
cargo test -p sharpebench-harness --test evidence_fields_no_clone_merges
```

Rebuilds every committed evidence field twice: once over the streams as submitted, asserting zero merges at the collapse threshold, and once over the seed-averaged pooled streams that ranking actually clusters, asserting the surviving vote count implies the `trials_sr_std_source` stamped on each committed default cell.

The checks added in this revision (Section 3.2, Section 3.4, Section 3.5, Section 3.8, Section 3, Section 4).

```
cargo test -p sharpebench-core    process
cargo test -p sharpebench-core    entrants
cargo test -p sharpebench-core    evidence_coverage
cargo test -p sharpebench-stats   agreement
cargo test -p sharpebench-stats   dissent
cargo test -p sharpebench-protocol --test schema_drift
cargo test -p sharpebench-core    forecast
cd formal && lake build
```

The first covers the lifecycle ordering check, including the case that a passing risk evaluation on one instrument does not authorize an order on another. The third is the audit that fails when a new evidence field is neither covered nor excluded with a reason. The last checks the published JSON Schemas against the Rust wire types in both directions. None of these produces a number reported as a result; they are the executable form of the claims in the named sections. The full local suite on the audited engineering tree is

```
cargo test --workspace
```

The manifest and continuous-integration records, rather than a copied count in prose, identify the exact current test surface.

The live container boundary (Section 7).

```
docker pull alpine:3.22
SHARPEBENCH_SANDBOX_FIXTURE=<digest-pinned image> \
cargo test -p sharpebench-arena -- --ignored --nocapture
```

The ten live-boundary tests are `#[ignore]`d, so this is the invocation that runs them; the sandbox continuous-integration job runs exactly this after resolving the fixture to a digest and refusing an unpinned one. They cover the in-container hardening probe, six remote egress classes, host-loopback egress with a live-listener prerequisite, production spawn and a real cgroup OOM verdict. On a machine with no Docker daemon the tests report as ignored rather than as passes, which prevents missing infrastructure from being counted as evidence.

Golden fixtures and the parity test.

```
cargo test -p sharpebench-core --test golden_scores
cargo test -p sharpebench-sim  --test golden_input
cargo test -p sharpebench-wasm --test native_parity
```

Regeneration requires `SHARPEBENCH_UPDATE_GOLDEN=1` and refuses to run when CI is set.

Signing and verifying a board.

```
sharpebench sign    subs.json <hmac-key> board.json --ed25519 <secret>
sharpebench verify board.json --public
sharpebench verify board.json --pubkey <hex>
```

The second form verifies the Ed25519 chain from the document's embedded public key; it establishes internal consistency under that key, not the host's identity. The third pins the expected public key out of band and rejects a board re-signed under a different one. The positional HMAC key is shared: anyone who can verify with it can also forge, so the HMAC chain is suitable only for private integrity among key-holders.

B Construct validity checklist

Bean et al. [2025] review 445 benchmarks and recommend that a benchmark paper answer an operational checklist. Each answer points to the section that supports it.

Define the phenomenon.

Yes. The phenomenon is skill that survives five eligibility gates, each of which must also be estimable: deflation for the number of trials, pass^k across seeds and windows, process discipline, a bootstrap null and the configured mandate. Section 2 states it and Equation (5) operationalizes it exactly.

Measure only the phenomenon.

Partly. The eligibility predicate measures the phenomenon and nothing else. The scorer also reports fourteen statistics that do not gate, and Section 3.3 separates the two so a reader can tell which number moved the rank.

Construct representative datasets.

Partly. Eight tradable-price datasets across four asset classes, plus one rates-yield stress series, use four bar sizes and are scored for realism before use (Section C.2), with provenance in Section D. The coverage gaps, unresolved distribution rights, two realism failures and twenty-close observation are stated in Section 7.

Acknowledge limitations of reused datasets.

Yes. Every dataset's source, license, caveats and failing stylized facts are in Table 1 and the data scripts. The commodities file keeps a negative oil close as published and documents the opt-out.

Prepare for contamination.

Yes, at the mechanism level. Sealed held-out datasets under a published hash, a canary token, and a masked view that defeats ticker memorization (Section C.4). A forward commitment binds artifact bytes before an operator-declared deadline, while wall time, custody and prior non-observation remain trust assumptions (Section 4.3). The forward league that would exercise it has not run.

Use statistical methods.

Yes. Every rank is gated on a deflated Sharpe with its number of trials, a block bootstrap with a fixed seed, and a per-run reliability test; field-wide Reality Check, SPA and step-down are reported; a bootstrap confidence interval on the DSR marks tied entries as tied, and an interval that cannot be estimated is reported as unavailable rather than as a numeric no-skill value. The annualization convention is stated once in Section 3.1 because getting it wrong was the first finding.

Conduct error analysis.

Yes. Section 5 traces every refusal to the gate that produced it and the quantity that justified it, and one of the four numbered findings began as an error in the benchmark itself, with three further defects recorded in Section 7.

Justify construct validity.

Partly, and the gap is named. The benchmark refuses every reference agent for reasons the data makes explicit, the luck floor never beats a reference agent, and the synthetic witness of Section 5.11 shows the acceptance region is nonempty and locates its boundary. What it has not shown is a real agent

being admitted, so the claim that a real strategy can attain eligibility awaits the competing agent listed first in Section 7.

Statistical reporting. Every Sharpe in this paper is per period unless stated annualized, and Table 3 gives the conversion at every bar size. Every deflated Sharpe is reported beside its N . All runs use eight execution seeds and six windows; the per-cell record carries both. Costs are the typical profile of Section C. Block bootstrap, not i.i.d., throughout.

Licenses and maintenance. Both are in Section D, under Distribution and Maintenance. In summary: the code is MIT or Apache-2.0, but that license does not cover the observations. The three US index series require pre-approval, no Binance redistribution grant is recorded, and the current FRED service terms constrain the other series' use. This remains a distribution blocker. Historical dataset identities are frozen by hash and retired rather than edited; a permission-compatible replacement requires new identities and a new empirical run. Golden fixtures change only by a local command that refuses to run in continuous integration.

Compute. Hardware, wall clock, peak memory and the zero-API-cost claim are reported in Section 5: the full 4,608-cell grid regenerates in 2,078 seconds on one core, with no network, no key and no GPU.

Ethics. Section 10 states what an eligibility verdict does and does not assert, the dual-use exposure, the host's conflict of interest, and the release calculus. Section 4.4 states who may enter, that a published score cannot be retracted, the per-window variant cap, how a scoring defect is handled after publication, and that dispute adjudication is currently the operator's.

C Simulator, data, replay and contamination

C.1 Simulator and cost model

The data store returns rows at or before the decision index. Execution charges a per-trade fee, seeded slippage, own-order impact that scales with the square root of participation, and financing on gross exposure above one times net asset value. Fills can be capped at a fraction of participation with the remainder deferred. Four named cost profiles ship, and each is reported under the name used throughout this paper rather than under its Rust identifier: frictionless (`CostProfile::None`), typical (`CostProfile::Typical`), stressed (`CostProfile::WorstCase`) and realistic (`CostProfile::Realistic`). The typical profile is 2 basis points fee, 3 slippage, 50 impact and 5 financing; stressed is 10, 15, 150 and 20 with a participation cap of 10 percent and a two-bar delay; frictionless charges none of them. Realistic is typical plus seeded fill delay, partial fills and queue-position slippage, and is exercised by the seed leg rather than by any table here, so the sweeps in this paper cover the first three. All experiments outside Section 5.6 use the typical profile. The seeded slippage is what makes different execution seeds produce different runs and gives pass^k something to measure.

C.2 Data

Nine frozen datasets supplied the historical benchmark evidence, each with a SHA-256 sidecar and a deterministic fetch or derivation script. The source requests needed no API key, but that access fact did not grant redistribution permission; Section D records the unresolved rights blocker. Table 1 lists the exact historical bytes and date ranges. Before any agent is scored on a dataset, the dataset is scored: a battery of stylized facts after Cont [2001] tests excess kurtosis, volatility clustering, gain-loss asymmetry, aggregational Gaussianity and time-reversal asymmetry, and issues a verdict. Seven of the nine pass. Weekly US indices fail the aggregation test because 522 bars is too short for it, and FX majors fail time-reversal asymmetry, which is weak in a carry-free spot series. Both ship with the verdict recorded.

Two datasets carry caveats the scripts document. The commodities file keeps the WTI close of -36.98 on 20 April 2020 as published, which dominates its moments, so every commodities statistic in the paper inherits that outlier; a start date of May 2020 is the documented opt-out and removes it. The rates file's close column is a yield in percent, not a price. The simulator nevertheless treats that column as a price level, so its percentage changes are neither Treasury total returns nor the P&L of a duration-specified bond position. That row is a numerical stress case for the protocol on a one-symbol universe and must not support an economic performance claim: cross-sectional momentum is undefined there, and the luck floor requires the single-symbol randomization of Section 5.12 to be a control at all. Section 7 states the coverage gaps.

C.3 Replay and recompute

A captured trajectory contains the agent's raw decisions and a schema-v2 `TrajectoryContract`, never a return or self-reported statistic. The contract names the dataset and cost-model digests, engine version, exact ordered windows and seeds and, for command-line capture, the runner executable digest. Strict verification refuses an empty artifact, a missing or old contract, any identity mismatch, duplicate or out-of-range geometry, a run matrix other than the complete window-major Cartesian product, an incorrect number or order of decision steps, and an observation date different from the frozen bar at that step. A separate verifier then replays the decisions through the same engine against the frozen dataset and recomputes the score. Capture and replay share the engine path, and a test asserts that doubling every order in a captured trajectory recomputes to different returns. The default command is strict; `-allow-unbound-trajectory` is an explicit diagnostic escape hatch for legacy or cross-version regrading and cannot be mistaken for the normal verifier. An artifact therefore cannot substitute self-reported returns or silently replay against different recorded conditions. What it cannot do is prove that the committed strategy produced those decisions; the trajectory contract carries no entrant commitment, so that linkage remains separate from the commitment in Section 4.3.

C.4 Contamination

Two defenses ship. A held-out dataset can be sealed under a published commitment to its hash, so the benchmark can show later that the data a model was scored on is byte-identical to the data it committed to before scoring. As in Section 4.3, the commitment binds bytes and not chronology. A canary token with the standard training-corpus warning is embedded beside it, so post-hoc detection of a verbatim copy of the scenarios in a model's output is possible. The signal is positive-only and literal-bytes-only: a hit is evidence of contamination, and a miss is not evidence of its absence, because a paraphrased, compressed, encoded or model-internalized copy carries none of the protected bytes. Separately, a masked view of any dataset renames symbols and dates to opaque identifiers, which defeats an agent that pattern-matches memorized tickers.

C.5 The units correction, in detail

The fix, shipped in v0.3.0, leaves the per-period kernel untouched and gives the configuration a periods-per-year field so that σ_{trials} is stated annualized and converted once at the point of use. A test asserts the conversion is applied exactly once on the configured path and never on the measured path. Another asserts that an SPX-like series is eligible under an annualized prior of 0.1 at 252 periods per year and ineligible under the same prior at one period per year, which reproduces the old units exactly. A test that the same series clears the original 0.5 prior was not written, because it cannot: 0.5 annualized at 50 trials is a benchmark of 1.14 annualized, and the index sits below it at any track length. That is the correct verdict for that prior, and the benchmark now states it in units a reader can check.

Table 12: Provenance of the nine frozen datasets. Rows exclude the header. Every file carries a .sha256 sidecar. Eight are fetched from a pinned upstream window; the ninth, `us-indices-1w`, is derived from the committed daily file. Check mode compares regenerated and committed bytes, but the five terminal rows of `crypto-majors-1d` no longer match because they were captured from a still-open daily bar, as recorded in Section C.

Dataset	Upstream source	Bytes	Rows	Frozen
<code>us-indices-1d</code>	FRED SP500, DJIA, NASDAQCOM	195022	7539	2026-06-22
<code>us-indices-1w</code>	derived from <code>us-indices-1d.csv</code>	40524	1566	2026-08-23
<code>crypto-majors-1h</code>	Binance <code>/api/v3/klines</code>	4103983	120000	2026-08-23
<code>crypto-majors-4h</code>	Binance <code>/api/v3/klines</code>	1026006	30000	2026-08-23
<code>crypto-majors-1d</code>	Binance <code>/api/v3/klines</code>	141017	5000	2026-06-22
<code>crypto-majors-1w</code>	Binance <code>/api/v3/klines</code>	43091	1535	2026-08-23
<code>fx-majors-1d</code>	FRED DEXUSEU, DEXUSUK, DEXUSAL, DEXJPUS, DEXSZUS	602783	20785	2026-08-23
<code>commodities-1d</code>	FRED DCOILWTICO, DCOILBRENTU	199499	8252	2026-08-23
<code>rates-1d</code>	FRED DGS10	103943	4157	2026-08-23

D Datasheet for the SharpeBench datasets

Organized under the seven headings of Gebru et al. [2021]. Table 1 gives the shape of each dataset and its realism verdict; Section C gives the two data caveats and the contamination defenses. This appendix gives provenance, licensing, collection and maintenance, and it is the authoritative record for all four.

D.1 Motivation

The datasets exist to score trading agents under a deflation gate whose behavior depends on track length, bar size and return moments. Three requirements guided the original collection. First, no API key was required, so the collection did not depend on a subscription. Second, the frozen bytes were intended to ship with the repository so a historical score could be reproduced. A later rights audit found that public access did not establish redistribution permission: the three US index series require pre-approval, no Binance redistribution grant is recorded, and the current FRED terms restrict the benchmark’s software and archive use. That unresolved distribution blocker is separate from the reproducibility of the already-committed historical evidence. Third, the same underlying price history had to be available at several bar sizes, because the deflation benchmark is a function of periods per year and the first finding of this paper is a units error in exactly that conversion (Section 5.2). No funding was received for the collection. The collection was performed by the author.

D.2 Composition

Nine files: eight tradable-price datasets across cryptoassets, US equity indices, foreign exchange and commodities, plus one rates-yield stress series. The files use four bar sizes, but there is not one file per asset-class/bar-size pair: crypto has four, US indices two, and the other classes one each. Each instance is one bar: a date, a symbol and a close, in a long format sorted by date then symbol. There is no label, no split into training and test, and no personal data of any kind; every value is a published market observation or an official rate. Counts, in bytes and data rows excluding the header:

The nine files total 6,455,868 bytes and 198,834 data rows. They are not nine independent samples: the four crypto files resample one price history, the two US index files resample another, and WTI co-moves with Brent, so counting claims in Section 5 are phrased over roughly five independent market panels. Each file is complete in the sense that its date axis is the intersection of the dates on which every one of its symbols traded; instances dropped by that intersection are documented per file, and for commodities they are the 42 WTI and 79 Brent observations outside the common axis. Nothing is redacted and nothing is confidential. The rates file’s `close` column is a yield in percent and not a tradable price, which is the single most consequential composition caveat and is stated

wherever that row is used (Sections C and 7). Errors and noise inherited from upstream are kept as published, including the WTI close of -36.98 on 20 April 2020.

D.3 Collection process

Every source file was fetched over public HTTP with no authentication, by a committed script that pins its own date window. Public reachability is an access fact, not a redistribution grant. The FRED files came from the `fredgraph.csv` endpoint; the crypto files came from the Binance klines endpoint, paginated at 1,000 klines per request, with the kline close stamped at the kline open time in UTC. The weekly US index file is derived offline as a pure function of the daily file's bytes, taking the last close of each ISO week, and touches no network. FX quotes are the New York noon buying rates from the Federal Reserve H.10 release, with JPY and CHF inverted so that every close is the USD price of one unit of the base currency; nothing else is transformed. The FRED windows are capped at 2010-01-01 for comparability across the FRED files and at 2026-08-14, the last date published for all five FX series at freeze time.

The sampling is not random. The crypto universe is the five majors by capitalization at collection time and the index universe is three headline US indices, both selected with hindsight from instruments that survived into the collection period, so the fixed universes do not close survivorship bias (Section 7). Two intended sources were dropped and neither was substituted: single-name equities, because no keyless daily source was found, and gold, because FRED withdrew the daily LBMA series and no suitable keyless replacement was found. Neither was replaced from a licensed feed. No human subjects were involved, no crowdworkers were used and no ethical review process applies.

D.4 Preprocessing, cleaning and labeling

Preprocessing is limited to the date intersection, the JPY and CHF inversion, the weekly derivation, and fixed-decimal formatting with rows sorted by date then symbol and no timestamps written. No outlier is removed, no value is interpolated and no return is winsorized. The raw upstream responses are not archived in the repository. The producer scripts pin their windows and compare regenerated bytes with the committed file; `us-indices-1w` is reproduced from the committed daily input rather than fetched. The partial terminal bar in `crypto-majors-1d` is an explicit exception to source re-fetch identity: the upstream endpoint now returns completed bars for that date. A `.sha256` sidecar verifies the committed bytes; it does not prove that an upstream response remains unchanged.

Before any agent is scored on a dataset, the dataset is scored: the stylized-facts battery after Cont [2001] tests excess kurtosis, volatility clustering, gain-loss asymmetry, aggregational Gaussianity and time-reversal asymmetry, and issues a verdict that ships with the file. Seven of the nine pass. Weekly US indices fail aggregational Gaussianity because 522 bars is too short for the test, and FX majors fail time-reversal asymmetry. Both ship with the failing verdict recorded in Table 1, and Section 5.14 reports that the eligibility conclusions are unchanged when the two are excluded.

D.5 Uses

The datasets have been used only to score the agents reported in this paper. They are intended for evaluating trading agents under the protocol of Section 3, and for studying how the deflation gate behaves as a function of track length and bar size. Three uses they should not be put to. They are not a source of economic conclusions about any instrument: the universes are narrow, the observation is twenty trailing closes, and the execution model is stylized. They are not a bond dataset, for the reason above. And they are not a substitute for a survivorship-free universe in any study whose conclusion depends on delisted instruments. A user who reports commodities statistics without stating the retained negative WTI close will report moments dominated by one bar; the documented opt-out is a 2020-05-01 start date.

D.6 Distribution

The code is dual-licensed MIT or Apache-2.0; those licenses do not cover the third-party observations in these files. A series-by-series audit on 1 September 2026 found no recorded redistribution grant for the Binance observations. FRED labels the federal FX, oil and yield series “Public Domain: Citation Requested”, but says that category can include copyrighted material and remains subject to its service-wide prohibited uses. The SP500, DJIA and NASDAQCOM pages are copyrighted and marked pre-approval required; no approval is recorded here. FRED’s current legal notice also restricts use of its service or content in software-system development and, without consent, caching, archiving and provision to third parties. The public source distribution therefore has an unresolved rights blocker. Replacing or removing the affected inputs would change dataset identities and requires a new empirical run; it cannot retroactively change what the historical evidence measured. The repository’s `data/RIGHTS.md` records each series, the controlling notices and the available remediation paths.

D.7 Maintenance

The author maintains the datasets, and the contact address is the one on the title page. The datasets are versioned with the benchmark: one workspace version covers every crate and every registry package, a tag on the main branch publishes to all of them, and a verification job confirms each registry serves the tag.

A frozen file is never edited in place. Adding a dataset means adding four things together: a hash-pinned file, a committed producer with a validation mode, a realism verdict, and a periods-per-year value; a dataset with fewer than four is not admissible. The producer may fetch a pinned upstream window or derive from another hash-pinned input, and its validation result must record any non-reproducible upstream state such as a partial bar. Retiring one means marking it retired in the registry and leaving the bytes and the sidecar in place, so a published board that scored on it stays verifiable. Upstream withdrawal is the expected retirement trigger and it has already happened once: the daily LBMA gold series was withdrawn from FRED before collection, which is why gold is absent rather than stale. If a stylized-fact verdict changes, because the battery is corrected rather than because the data moves, the new verdict is recorded beside the old one and the boards scored under the old verdict are not rescored; the same supersession discipline applies as for a scoring defect (Section 4.4). Users who want to extend a dataset add a producer rather than appending to a frozen file, because appending changes the hash and therefore invalidates every board scored on it.

Defect reports and dataset proposals go through the repository’s public issue tracker, which is the feedback channel. There is no mailing list and no separate support address. Older versions remain retrievable through the registries and through the repository’s tags for as long as those services host them, which is a dependency on third parties that we do not control and do not guarantee.